透明ゼロ知識証明入門: Ligero・Aurora・Virgo・zk-STARK を比較して学ぶ

Ligero・Aurora・Virgo・zk-STARK の仕組み・性能・ユースケース比較

Ligero / Aurora / Virgo / zk-STARK

Ligero・Aurora・Virgo・zk-STARK を、数学的下準備・仕組み・具体例・性能比較・ユースケース比較・実装上の注意まで含めて、初学者と学部生向けに教科書的に解説する Zenn Book。

PDF 版

透明なゼロ知識証明を、共通基礎から 4 方式比較まで一冊で俯瞰するための教材

目次

はじめに：この本の狙いと読み方 1
本書の方針 1
この本で得られる見取り図 2
本書の読み方 3
4 方式を一文ずつで言うと 3
本書の構成 4
最後に 5

第 1 章ゼロ知識証明の基礎 6

第 2 章 有限体・多項式・Reed-Solomon 符号・Merkle 木 13

第 3 章 R1CS・layered circuit・IOP・sumcheck・GKR 22
算術回路とは何か 22
layered arithmetic circuit 23
R1CS とは何か 24
なぜ多項式が出てくるのか 25
Reed–Solomon 符号 26
関数の拡張：multilinear extension 26
IOP とは何か 27
sumcheck の直感 28
FRI の直感 29
GKR の直感 30
「structured circuit」とは何か 31
Aurora と Virgo の違いを先取りする 32
この章のまとめ 32

第 4 章 4 方式を同じ地図で眺める 33
4 方式を最短で比較する表 33

4 つの「自然言語」 34
どの幾何学で多項式を見るか 35
何を 1 本に圧縮しているのか 36
低次数性は「主役」か「支援役」か 37
共通部品は意外と多い 38
「どれが一番強いか」と聞かれたら 38
この章のまとめ 39

第 5 章 Ligero の全体像 40

第 6 章 Ligero の中身を分解する 47

第 7 章 Ligero を小さな回路で追う 57

第 8 章 Aurora の全体像 67
Aurora とは何か 67
Aurora が解きたい問題 67
Aurora の基本アイデア 68
なぜ R1CS から多項式へ行くのか 68
Aurora を構成する主要部品 69
Aurora の複雑性をどう読むか 70
Aurora の何が新しかったのか 71
Aurora の直感的な図式 71
Aurora が向く場面 72
Aurora と STARK の関係をどう見るか 72
Aurora を読むときの心の置き場 72
この章のまとめ 73

第 9 章 Aurora を手で追う 74
例題 74
Step 1: R1CS に落とす 74
Step 2: 証明者が大きな表を作る 75
Step 3: 制約違反を多項式恒等式へまとめる 75
Step 4: univariate sumcheck 的な圧縮 76
Step 5: でも “都合のよい表” を出されたら困る 77
Step 6: そこで FRI を使う 77
Step 7: オラクルは全部見せない。Merkle で固定する 78
Step 8: Fiat–Shamir で非対話化する 78

Step 9: ゼロ知識性はどう確保するのか 79
1 本の Aurora 証明に何が入っているか 79
例題に戻ると何が起きているのか 80
Aurora の理解でつまずきやすい点 80
Aurora を一行で再定義する 81
この章のまとめ 81

第 10 章 Virgo の全体像 82
Virgo とは何か 82
Virgo が解きたい問題 82
Virgo の backbone は GKR 83
でも GKR だけでは Virgo にならない 83
zkVPD とは何か 83
Virgo の大まかな流れ 84
structured circuit とは何か 85
Virgo の複雑性をどう理解するか 86
Virgo の concrete result 87
Virgo の新しさを一言でいうと 87
Virgo が向く場面 87
Virgo のトレードオフ 88
Aurora と見比べたときの第一印象 88
Virgo を読むときの心の置き場 89
この章のまとめ 89

第 11 章 Virgo を手で追う 90
例題となる layered circuit 90
Step 1: 各 layer の値を表にする 91
Step 2: その表を multilinear extension に持ち上げる 91
Step 3: 検証したい主張を 1 個に圧縮する 92
Step 4: GKR が layer 0 の主張を layer 1 へ落とす 92
Step 5: sumcheck による圧縮のイメージ 93
Step 6: 同じことを layer 1 から入力 layer へ繰り返す 93
Step 7: では $\tilde{V}_i(r)$ は誰がどう計算するのか 94
Step 8: zkVPD の役割を直感で理解する 94
Step 9: structured circuit が効く場所 95
Step 10: 非対話化とゼロ知識化 95

この例で verifier が本当に納得するもの 96
Virgo の理解でつまずきやすい点 96
Virgo を一行で再定義する 97
この章のまとめ 97

第 12 章 zk-STARK の全体像 98

第 13 章 AIR：計算を状態遷移として書く 105

第 14 章 FRI・ALI・Composition による制約圧縮 114

第 15 章 zk-STARK をフィボナッチで追う 123

第 16 章性能比較：理論量・具体感・トレードオフ 130
まず理論量の表を置く 130
原論文の具体値で見る 131
Proof size をどう読むか 132
Verifier をどう読むか 133
Prover をどう読むか 134
メモリと定数因子は複雑性式より重要なことがある 135
方式別に一言で言うと 135
比較を誤らないためのチェックリスト 136
この章のまとめ 136

第 17 章ユースケース比較：どの問題にどの方式が自然か 137
まず結論の表 137
ユースケース 1: 秘密を隠したまま「条件成立」だけ示したい 138
ユースケース 2: 一般計算を R1CS-first で扱いたい 138
ユースケース 3: Merkle tree や規則的回路を軽い verifier で見たい 139
ユースケース 4: 長い逐次計算、VM、ロールアップ 140
ユースケース 5: 多数インスタンスの一括証明 141
ユースケース 6: transparent で短い proof が欲しい 141
ユースケース 7: verifier をとにかく軽くしたい 141
ユースケース 8: 研究・教育で設計思想を学びたい 142
方式選択の実務的な順序 143
4 方式を使い分ける一言まとめ 143
この章のまとめ 144

第 18 章実装・エコシステム・研究上の注意点 145

まず 4 方式の現在位置を一表にする 145
Ligero: どこまでを期待すべきか 146
Aurora: 何が今でも重要か 147
Virgo: 2025 年以降は必ず後続研究まで読む 147
zk-STARK: 現在の実務で最も厚いエコシステム 148
FRI 系の Fiat-Shamir 安全性は自明ではない 149
transparent / lightweight は「楽に実装できる」を意味しない 150
field choice は最後の微調整ではない 150
range check と bit decomposition を甘く見ない 151
メモリは計算時間と同じくらい重要 151
本書としての実務的助言 152
この章のまとめ 152

第 19 章 演習問題 154
A. ゼロ知識証明の基本 154
B. 有限体・多項式・符号 155
C. R1CS・layered circuit・GKR 155
D. Ligero 156
E. Aurora 157
F. Virgo 157
G. zk-STARK 158
H. 比較問題 159
I. 設計・研究の視点 159
J. 発展問題 160
学習の目安 161

第 20 章参考文献と学習ロードマップ 162
最重要文献 162
背景文献 163
後続研究・注意喚起 164
実装・公式資料(2026-04-05 閲覧) 166
どの順に読むとよいか 168
原論文を読むときのコツ 169
この章のまとめ 170

付録 用語集 171

はじめに：この本の狙いと読み方

本書は、**Ligero・Aurora・Virgo・zk-STARK** という 4 つの代表的な透明系ゼロ知識証明を、**初学者から学部生でも追える教科書のかたち**でまとめ直したものです。

4 方式を一冊で扱う理由は単純です。
これらはどれも「trusted setup を避けたい」「ハッシュ関数・符号理論・多項式検査を中心にしたい」「巨大な計算を少数の検査に圧縮したい」という方向を共有しながら、**何を "自然な証明対象" と見るか**がはっきり違うからです。

- **Ligero** は、回路の満足性を **行列 + 符号 + spot-check** に落とす系統です。
- **Aurora** は、R1CS を **Reed-Solomon 符号語 + FRI** の世界へ埋め込む系統です。
- **Virgo** は、layered arithmetic circuit を **GKR + zkVPD** で圧縮する系統です。
- **zk-STARK** は、計算全体を **execution trace + AIR + FRI** として見る系統です。

同じ「透明な zk 証明」でも、見ている幾何学はかなり違います。
本書の最大の狙いは、その違いを「単なるブランド名の違い」ではなく、**表現・多項式・検査戦略の違い**として理解できるようにすることです。

本書の方針

本書では、次の 3 点を重視します。

1. まず直感を作る

いきなり定義や補題に飛ばず、

- 何を圧縮したいのか
- なぜ少数点検査でよいのか
- 低次数性がどこで効くのか

を先に掴みます。

2. 数学的な足場を省略しない

透明系 zk 証明では、

- 有限体
- 多項式補間
- Reed-Solomon 符号
- Merkle 木
- IOP / IOPP
- sumcheck
- GKR
- multilinear extension
- AIR / FRI

がほぼ必ず現れます。
これらを飛ばすと、個々の方式の説明が暗記になってしまいます。そこで本書では、必要な範囲の数学と証明モデルを独立に整理します。

3. 事実と解釈を分ける

本書では、できるだけ次を区別します。

- **原論文が明示的に主張している複雑性や設計**
- **公式実装・公式ドキュメントから確認できる事実**
- **本書としての比較・設計判断・ユースケース整理**

特に後半の比較章と実装章では、この区別を意識して記述します。

この本で得られる見取り図

読み終えたときに、少なくとも次を説明できるようになることを目標にします。

- なぜ Ligero では「行列の縦横バランス」から平方根型の通信が現れるのか。
- なぜ Aurora では R1CS を低次数コード語へ埋め込む発想が自然なのか。
- なぜ Virgo では layered circuit と GKR が中心になるのか。
- なぜ zk-STARK では execution trace と AIR が主役なのか。
- 4 方式は性能・証明サイズ・検証量・向くユースケースの観点でどう違うのか。
- それでもなお、4 方式がどこで共通しているのか。

本書の読み方

ゼロ知識証明が初めての人

次の順がおすすめです。

1. 第 1 章「ゼロ知識証明の基礎」
2. 第 2 章・第 3 章「数学と証明モデルの下準備」
3. 第 4 章「4 方式を同じ地図で眺める」
4. その後、Ligero → Aurora → Virgo → zk-STARK の順に読む

すでに暗号学や計算理論に慣れている人

先に第 4 章・第 16 章・第 17 章を読むと、比較軸が先に手に入ります。その後で個別方式に戻ると整理しやすくなります。

実装や研究テーマの視点が欲しい人

第 18 章を早めに読んでください。
特に、

- Virgo の後続研究による再検討
- FRI 系の Fiat-Shamir 後の安全性
- 現行の STARK 系エコシステム

を知っておくと、「論文として学ぶ」ことと「実際に採用する」ことの差が見えます。

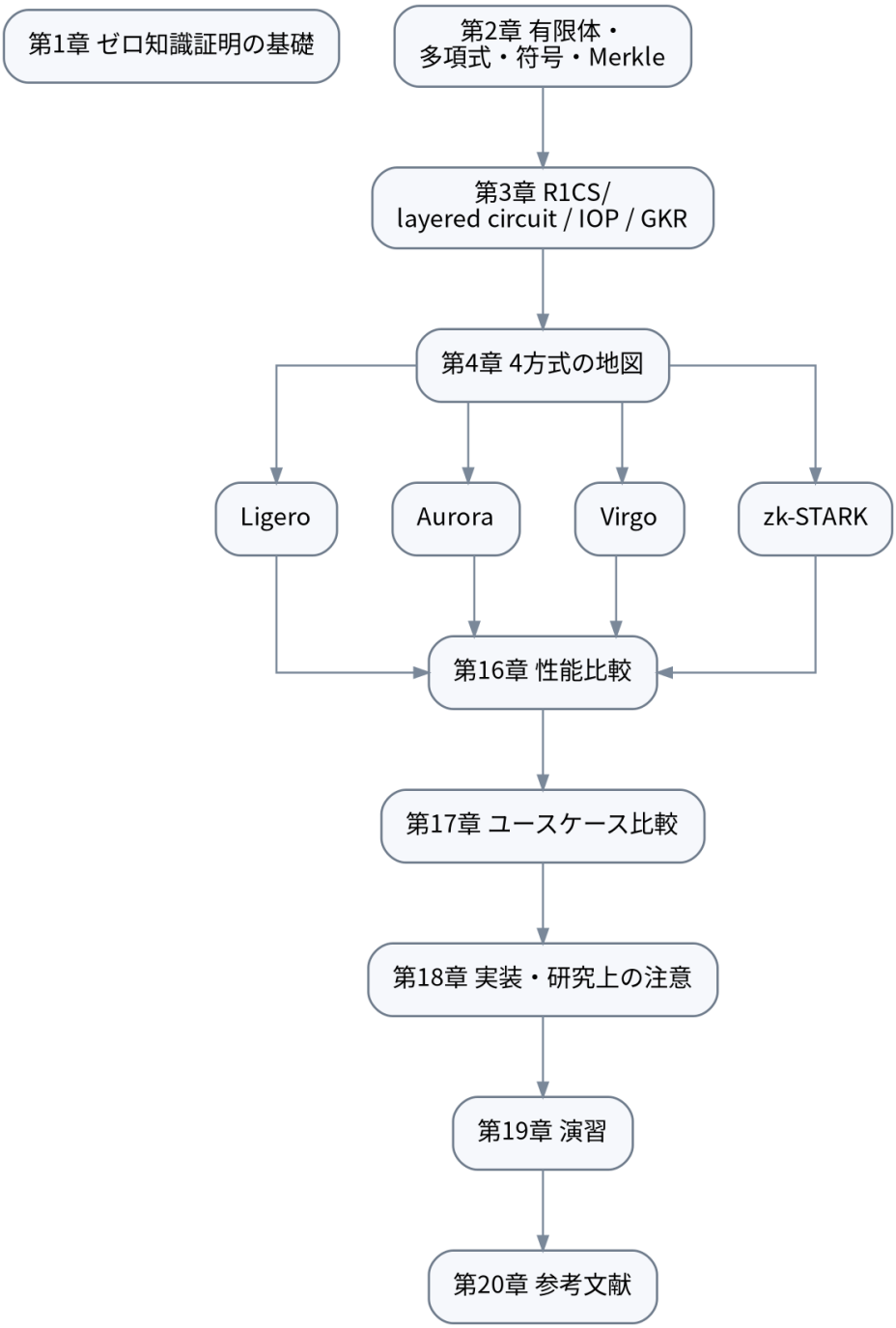
4 方式を一文ずつで言うと

まずは極端に単純化した一文を置いておきます。

- **Ligero**: 回路の大量の値を行列に並べ、符号化し、ランダム線形結合と列開示で検査する。
- **Aurora**: R1CS の satisfiability を、低次数オラクルの整合性と FRI へ翻訳する。
- **Virgo**: layered circuit の正しさを GKR で縮約し、評価主張を zkVPD で短く支える。
- **zk-STARK**: 長い計算履歴を execution trace と見て、AIR と FRI でその正しさを示す。

本書全体は、この 4 文を少しずつ厳密にしていく作業だと思ってください。

本書の構成



本書全体の学習導線

最後に

本書は、透明系 zk 証明を「4 本の別々の論文」としてではなく、**同じ地形上の異なる設計点**として読むための本です。
どれが絶対に優れているかを決めるのではなく、

- どの表現が自然か
- どの低次数性を検査しているのか
- どこでランダム線形結合を使っているか
- verifier / prover / proof size のどれを優先しているか

を見抜けるようになることが重要です。

では、第 1 章から始めましょう。

第 1 章ゼロ知識証明の基礎

ゼロ知識証明は、一言で言えば

> **「秘密そのものを出さずに、秘密を知っていること、あるいは秘密が条件を満たすことだけを示す」**

ための仕組みです。

ただし、この一言では大事な点が隠れています。
「何を誰に示すのか」「どの程度まで騙せないのか」「なぜ秘密が漏れないのか」を、順に整理しましょう。

1.1 statement と witness

ゼロ知識証明では、普通は次の形の問題を考えます。

- 公開される主張：`statement` x
- 秘密の補助情報：`witness` w

そして「関係」$R(x, w)$ が定義されていて、

$$R(x, w) = 1$$

なら、「w は x の正しい witness である」と言います。

例 1：ハッシュ前像
公開された値 h に対し、

- statement: $x = h$
- witness: $w = m$
- relation: $R(h, m) = 1 \iff H(m) = h$

です。

ここで示したいことは「**私は m を知っている**」であって、「m 自体を公開する」ではありません。

例 2：数独

完成済みの数独の盤面を witness とし、

- 与えられた数字は変えていない
- 各行・各列・各 3×3 ブロックに 1〜9 が一回ずつ現れる

という relation を満たすことを示すこともできます。
このとき、解答そのものは秘密にしたいかもしれません。

例 3：ブロックチェーンの状態更新

L2 システムでは、

- statement:「この新しい状態ルートは正しい」
- witness: 実際の取引列、各アカウントの更新過程、中間状態

のように考えます。
ユーザやバリデータは中間計算を全部やり直さずに、最終更新が正しいと納得したいわけです。

1.2 三つの基本性質

ゼロ知識証明を理解するうえで最も重要なのは、**完全性・健全性・零知識性**の三つです。

完全性（completeness）

主張が本当に正しく、正直な証明者が正しい witness を持っているなら、検証者は受理する。

直観的には、

> **本当のことを言っている人が不当に落とされない**

という性質です。

健全性（soundness）

主張が偽であるなら、不正な証明者がどれほど工夫しても、検証者を騙して受理させる確率は小さい。

つまり、

> **嘘つきが通ってしまう確率を十分小さく抑える**

という性質です。

ゼロでなく「小さい」と言うのは、現代の多くの証明系が確率的だからです。
たとえば健全性誤差が 2^{-80} なら、偽の主張が受理される確率は高々 2^{-80} とみなします。

零知識性（zero-knowledge）
検証者は、主張が真であること以外の有益な情報を得ない。

ここでの厳密な定式化は、「**シミュレータ**」という仮想的なアルゴリズムが存在し、真のやりとりを見たのと区別できない transcript を生成できる、というものです。

直観的には、

> **見た目には会話があったが、実は witness を知らなくても、その会話らしいものを作れてしまう**

なら、会話そのものから新しい秘密情報は得られていない、という考えです。

1.3 proof と argument の違い

日常語ではどちらも「証明」と言いたくなりますが、暗号では区別が重要です。

Proof
どれほど計算能力の大きい相手に対しても健全性が成り立つもの。
つまり、**情報理論的**あるいは**無条件**に近い強さを持つことがあります。

Argument
健全性が、ある計算問題の難しさに依存するもの。
たとえばハッシュ関数の衝突耐性や離散対数問題の難しさに依存します。

現代の実用的なゼロ知識システムの多くは、厳密には **argument** です。
Ligero・Aurora・Virgo・zk-STARK も、実用形では通常 argument として理解するのが自然です。

1.4 対話型と非対話型

対話型（interactive）
証明者 P と検証者 V が、複数回メッセージをやり取りします。

- V がランダムなチャレンジを送る
- P がそれに応じて応答する

- これを何ラウンドか繰り返す

という形です。

非対話型（non-interactive）
証明者が**一つの証明文字列**を作り、検証者がそれを単独で検査します。

ブロックチェーンのように「後から誰でも検証したい」場面では非対話型が特に便利です。

Fiat–Shamir **変換**
多くの**公開コイン**対話型プロトコルでは、検証者のランダムチャレンジをハッシュ関数で置き換えることで、非対話化できます。これを **Fiat–Shamir 変換** と呼びます。

雑に言えば、

1. 証明者はこれまでの transcript をハッシュする
2. ハッシュ値を「検証者が送ったはずの乱数」とみなす
3. その乱数に対する応答を続ける

という手順です。

この変換は非常に重要ですが、厳密には**ランダムオラクルモデル**などの仮定のもとで議論されることが多いです。

1.5 公開コインと透明性

公開コイン（public-coin）
検証者のメッセージが「ただのランダム文字列」であるタイプのプロトコルです。
検証者が難しい秘密計算をする必要がありません。

Ligero・Aurora・Virgo・zk-STARK のような透明系では、この公開コイン性が特に重要です。
なぜなら、

- 実装が単純になる
- Fiat–Shamir に乗せやすい
- trusted setup を避けやすい

からです。

透明性（transparent）
STARK の文脈で特に強調される言葉で、ざっくり言えば

秘密の共通参照文字列や trapdoor を必要としない

という意味です。

pairing-based SNARK の多くでは、最初に「安全に作られた設定」が必要でした。
これが漏れると偽証が可能になることがあります。
一方、transparent な系では、公開乱数やハッシュ関数を中心に構成するため、その種の setup 依存を避けられます。

1.6 PCP, IOP, IOPP を最小限だけ

Ligero・Aurora・zk-STARK を読むと PCP や IOP が、Virgo を読むと GKR や多項式評価委任が前面に出てきます。

PCP（Probabilistically Checkable Proof）
検証者が証明全体を読まず、**ランダムに少数箇所だけ読んで**高い確率で正しさを判定できる証明の概念です。

IOP（Interactive Oracle Proof）
証明者が大きなオラクル（長い文字列）を提示し、検証者がそこへランダムアクセスしながら、さらに対話も行うモデルです。

IOPP（Interactive Oracle Proof of Proximity）
ある長い文字列が、ある符号や集合に「属している」あるいは「近い」ことを検査するための IOP です。

STARK の FRI は、この IOPP の文脈で理解すると見通しが良くなります。
Ligero・Aurora・zk-STARK は、巨大な構造へランダムアクセスしながら矛盾を探すという意味で、この世界観に強く属します。

1.7 なぜ「全部見ない」のに信用できるのか

初学者が最も感じる違和感はここでしょう。

理由 1：嘘をつくと、多くの場所で矛盾するように設計されている
たとえば低次数多項式であるはずの表が、実はバラバラな値の寄せ集めなら、ランダムに選んだ少数点でも不整合に当たりやすくなります。

理由 2：ランダム線形結合で、多数の制約を一つに圧縮できる

100 個の制約をそのまま全部見る代わりに、ランダム係数で混ぜた一つの制約を見ます。

もし元の制約群のどれかが壊れていれば、混ぜた結果も高確率で壊れます。

理由 3：コミットメントで後出し変更を防げる

証明者は最初に長い表やオラクルにコミットし、その後でランダム位置を開きます。

Merkle 木は、その「後出しの改ざん防止」に使われます。

この三つの組み合わせは、程度の差こそあれ Ligero・Aurora・Virgo・zk-STARK を理解する共通骨格です。

1.8 典型的な誤解

誤解 1：「零知識なら何も情報を出してはいけない」

実際には、「**証明対象が真である**」という事実は当然伝わります。

隠したいのは witness の中身や、そこから得られる追加情報です。

誤解 2：「健全性誤差があるなら危険」

健全性誤差は暗号で普通に使われる概念です。

重要なのは、その値を十分小さくできることです。

誤解 3：「透明なら自動的に軽い」

透明性と軽さは別問題です。

transparent でも証明サイズが大きいことはあります。

実際、STARK は透明ですが、pairing-based SNARK に比べると証明が大きいことが多いです。

誤解 4：「ゼロ知識証明は秘密の開示ゼロなので、計算の痕跡も不要」

実際には、証明者はかなり複雑な内部計算をします。

秘密を出さないために、むしろ**余分な符号化・ブラインディング・ハッシュ**が必要になることも多いです。

1.9 小さな例：パスワードを見せずに知っていることを示す

公開値として $h = H(\text{password})$ が登録されているとします。

ログイン時にサーバへパスワードそのものを送る代わりに、

「私は $H(w) = h$ を満たす w を知っている」

とゼロ知識で示せれば、平文パスワードは渡さずに済みます。

もちろん、実際に安全なプロトコルにするには、再利用攻撃や辞書攻撃、セッション束縛など多くの注意が必要です。
しかし「秘密を送らずに条件成立だけを示す」というゼロ知識の感覚は、この例で掴みやすいです。

1.10 この章のまとめ

- ゼロ知識証明では、公開主張 x と秘密 witness w の関係 $R(x, w)$ を扱う。
- 核となる性質は **完全性・健全性・零知識性**の三つである。
- 実用系では proof より argument であることが多い。
- Ligero・Aurora・Virgo・zk-STARK を理解するには、**公開コイン**、**Fiat–Shamir**、**オラクル/ ランダム点評価**の感覚が重要である。
- 次章では、これらを支える数学的道具——有限体、多項式、Reed–Solomon 符号、Merkle 木——を準備する。

練習問題

1. ハッシュ前像の例で、statement と witness を自分の言葉で書いてください。
2. 「零知識なのに、主張が真だという事実は伝わる」のはなぜ矛盾しないか、説明してください。
3. proof と argument の違いが、どのような安全性の前提の違いに対応するか述べてください。
4. Fiat–Shamir 変換が便利な理由を、ブロックチェーンの利用例を意識して説明してください。

第 2 章 有限体・多項式・Reed-Solomon 符号・Merkle 木

この章では、Ligero・Aurora・Virgo・zk-STARK を読むために必要な道具を、必要最小限に絞って導入します。
本格的な代数の教科書ではありません。目的は、後の章で出てくる

- 有限体
- 多項式
- Reed–Solomon 符号
- 低次数性
- Merkle 木
- オラクルとランダム照会

を「読める」状態にすることです。

2.1 有限体とは何か

まずは $\mathbb{F}_p$ だけでよい
最初は、素数 p に対して

$$\mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

を「**p で割った余りの世界**」だと思えば十分です。
足し算も掛け算も、最後に p で割った余りを取ります。

たとえば $\mathbb{F}_{17}$ では、

- $15 + 5 = 20 \equiv 3 \pmod{17}$
- $4 \cdot 5 = 20 \equiv 3 \pmod{17}$

です。

なぜ有限体が必要なのか

ゼロ知識証明では、ランダム係数を選んだり、多項式を評価したり、符号語を作ったりします。
そのとき、

- 四則演算がきれいに閉じる
- ゼロでない元には逆元がある
- 多項式の性質が扱いやすい

という理由で、有限体が自然な舞台になります。

STARK では $\mathbb{F}_{2^m}$ もよく出る

STARK 系の文献では、$\mathbb{F}_{2^{64}}$ のような**拡大体**が現れます。
初学者は最初からこれを完全に理解しなくてよいです。
「0/1 のビット列に近い表現を持ちつつ、体として振る舞う大きな有限集合」くらいに見ておけば十分です。

本書の例は多くを $\mathbb{F}_{17}$ や小さな整数例で説明します。

2.2 多項式は "構造を持った関数" である

多項式

$$p(X) = a_0 + a_1 X + a_2 X^2 + \cdots + a_d X^d$$

は、単なる関数ではありません。
次数が低いというだけで、値の並び方に強い制約がかかります。

重要事実 1：次数 d の多項式は $d+1$ 点で決まる

たとえば一次式 $a + bX$ は 2 点、二次式は 3 点の値が分かれば一意に決まります。

これは「少数の情報から関数全体が決まる」という意味で、証明系にとって非常に便利です。

重要事実 2：異なる低次数多項式は、そんなに多くの点で一致できない

もし p と q が次数 $\leq d$ の異なる多項式なら、
$p(X) - q(X)$ は零多項式ではないので、根の数は高々 d 個です。

つまり、

異なる低次数多項式は、たまたま少数点で一致しても、多数点で一致し続けることはできない

のです。

この事実が、ランダム点検査の安全性を支えます。

2.3 補間：値の表から多項式を戻す

有限体上でも、実数と同じように補間ができます。
たとえば $\mathbb{F}_{17}$ で、点

- $p(1) = 2$
- $p(2) = 3$

を満たす一次多項式を探すと、

$$p(X) = 1 + X$$

でよいことが分かります。

なぜなら、

- $p(1) = 1 + 1 = 2$
- $p(2) = 1 + 2 = 3$

だからです。

Ligero では、少数個のメッセージ値から多項式を作り、それを多数点で評価して**符号語**にします。
STARK では、実行トレースの列を多項式として補間し、さらに大きな領域で評価します。

2.4 Reed–Solomon 符号の直観

Reed–Solomon（RS）符号は、ざっくり言えば

短いメッセージを、多項式の多数点評価として冗長に並べたもの

です。

基本形

- 低次数多項式 $p(X)$ を選ぶ
- あらかじめ決めた点 $\eta_1, \eta_2, \ldots, \eta_n$ で評価する

- 得られた値列
$$(p(\eta_1), p(\eta_2), \dots, p(\eta_n))$$
を符号語とする

という仕組みです。

小さな例

$\mathbb{F}_{17}$ で

$$p(X) = 1 + X$$

を考え、評価点を $0, 1, 2, 3$ とすると、

$$(1, 2, 3, 4)$$

が符号語です。

この列はただの 4 つ組ではありません。
「**一つの一次多項式から来ている**」という構造を持っています。

なぜ冗長化するのか

元のメッセージが 2 点分の情報しかなくても、4 点や 8 点に広げておけば、

- 一部が壊れていても検出しやすい
- ランダムに数点見るだけで不正を見つけやすい
- Merkle 木でコミットしやすい

といった利点があります。

2.5 “近い” とは何か：距離の考え方

符号理論では、二つの長い列がどれくらい違うかを、しばしば**ハミング距離**で測ります。

ハミング距離

二つの同長ベクトル u, v に対し、違う座標の個数を数えるものです。

たとえば

$$u = (1, 2, 3, 4), \quad v = (1, 9, 3, 8)$$

なら、違うのは第 2 成分と第 4 成分なので距離は 2 です。

“コードに属する” と “コードに近い”

STARK の FRI では、「この長い表は本当に低次数多項式の評価列か？」を直接判定する代わりに、

- コードに属するか
- あるいは、コードから十分遠いか

を見分ける**近接性検査（proximity testing）**を行います。

この “近い” という考え方が、IOPP の中心にあります。

2.6 低次数性がなぜ嬉しいのか

次の一文が非常に重要です。

> **低次数性は、巨大な表に “圧縮された一貫性” を与える。**

たとえば、長さ 1024 の表が「本当に次数 15 の多項式から来ている」と分かれば、その表は 16 個程度の係数で記述できる強い構造を持ちます。

このおかげで、

- 表全体を見なくても
- ランダムな数点と
- いくつかの整合性検査だけで

不正を炙り出せます。

Ligero は「各行が低次数多項式から来る」ことを使います。
zk-STARK は「トレース列や制約多項式が低次数である」ことを使います。

2.7 ランダム点検査の基本原理

低次数多項式の強さを、最も単純な形で言うとこうです。

命題（直観版）

異なる二つの次数 $\leq d$ の多項式 p, q を、大きな集合 S 上で比べるとする。
ランダムな $r \in S$ を選んで $p(r) = q(r)$ が成り立つ確率は、おおよそ $d/|S|$ 以下です。

つまり、評価集合 S が十分大きければ、ランダム一点を見るだけでもかなりの検査力があります。

ゼロ知識証明とのつながり

もちろん実際のプロトコルでは、比較したいものが単純な二つの多項式ではなく、

- 行列の各行
- トレースの各列
- 制約をまとめた composition polynomial

だったりします。
しかし本質は同じで、「**低次数なはずなのにそうでない**」というズレをランダム検査で見つけます。

2.8 Merkle 木：大きな表へのコミットメント

Ligero でも STARK でも、証明者は巨大な表やオラクルを用意します。
しかし検証者はその全体を受け取りません。そこで必要になるのが**コミットメント**です。

Merkle 木は、長いデータ列 $(a_1, \dots, a_n)$ に対し、

1. 各要素をハッシュして葉にする
2. 隣り合うハッシュをさらにハッシュして親を作る
3. これを根まで繰り返す

という二分木です。

最終的な root hash だけを公開すれば、後から任意の要素 a_i を開示するときに、

- その値
- 兄弟ノードのハッシュ列(authentication path)

を示すだけで、その値が最初の root に整合していることを検証できます。

なぜ便利なのか

- **バインディング**：後から都合よく値を差し替えにくい
- **部分開示**：必要な位置だけ見せればよい
- **公開検証**：誰でも root とパスから確認できる

STARK の「大きな評価表にコミットして、あとでランダム位置を開く」という流れは、Merkle 木を抜きに理解できません。

2.9 オラクルモデルとは何か

PCP/IOP の文脈で「オラクル」というと、しばしば

長い文字列にランダムアクセスできるブラックボックス

のことです。

検証者は、その全部を読む代わりに、必要な位置だけ問い合わせます。
たとえば「表の 57 番目と 913 番目を見せてください」と言うわけです。

実際の実装では、これは

- Merkle root にコミットされた大きな配列
- 指定位置の値と認証パスの返答

として実現されます。

2.10 低次数拡張（Low-Degree Extension; LDE）

STARK を理解するうえで重要な語が **LDE** です。

もともと短い領域 H の上に定義された列を、

1. それを補間して多項式 $p(X)$ とみなす
2. より大きな領域 L 上で p を評価する

ことで、より長い評価列に拡張します。

なぜ拡張するのか
拡張すると冗長性が増え、低次数性の検査がしやすくなります。
また、制約の一部を別の点で評価する余地ができ、ランダム検査がやりやすくなります。

STARK の実装では、この LDE が中心的です。
Ligero でも「少数のメッセージ値を多数点評価にする」という意味で、近い発想が見られます。

2.11 Interleaving の直観

Ligero で重要なのが **interleaved Reed–Solomon code** です。
これは難しそうに見えますが、直観は簡単です。

- 1 本の RS 符号語を使う代わりに

- **複数本の RS 符号語を行として並べた行列**を考える

だけです。

たとえば 3 行あるなら、

$$U = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}$$

で、各 u_i がそれぞれ RS 符号語です。

この表現が便利なのは、**行方向のランダム線形結合**を取ると、また 1 本の RS 符号語になるからです。
Ligero は、この性質を使って「多くの行をまとめて一度に検査」します。

2.12 ここまでの道具がどう使われるか

この章の道具が、後でどう現れるかを先にまとめておきます。

道具	Ligero	Aurora	Virgo	zk-STARK
有限体	回路値・乱数・符号化の舞台	R1CS と低次数オラクルの舞台	layer 値表と multilinear extension の舞台	トレース、AIR、FRI の舞台
多項式	行の符号化	R1CS 整合性の代数化	hypercube 上の関数拡張	トレース列・制約・低次数検査
Reed-Solomon 符号	行列の各行を冗長化	低次数オラクルの自然な器	直接の主役ではない	FRI の低次数コードの中心
ハミング距離 / 近接性	符号語からのズレの評価	低次数オラクルの正当性	点評価主張の背景直観	近接性検査の基本概念
Merkle 木	非対話化・公開検証の支え	オラクル群へのコミットメント	多項式委任や transcript の固定	巨大オラクルへのコミットメント
ランダム照会 / ランダム点	少数列だけ開く	少数点でオラクル整合性を確認	sumcheck / GKR / zkVPD で主張を絞る	少数位置だけ開示し FRI を回す

ここでは Reed-Solomon 符号や LDE の説明がやや Ligero / Aurora / zk-STARK 寄りに見えるかもしれません。
Virgo では別の幾何学、すなわち **multilinear extension と Boolean hypercube** が強く効くため、その違いは次章で補います。

2.13 この章のまとめ

- 有限体は、ゼロ知識証明の代数的な舞台である。
- 低次数多項式は、少数の情報で強い構造を持つ。
- Reed–Solomon 符号は、多項式の評価列として自然に作られる。
- ランダム点検査が効くのは、異なる低次数多項式は多くの点で一致できないからである。
- Merkle 木は、大きな表にコミットしつつ一部だけ開示するための道具である。
- Ligero・Aurora・Virgo・zk-STARK は、これらの道具を異なる流儀で組み合わせている。

練習問題

1. $\mathbb{F}_{17}$ で $p(X) = 3 + 2X$ を $X = 0, 1, 2, 3$ で評価してください。
2. なぜ次数 d の多項式は $d + 1$ 点で決まるのか、直観的に説明してください。
3. Reed–Solomon 符号が「冗長化」になっている理由を述べてください。
4. Merkle 木で root だけを先に公開することが、なぜ後出し変更の防止につながるのか説明してください。
5. interleaving を「複数の RS 符号語を縦に積むこと」と見たとき、ランダム線形結合がなぜ重要か考えてみてください。

第 3 章 R1CS・layered circuit・IOP・sumcheck・GKR

この章では、特に Aurora と Virgo で決定的に重要な基礎をまとめつつ、4 方式全体を読むための共通語彙も整えます。ここを押さえると、後の章で出てくる

- R1CS
- layered circuit
- Reed–Solomon 符号
- IOP
- sumcheck
- FRI
- GKR

が、ばらばらの専門用語ではなく、1 本の設計線上に並んで見えるようになります。

算術回路とは何か

算術回路は、有限体 $\mathbb{F}$ 上での計算を表す DAG（有向非巡回グラフ）です。各ノードは

- 加算
- 乗算
- 定数
- 入力

のいずれかです。

たとえば、

$$z = xy + x + 2$$

という式は、次のように分解できます。

1. $a = x \cdot y$
2. $b = a + x$
3. $z = b + 2$

ゲート単位で見ると、各条件は局所的です。
この「局所的な制約」を大量に並べると、大きな計算全体を表せます。

layered arithmetic circuit

Virgo が特に好むのは **layered arithmetic circuit** です。
これは、回路を層ごとに並べて、各層のゲートが次の層のゲートからのみ入力を受ける形に整えたものです。

イメージは次の通りです。

- 最下層: 入力
- 中間層: 加算ゲート・乗算ゲート
- 最上層: 出力

layered にすると、計算の正しさを「上の層から下の層へ」順にたどるプロトコルを設計しやすくなります。GKR はまさにこの形を利用します。

なぜ layer 化がうれしいのか

layer 化の利点は、「ある層の値が正しいかどうか」を、その一つ下の層の値に帰着できる点です。
つまり、

- layer i の主張
- $\Downarrow$
- layer $i+1$ の主張
- $\Downarrow$
- ...
- $\Downarrow$
- 最後は入力の主張

という形で、大きな計算を段階的に縮約できます。

R1CS とは何か

Aurora の中心表現は **R1CS (Rank-1 Constraint Satisfaction)** です。

R1CS では、各制約を

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$$

という形で書きます。ここで

- z: 定数 1、公開入力、秘密入力、中間値を並べたベクトル
- A_i, B_i, C_i: 線形形式を表す係数ベクトル

です。

例：$z = xy + x + 2$ を R1CS にする

変数ベクトルを

$$\mathbf{z} = (1, x, y, a, z)$$

としましょう。ここで $a = xy$ です。

すると、次の 2 制約で十分です。

制約 1

$$x \cdot y = a$$

これは

$$\langle A_1, \mathbf{z}\rangle = x, \quad \langle B_1, \mathbf{z}\rangle = y, \quad \langle C_1, \mathbf{z}\rangle = a$$

に対応します。

制約 2

$$(a + x + 2) \cdot 1 = z$$

これは

$$\langle A_2, \mathbf{z} \rangle = a + x + 2, \quad \langle B_2, \mathbf{z} \rangle = 1, \quad \langle C_2, \mathbf{z} \rangle = z$$

に対応します。

このように、回路の局所条件を「線形形式どうしの積 = 線形形式」という形にまとめるのが R1CS です。

なぜ R1CS が便利なのか

R1CS の良さは、制約が一様な形になることです。
どんな複雑な回路でも、最終的には

- 線形結合を作る
- それどうしを掛ける
- 右辺の線形結合と一致させる

という同じ型の制約に落ちます。Aurora はこの一様性を、多項式恒等式の形へ変換して扱います。

なぜ多項式が出てくるのか

ここは Aurora / Virgo の核心です。

大雑把に言うと、**巨大な表を、そのまま扱うのは重い。だから多項式の評価表だと思って扱う**のです。

多項式には強い「剛性」があります。
次数 d の多項式は、$d+1$ 個より多くの点で一致すれば、もはや同じ多項式です。
この剛性のおかげで、

- 本当に低次数なものか
- 一部だけ都合よく改ざんされていないか

を、少数のランダム検査で高確率に検出できます。

低次数性の直感

たとえば 1 次多項式 $f(X) = 3X + 2$ を $\mathbb{F}_{17}$ 上で考えます。
領域 $H = \{1, 2, 3, 4\}$ 上での評価は

$$(5, 8, 11, 14)$$

です。

もし誰かが 1 箇所だけ改ざんして

$$(5, 8, 12, 14)$$

を「同じくらい自然な表です」と主張しても、これは元の 1 次多項式の評価表ではありません。
「表として」見ると 1 箇所の違いに過ぎませんが、「低次数多項式の評価表」として見ると非常に不自然です。
このギャップを利用するのが低次数テストです。

Reed–Solomon 符号

Reed–Solomon 符号は、「低次数多項式をある評価領域で評価したベクトル全体」のことです。

- ある次数上限 d
- ある評価領域 $H \subseteq \mathbb{F}$
- 多項式 f の次数 $< d$

を決めると、コード語は

$$(f(\alpha))_{\alpha \in H}$$

です。

重要なのは、**符号語には冗長性がある**ことです。
この冗長性のおかげで、「たまたま少数点だけ合っている偽物」を見破りやすくなります。

Aurora はこの Reed–Solomon 的な見方を強く使います。

関数の拡張：multilinear extension

Virgo を理解するには、もう一つ別の見方が要ります。
それが **multilinear extension** です。

関数

$$V : \{0,1\}^\ell \to \mathbb{F}$$

が与えられたとき、これを Boolean hypercube の外へ自然に延長した多項式 $\tilde{V}$ を multilinear extension と呼びます。

$$\tilde{V}(x_1, \dots, x_\ell) = \sum_{b \in \{0,1\}^\ell} V(b) \prod_{j=1}^{\ell} ((1 - x_j)(1 - b_j) + x_j b_j).$$

意味は単純です。

- 元の表 V は $\{0,1\}^\ell$ 上だけで定義されている。
- それを「各変数について次数 1」の多項式へ拡張する。
- Boolean 点では元の値を保つ。

Virgo / GKR は「各 layer の gate 値表」を、この multilinear extension として見ます。

IOP とは何か

Aurora を読むと **IOP (Interactive Oracle Proof)** が出てきます。
これは interactive proof と PCP 的な発想の中間にあるモデルです。

平たく言えば、

- 証明者は長いメッセージを送る
- ただし検証者はその全部を読む必要はない
- 検証者は、その長いメッセージを「配列へのオラクルアクセス」として扱い、必要箇所だけランダムに読む

というモデルです。

なぜ IOP がうれしいのか

巨大な証明をまるごと読むのは重いですが、もしその証明が

- よく構造化されていて
- 少数点の検査だけで大域的正しさが言えれば

検証者はごく一部しか見なくて済みます。Aurora はこの IOP 的世界観の上で設計されています。

非対話化はどうするのか

IOP は本来対話的です。しかし実用上は 1 本の証明にしたい。
そのためによく使うのが

- **Merkle 木**で長いオラクルをコミットする
- **Fiat–Shamir 変換**で検証者のランダム性をハッシュから導く

という定石です。

すると、証明者は

1. オラクル列にコミットし、
2. ハッシュでチャレンジを決め、
3. 必要箇所の開示と認証パスを付ける

ことで、非対話証明にできます。

sumcheck の直感

Aurora と Virgo の両方で、**sumcheck 的発想**が極めて重要です。

最も簡単な 2 変数版を見ます。
多項式 $g(X_1, X_2)$ について

$$T = \sum_{b_1, b_2 \in \{0,1\}} g(b_1, b_2)$$

という主張を検証したいとします。

1 ラウンド目

証明者は

$$s_1(X) = g(X, 0) + g(X, 1)$$

を送ります。検証者は

$$s_1(0) + s_1(1) = T$$

を確認します。

2 ラウンド目

検証者がランダム r_1 を選び、証明者は

$$s_2(X) = g(r_1, X)$$

を送ります。検証者は

$$s_2(0) + s_2(1) = s_1(r_1)$$

を確認します。

最後

検証者がランダム r_2 を選び、最後に

$$s_2(r_2) = g(r_1, r_2)$$

を確認すればよい。

こうして、「4 点の総和」の検査が「ランダム点での 1 回の評価」に圧縮されました。
高次元ならこの考えを何ラウンドも繰り返します。

なぜこれが効くのか

不正な証明者が途中で偽の多項式を送ると、どこかで次数や整合性が崩れます。ランダム点 r_i での検査は、その崩れを高確率で露呈させます。

- Aurora では **univariate 版の和検査**
- Virgo/GKR では **multilinear な和検査**

が登場します。

FRI の直感

FRI (Fast Reed–Solomon Interactive Oracle Proofs of Proximity) は、ざっくり言えば

> 「与えられた巨大な表が、本当に低次数多項式の評価表に近いか」

を効率よく確かめるプロトコルです。

何をしたいのか

証明者が巨大なオラクル表 f を出してきたとき、検証者は全部を読みたくありません。
でも「これは低次数多項式の評価表です」と言われた以上、その主張が本当かを確かめたい。

そこで FRI は、表を段階的に「折りたたむ」ことで、問題をどんどん小さくします。
低次数なら、折りたたんだ後も低次数性が保たれます。偽物なら、どこかで整合性が崩れます。

直感的な折りたたみ

FRI の詳細な式はやや技巧的ですが、感覚としては次のようなものです。

1. 大きな評価表を取る。
2. ランダム係数を使って、表のペアを混ぜ合わせる。
3. それに対応する「半分くらい小さい」表を作る。
4. これを繰り返して、最後は非常に小さな問題にする。

各段階で検証者は少数点だけを見るので済みます。Aurora では、この FRI が低次数性チェックの中核になります。

GKR の直感

GKR（Goldwasser–Kalai–Rothblum）プロトコルは、layered circuit の計算正しさを効率的に検証するための対話証明です。

基本のアイデアは、

> 「各 layer の全ゲート値を 1 個ずつ確かめるのではなく、その layer 全体の整合性を、1 本の和検査で圧縮する」

というものです。

layer ごとの値表

layer i の各ゲート値を並べた表を V_i とします。
GKR は、この表を multilinear extension $\tilde{V}_i$ として扱います。

検証者は「layer i の全ゲートが正しいか」を直接見る代わりに、ランダム点 r_i における $\tilde{V}_i(r_i)$ の主

張へ圧縮します。

すると、次はその主張を「ひとつ下の layer の主張」へ帰着できます。これを繰り返せば、最終的に入力 layer まで降りていきます。

wiring predicate

なぜ 1 つ下に降りられるのでしょうか。
それは、各ゲートが

- 加算ゲートか
- 乗算ゲートか
- どの 2 つの子ゲートに接続しているか

という wiring 情報を持つからです。

この wiring を表す述語が efficient に計算できると、各 layer の正しさを「次 layer の値表」との関係として低次数多項式で表現できます。Virgo では、この GKR 的縮約が backbone になります。

「structured circuit」とは何か

Virgo の複雑性で出てくる「structured circuit」は、初心者がつまずきやすい語です。
直感的には、

> ゲートのつながり方が規則的で、検証者が wiring を速く計算できる回路

と理解すれば十分です。

典型例は、

- 行列演算
- FFT
- Merkle 木
- 各層がほぼ同じ形を繰り返すデータ並列回路

などです。

逆に、接続がぐちゃぐちゃで、検証者が「その gate はどことつながるのか」を毎回重く計算しないといけない回路では、Virgo の verifier succinctness の旨味が薄れます。

Aurora と Virgo の違いを先取りする

ここまでの準備を使うと、両者の違いをかなり短く言えます。

Aurora 的世界観

- 計算を R1CS にする
- それを低次数なコード語の検査へ変換する
- IOP + FRI で低次数性を保証する
- 少数点の開示で全体の正しさを主張する

Virgo 的世界観

- 計算を layered circuit にする
- layer ごとの値表を multilinear extension として扱う
- GKR で上の layer の正しさを下の layer へ縮約する
- 多項式評価の主張を zkVPD で短く証明する

前者は **univariate / Reed–Solomon / FRI**、後者は **multilinear / GKR / zkVPD** です。

この章のまとめ

この章の要点を整理します。

- 算術回路は、大きな計算を加算・乗算の局所制約に分解するための表現である。
- R1CS は Aurora 側の標準表現であり、制約を $\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$ に揃える。
- layered circuit は Virgo 側の自然な表現であり、layer ごとに正しさを縮約しやすい。
- 低次数多項式は「剛性」が高く、少数のランダム検査と相性がよい。
- Reed–Solomon 符号は Aurora の視点、multilinear extension は Virgo / GKR の視点である。
- IOP は「長い証明を全部読まない」ためのモデルであり、Merkle + Fiat–Shamir で非対話化できる。
- sumcheck は「大量の和の主張をランダム点評価へ圧縮する」道具である。
- FRI は低次数性を効率よく検査する。
- GKR は layer ごとに回路の正しさを縮約する。

次章では、ここまでの下準備をもとに 4 方式を同じ地図で比較します。

第 4 章 4 方式を同じ地図で眺める

ここまでで、ゼロ知識証明の基本と、有限体・多項式・R1CS・layered circuit・IOP・GKR といった下準備を見ました。
この章では、**Ligero / Aurora / Virgo / zk-STARK** を同じ座標軸で並べます。

この章の狙いは、「4 方式の違いを、ブランド名ではなく**証明対象の表現・多項式の幾何学・制約圧縮の方法**として理解する」ことです。

4 方式を最短で比較する表

まず全体表を置きます。
細部はこのあと順に解説します。

方式	自然な入力表現	主役の多項式 / 符号	コアとなる圧縮	低次数検査の位置づけ	特に向く問題の形
Ligero	算術回路満足性	interleaved Reed-Solomon 符号	行列化 + ランダム線形結合 + spot-check	行ごとの符号語性を保つために使う	固定的な回路 relation、多数インスタンス
Aurora	R1CS	RS 符号語、低次数オラクル	Encoded IOP + algebraic checks + FRI	中心的。低次数オラクルの正当性を支える	R1CS に落としやすい一般 relation
Virgo	layered arithmetic circuit	multilinear extension、hypercube 上の多項式	GKR + zkVPD	点評価主張を支える部品として現れる	規則的な層構造を持つ回路
zk-STARK	execution trace / AIR	トレース多項式、composition polynomial、FRI	AIR + composition/ALI + FRI	中心的。トレースと制約の低次数性を検査	長い逐次計算、VM 実行、状態遷移

この表から分かる最重要点は、**4 方式とも「巨大な計算を少数の検査へ圧縮する」ことは同じだが、最初に取る表現が違う**ということです。

4 つの「自然言語」

証明系には、それぞれ「この形の計算なら自然に書ける」という言語があります。

Ligero の自然言語：回路の値表

Ligero は、回路の配線値や制約値を**行列**に並べて考えます。
核心は、「巨大な 1 本の witness ベクトルを 2 次元の表へ変え、行方向と列方向に別々の検査をかける」ことです。[Ligero17]

ここでは、

- 行方向に符号化する
- ランダムな線形結合で多数行を 1 本へ潰す
- 少数列だけを開く

という構図が自然に現れます。

Aurora の自然言語：R1CS

Aurora は、対象を R1CS として受け取るところから始まります。[Aurora19]
つまり「一般の計算」ではなく、「R1CS へ落としたあとの代数制約系」をネイティブ入力と見ます。

R1CS を選ぶ利点は、

- 制約がすべて同じ形に揃う
- 多項式恒等式へ落としやすい
- 低次数オラクルの整合性と結びつけやすい

ことです。

Virgo の自然言語：layered circuit

Virgo が重視するのは **layered arithmetic circuit** です。[Virgo20]
layered にすると、各層の主張を 1 つ下の層の主張へ落としていく GKR の強みが活きます。[GKR08]

したがって Virgo の視点は、R1CS のような「均質な制約列」というより、

- どのゲートがどの層にあるか

- wiring がどう規則的か
- 各層の値表をどう hypercube 上の関数として見るか

にあります。

zk-STARK の自然言語：execution trace

zk-STARK は、計算を**時系列の状態遷移**として見る系統です。[STARK18]
したがって主役は、

- ある時刻の状態
- 次時刻への遷移規則
- 境界条件
- それらを満たす長い execution trace

です。

これは VM 実行やロールアップの状態更新と非常に相性がよく、後年の zkVM / validity rollup の系譜に直接つながっていきます。[StarknetDocs] [StarkExDocs]

どの幾何学で多項式を見るか

4 方式の差は、単に「FRI を使うか」「GKR を使うか」ではありません。
もっと深い差は、**巨大な計算表をどの幾何学で眺めるか**です。

Ligero / Aurora / zk-STARK は univariate 的

Ligero、Aurora、zk-STARK は、それぞれ形は違うものの、

- ある評価領域上の
- 低次数な **univariate 多項式**
- またはその評価表

という見方が強いです。

Ligero では各行を低次数多項式として符号化し、Aurora では低次数オラクル群を構成し、zk-STARK ではトレース列や制約多項式を大きな領域へ拡張して扱います。

Virgo は multilinear / hypercube 的

Virgo の世界では、各層の値表は $\{0,1\}^\ell$ 上の関数として与えられ、それを **multilinear extension** に持ち上げます。
ここでの幾何学は「評価領域上の 1 変数多項式」よりも、「hypercube 上の多変数関数を多項式で延長する」見方です。

この差が、Aurora と Virgo の間で最も本質的です。

何を 1 本に圧縮しているのか

透明系 zk 証明では、「たくさんある条件を 1 つのランダムな条件に圧縮する」ことが繰り返し現れます。
ただし、何を圧縮しているかは方式ごとに違います。

Ligero

- 行列の多数行
- 線形制約
- 乗算制約

をランダム線形結合でまとめます。

Aurora

- 多数の R1CS 制約
- 低次数オラクルどうしの整合性
- 制約違反の残差

を少数の algebraic check に集約し、最終的に FRI で低次数性を確認します。

Virgo

- 各 layer の値表の正しさ
- wiring を介した親子関係
- 途中で現れる多項式評価主張

を GKR と zkVPD で圧縮します。

zk-STARK

- AIR の多数の局所制約
- 境界条件
- トレース列の整合性

を composition polynomial（あるいは歴史的には ALI 的な考え方）へ集約し、FRI で低次数性を検査します。

低次数性は「主役」か「支援役」か

この観点も重要です。

Ligero

低次数性は重要ですが、読者の体感としては「行がちゃんと符号語であることを支える部品」という側面が強いでしょう。
主役はあくまで**行列・ランダム結合・spot-check** です。

Aurora

低次数性はほぼ主役です。
オラクルが本当に低次数多項式から来ていることを FRI が支えなければ、証明者は局所的に整合する偽オラクルを作れてしまいます。[Aurora19] [FRI18]

Virgo

低次数性より前に、まず **GKR の layer 縮約**が主役です。
ただし、GKR が吐き出す評価主張を短く安全に扱うには zkVPD が必要で、そこでは多項式評価の検証が本質的になります。[Virgo20]

zk-STARK

低次数性は完全に主役です。
execution trace を多項式として見て、AIR を満たすときだけ composition polynomial が低次数になるよう設計し、FRI でその近接性を検査します。[STARK18] [FRI18]

共通部品は意外と多い

違いばかりが目立ちますが、共通点もかなりあります。

1. ランダム線形結合

4 方式とも、「多数の条件をランダム係数で混ぜて 1 本化する」発想を使います。

- Ligero では行や制約を混ぜる
- Aurora では残差やオラクル整合性をまとめる
- Virgo では sumcheck/GKR のランダム点が主張を絞る
- zk-STARK では AIR 制約を composition polynomial にまとめる

2. ハッシュベースのコミットメント

大量データを先に固定するため、Merkle 木やハッシュ中心の transcript 設計が多用されます。
このことが、これらの方式が「透明」「比較的ポスト量子親和的」と言われる大きな理由です。[Aurora19] [Virgo20] [STARK18]

3. Fiat-Shamir による非対話化

元は公開コインの対話型として理解すると構造が見えやすく、実用では Fiat-Shamir で非対話化することが多い。
ただし、FRI 系では Fiat-Shamir 後の安全性解析も後続研究で精密化されており、実装では transcript 設計を軽視できません。[FSFRI23]

「どれが一番強いか」と聞かれたら

この質問には慎重に答える必要があります。
なぜなら 4 方式は、同じ土俵で単純比較できないからです。

- Ligero は通信量と単純な部品の美しさが光る
- Aurora は transparent なのに proof size がかなり短い
- Virgo は structured circuit で verifier をかなり軽くできる
- zk-STARK は長い逐次計算や VM 実行で自然な表現を持つ

したがって正しい問いは、

「どの計算を、どの表現で書くのが最も自然か」

です。

この章のまとめ

- 4 方式の最も大きな違いは、**証明対象をどう表現するか** にある。
- Ligero は行列、Aurora は R1CS、Virgo は layered circuit、zk-STARK は execution trace を自然言語とする。
- Ligero / Aurora / zk-STARK は univariate 的、Virgo は multilinear / hypercube 的な見方が強い。
- 4 方式すべてで、ランダム線形結合・ハッシュコミットメント・Fiat-Shamir が重要である。
- 「どれが最強か」ではなく、「どの計算にどれが自然か」で選ぶべきである。

次章からは、Ligero → Aurora → Virgo → zk-STARK の順に、各方式の内部を丁寧に見ていきます。

第 5 章 Ligero の全体像

この章では、Ligero を**細部に入る前に**大づかみに理解します。
Ligero の価値は、単に「trusted setup が要らない」ことだけではありません。

原論文が強く打ち出したのは、次の組み合わせです。

- **サブリニア通信**
- **単純な暗号部品**
- **公開コイン**
- **具体的にもそこそこ軽い**
- **複数インスタンスでの償却効果**

まずは「Ligero が何を狙ったプロトコルか」を掴みましょう。

3.1 Ligero とは何か

Ligero は、2017 年の CCS に登場した
"Lightweight Sublinear Arguments Without a Trusted Setup"
という論文に由来する零知識 argument 系です。

ざっくり言うと、Ligero は

> **NP の関係を満たす witness の存在を、回路サイズの平方根に比例する程度の通信で示したい**

という発想に立つプロトコルです。

ここでの「平方根」は非常に重要です。
従来、回路サイズ s に対して証明通信が $O(s)$ 近くになると、大きな回路では厳しい。
Ligero は、適切に構造化すれば、通信を概ね $O(\sqrt{s})$ に近づけられることを示しました。

3.2 どうして平方根が出てくるのか

Ligero の心臓部は、「証明すべき大量のデータ」を**行列**に並べることです。

たとえば総量が s のデータを、

- 行数 m
- 列数 ℓ

の行列に並べて $m\ell \approx s$ とします。

このとき、典型的には

- 行に関するコストが m に比例
- 列を開くコストが ℓ に比例

するような設計になります。

すると総コストは大雑把に

$$m + \ell$$

の形になります。
ここで $m\ell \approx s$ を保ったまま $m + \ell$ を最小にするには、
バランスのよい

$$m \approx \ell \approx \sqrt{s}$$

を選べばよい。

これが、Ligero の「平方根」直観です。

大事な注意
これは**本質を掴むための簡略化**です。
実際の定数や追加項はもっと複雑で、符号長、繰り返し回数、健全性誤差、体サイズなどに依存します。
しかし初学者にとっては、まずこの「**二方向のコストの釣り合い**」が見えていることが重要です。

3.3 Ligero の設計思想

Ligero の設計思想を四つに分けると理解しやすいです。

1. witness と回路制約を “表” にする

証明したいのは「ある witness が回路を満たす」という事実です。

Ligero はこれを、行列やベクトルの整合性チェックに落とします。

2. 各行を Reed–Solomon 符号で冗長化する

行の値そのものを渡す代わりに、各行を**低次数多項式の評価列**として表現します。

これにより、「ちゃんとした行かどうか」を少数の検査で見やすくなります。

3. ランダム線形結合で多数の条件を一つに縮約する

各行を一個ずつ検査すると重いので、検証者はランダム係数を選び、

それらの線形結合だけを見ます。

壊れた行がどこかにあれば、高確率で線形結合にも矛盾が現れます。

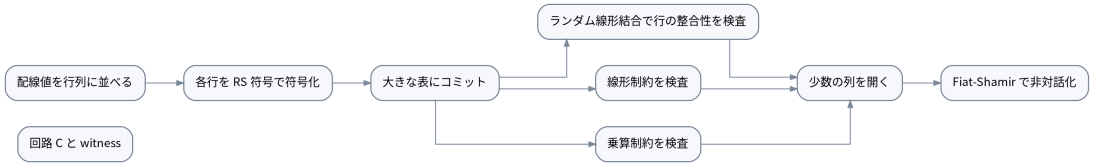
4. 少数の列だけを開かせる

証明者に表の全部を見せさせるのではなく、ランダムに選んだ少数列だけを開かせます。

これにより、通信が大きくならないようにします。

3.4 Ligero の全体フロー

細部を省いて全体像だけ描くと、Ligero は次のように見えます。



Ligero **の全体像**

このフローから分かるように、Ligero の主要部品は

- **RS 符号**
- **ランダム線形結合**
- **少数列の開示**
- **ハッシュによるコミットメント**

です。

pairing や楕円曲線ペアリングのような重い公開鍵暗号を中核に置かないところが、Ligero の特徴

です。

3.5 何が嬉しいのか

Ligero の魅力は、理論と実装の中間にあります。

trusted setup が不要

pairing-based SNARK の多くでは、危険な setup が必要でした。
Ligero は、その類の trapdoor 依存を避ける方向にあります。

使う部品が比較的単純

衝突耐性ハッシュ関数や、有限体上の演算、符号理論が中心です。
そのため、設計思想が追いやすく、量子計算への耐性を議論しやすい立場にあります。

通信がサブリニア

証明対象が十分大きいとき、証明者が表全体を送る必要がなく、回路サイズよりかなり小さい通信で済みます。

複数インスタンスの償却が効く

原論文では、多数の同型インスタンスをまとめて扱うと、一件あたりの通信がさらに下がることが示されています。
これは「同じ回路で多くの証明をまとめたい」状況で魅力的です。

3.6 原論文の代表的な結果をどう読むか

Ligero の原論文では、SHA-256 前像を例に、かなり具体的な性能評価が示されました。
有名なのは次のような主張です。

- 健全性誤差 2^{-40} の設定で、SHA-256 前像のゼロ知識証明に対し、およそ **44KB** 程度の通信
- 特定の仮定のもとではそれより短い通信
- 多数インスタンスでは、1 件あたり通信をさらに償却できる

ここで重要なのは、数値そのもの以上に、

> **“透明性寄りの系で、具体値としても十分戦える”**

と示した点です。

ただし、これを今日のあらゆる証明系と単純比較してはいけません。

証明サイズ、証明時間、検証時間は、

- 回路の種類
- 体の選び方
- 実装言語
- 健全性誤差
- 非対話化の方法
- バッチの有無

で大きく変わるからです。

3.7 Ligero はどんな問題に向くか

Ligero は、ざっくり言うと次のような場面に向いています。

1. **中規模〜大規模の回路を、trusted setup なしで証明したい**
たとえば

- ハッシュ計算の正しさ
- 認証・前像知識
- 回路として表しやすい検査

などです。

2. **複数の同種インスタンスをまとめたい**
たとえば多数の証明要求が同じ回路を共有する場合、償却の恩恵が得やすいです。

3. **重い公開鍵暗号ベースの SNARK を避けたい**
システム上の理由や安全性の好みから、ハッシュ・符号ベースの構成を好む場合に選択肢になります。

3.8 Ligero が万能ではない理由

Ligero を過大評価しないために、弱みも先に挙げておきます。

超巨大計算では STARK 的発想が強いことが多い

非常に長い逐次計算や VM 実行全体を証明するなら、トレースベースの STARK が自然なことが多いです。

証明サイズが pairings 系 SNARK より小さいとは限らない

「trusted setup は嫌だが、証明は極小がよい」という要件では、別系統の方が有利なことがあります。

数式の見通しはよいが、実装は依然として簡単ではない

RS 符号・行列配置・制約圧縮・ハッシュコミットメントを誤りなく組み合わせる必要があるため、研究実装としては十分に難しいです。

3.9 Ligero を一文で言うなら

Ligero を一文で言うなら、私は次のように要約します。

> **「回路満足性を、行列化された低次数構造のランダム検査に落とし、平方根オーダの通信で示す、透明寄りの零知識 argument」**

この一文の中に、

- 回路
- 行列
- 低次数
- ランダム検査
- 平方根
- 透明性

というキーワードが全部入っています。
以後の章では、この一つ一つを分解します。

3.10 この章のまとめ

- Ligero は、回路サイズの平方根に比例する通信を狙う零知識 argument 系である。
- 核心は、**行列配置・RS 符号・ランダム線形結合・少数列開示**である。
- 「平方根」の直観は、行方向と列方向のコストを釣り合わせることから出てくる。
- trusted setup を避けつつ、具体性能でも意味のある領域を狙った点が Ligero の大きな価値である。

- 次章では、行列・interleaved RS 符号・線形制約検査・乗算制約検査という内部構造を詳しく見る。

練習問題

1. Ligero の通信量に平方根が現れる直観を、自分の言葉で説明してください。
2. 「各行を RS 符号で冗長化する」とは、後でどのような検査を可能にするためか考えてみてください。
3. Ligero の利点と欠点を、それぞれ 2 つずつ挙げてください。
4. 同じ “transparent な証明系” でも、Ligero と STARK が異なる思想を持つことを一文で表現してみてください。

第 6 章 Ligero の中身を分解する

ここからは、Ligero の内部をもう少し厳密に見ます。
ただし原論文そのままの記法を全部追うのではなく、理解の流れを優先して

1. 何を行列に入れるのか
2. interleaved Reed–Solomon 符号とは何か
3. 行が正しい符号語であることをどう検査するか
4. 回路の線形制約・乗算制約をどう検査するか
5. どうやって零知識・非対話化を得るか

の順で説明します。

4.1 出発点：回路満足性

Ligero が最終的に示したいのは、

ある算術回路 C に対して、秘密 witness w を入れると $C(w) = 1$ である

という事実です。

なぜ算術回路なのか

ビット演算や論理回路も最終的には多くの場合、有限体上の算術制約に変換して扱えます。
たとえばブール値 $b \in \{0, 1\}$ は

$$b(b-1) = 0$$

という式で表せます。

AND, XOR, 加算器なども有限体上の式に落とせます。

そのため、まずは「加算ゲートと乗算ゲートからなる算術回路」として理解するのがよいです。

4.2 witness と配線値を行列に並べる

回路の各ワイヤに入る値をすべて集めると、長いベクトルになります。
Ligero はこの長いベクトルを、適当な行数 m と列数 ℓ を使って行列に並べます。

$$M \in \mathbb{F}^{m \times \ell}$$

としましょう。

この行列化の意味は二つあります。

意味 1：巨大な witness を二次元にする
一次元の長い列のままだと、「どこをどう開くか」「何をまとめて検査するか」が扱いにくいです。
二次元にすると、

- 行方向にまとめて圧縮する
- 列方向に spot-check する

という役割分担が自然になります。

意味 2：コスト最適化に使える
前章で見たように、行数と列数をうまく選ぶと、
証明に必要な通信を $m + \ell$ 型でバランスできます。

4.3 各行を RS 符号で符号化する

ここが Ligero の代数的なコアです。

各行 M_i を、少数点のメッセージから**低次数多項式** $p_i(X)$ として補間し、
より多くの評価点 $\eta_1, \dots, \eta_n$ で評価して、長い符号語

$$U_i = (p_i(\eta_1), \dots, p_i(\eta_n))$$

にします。

行全体を並べると、

$$U = \begin{pmatrix} U_1 \\ \vdots \\ U_m \end{pmatrix}$$

という $m \times n$ 行列になります。
各行が RS 符号語であるので、これは **interleaved Reed–Solomon code** の語だと考えられます。

interleaved とは何か

難しく見えますが、意味は単純です。

> **“複数本の RS 符号語を、行として縦に積んだもの”**

です。

なぜこの表現が強いのか

RS 符号は線形コードなので、行に対してランダムな線形結合を取ると、また RS 符号語になります。
つまり、検証者がランダム係数 $r_1, \dots, r_m$ を選び、

$$w = \sum_{i=1}^{m} r_i U_i$$

を考えると、正直な場合には w も RS 符号語です。

この一事が、Ligero の検査を劇的に軽くします。

4.4 行の “符号語らしさ” をどう検査するか

もし証明者が勝手な行列 U を出しても、各行が本当に RS 符号語かどうかを全部読むのは重すぎます。
そこで Ligero は、次のような検査をします。

membership test の直観

1. 検証者はランダム係数ベクトル $r \in \mathbb{F}^m$ を選ぶ

2. 証明者は線形結合
$$w = r^T U$$
を返す

3. 検証者は、w が RS 符号語であることを確認する
4. さらに、ランダムに少数列 j を選び、その列で

$$w_j = \sum_i r_i U_{i,j}$$

が成り立つことを確認する

これでなぜ十分なのか

もし多数の行が怪しいなら、そのランダム線形結合 w も高確率で怪しくなります。
一方、証明者が都合よく w を別物として偽装しようとしても、後でランダム列を開かせれば整合性が崩れやすい。

要するに、

- **行方向**はランダム線形結合で圧縮し
- **列方向**はランダム列開示で整合性を見る

という二段構えです。

4.5 なぜランダム線形結合が効くのか

Ligero の核心を一行で言えばこれです。

> **多数の条件を、ランダムに混ぜた一つの条件として検査する。**

この発想は、後で STARK の ALI や composition polynomial にも再び現れます。

一般原理

もしベクトル $v_1, \ldots, v_m$ のどれかが壊れているなら、
ランダム係数 r_i に対する和 $\sum_i r_i v_i$ も、高確率で壊れたままです。

もちろん「たまたま打ち消し合う」可能性はあります。
しかし係数を大きな有限体からランダムに選べば、その確率は十分小さくできます。

初学者向けの比喩

100 枚の答案のうち数枚が改ざんされているとして、各答案をランダム係数で混ぜた “合成答案” を作るとします。
改ざん箇所を正確に相殺するのは難しい。
だから合成答案の検査で、不正が表面化しやすい。
これがランダム線形結合の感覚です。

4.6 線形制約の検査

回路の加算ゲートやコピー制約は、**線形制約**として表せます。
たとえば

$$z = x + y$$

は

$$x + y - z = 0$$

という一次式です。

多くの線形制約をまとめると、行列表現で

$$Ax = b$$

の形になります。

Ligero のやり方

Ligero は、この $Ax = b$ を各制約ごとに別々に見るのではなく、
やはりランダム係数で混ぜた一つの制約として見ます。

1. 検証者はランダムベクトル r を選ぶ
2. 証明者は $(r^T A)x$ を表す多項式の係数を送る
3. 検証者は
   - その多項式が $r^T b$ と整合すること
   - もとの RS 符号化された表と整合すること を、少数の点で確認する

直観

全部の線形制約を読む代わりに、「ランダムな重み付き和が 0 であること」を見ます。
どれかの制約が壊れていれば、その重み付き和も高確率で壊れます。

4.7 乗算制約の検査

回路の本質的な難しさは、線形ではなく**乗算**です。
たとえばゲート

$$z = x \cdot y$$

は二次制約です。

Ligero では、x, y, z に対応する符号化表を用意し、
それらが

$$x \circ y = z$$

($\circ$ は成分ごとの積)を満たすかを検査します。

基本アイデア

- x, y, z の各行を表す多項式を考える
- ランダム線形結合を取る
- 積の関係を満たすはずの多項式 p_0 を証明者に送らせる
- ランダム列で実際の値と整合しているかを見る

なぜ難しいのか

線形制約なら線形結合がそのまま使えますが、積が入ると次数が上がります。
そのため、検査対象の多項式の次数や符号が少し変わります。
原論文では、この点を丁寧に扱って soundness を示しています。

初学者向けの見方

「乗算ゲートが正しいか」を全部見る代わりに、
多くの乗算ゲートをランダム係数で混ぜた一つの “合成乗算チェック” を作り、
それが正しいはずの多項式になるかを少数点で検査する、と思えばよいです。

4.8 回路全体はどう処理されるか

結局、算術回路は

- 入力・出力の整合性
- コピーや加算などの線形制約
- 乗算ゲートの二次制約

の組み合わせです。
Ligero はこれらをすべて、RS 符号化された行列の上の検査に落とします。

この観点では、Ligero は

「回路を符号化された表に写し、表がいくつかの代数的性質を持つことを示すプロトコル」

と言えます。

4.9 零知識はどう入るのか

ここまでの説明だけだと、「ランダム列を開かせたら witness が漏れそうだ」と思うはずです。
その通りで、**そのままでは零知識になりません**。

Ligero の零知識性は、もともと MPC-in-the-head / IKOS 系の変換に由来する視点から理解すると分かりやすいです。

非形式的な説明

- witness をそのまま見せるのではなく、マスクされた形・共有された形で扱う
- 検証者が開かせるのは、その一部の整合したビューだけにする
- それでも全体の整合性は高確率で保証されるよう設計する

原論文では、基礎にある MPC プロトコルのプライバシーから零知識性を導いています。

初学者が覚えるべき点

零知識のためには、単に「少ししか開かない」だけでは足りません。
開かれる少数の情報から witness が推測できないように、事前にランダム化されていることが必要です。

4.10 コミットメントと非対話化

対話型の Ligero では、検証者が乱数を送り、それに応じて証明者が返答します。
これを非対話型にしたいときは、

- 各オラクルや表にハッシュでコミットし
- チャレンジを transcript から Fiat–Shamir で導き

- 必要な位置だけ Merkle path 付きで開く

という形を取ります。

ここでハッシュが重要になる理由

- 証明者は先に表を固定しなければならない
- あとからチャレンジを見て都合よく内容を変えてはいけない
- そのため、コミットメントが必要
- Merkle 木は巨大な表に対する効率的なコミットメントになる

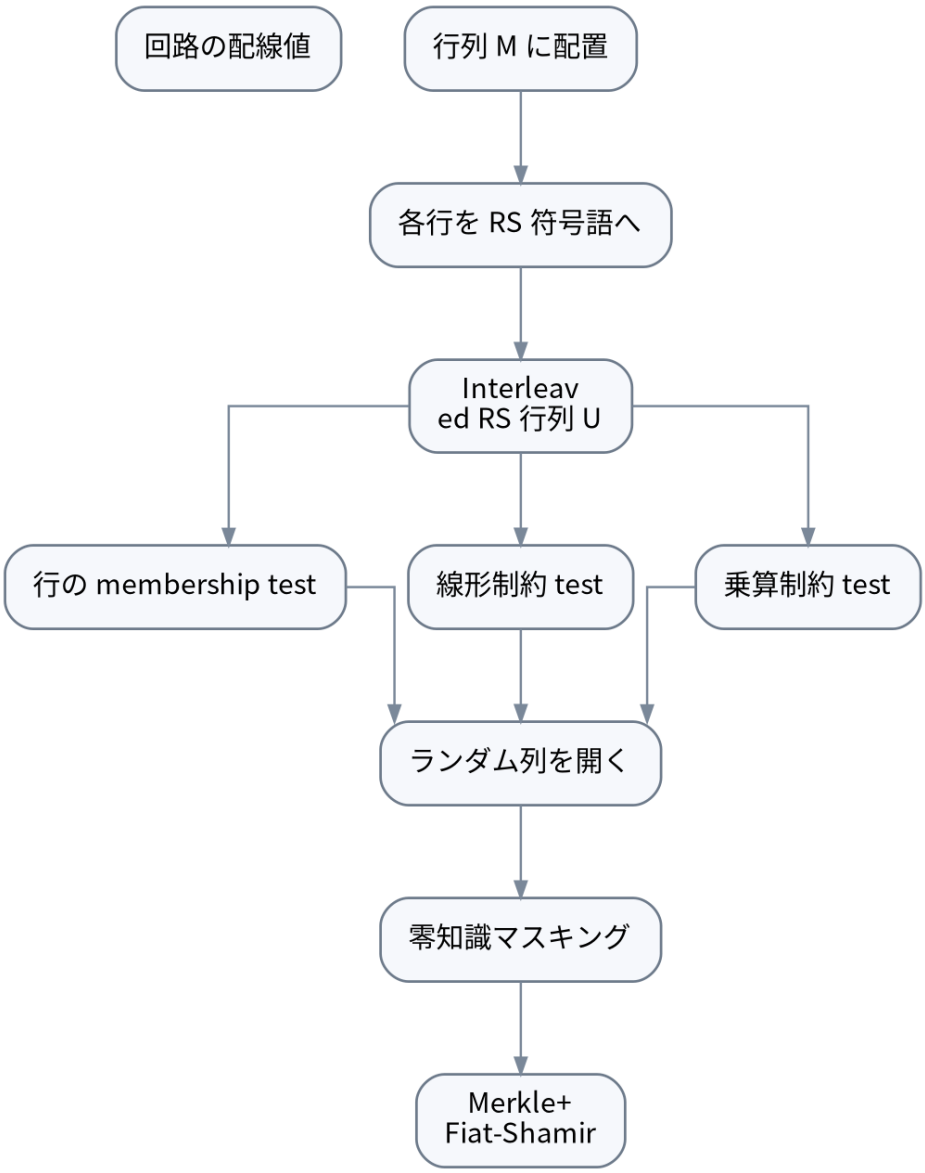
proof ではなく argument になる理由

この段階で安全性は、通常

- ハッシュの衝突耐性
- Fiat–Shamir の安全性仮定（ランダムオラクル等）

に依存します。
したがって実用形は argument と呼ぶのが自然です。

4.11 Ligero の内部を一枚絵にすると



Ligero **の中核検査フロー**

この図で重要なのは、「回路を直接検査している」のではなく、
符号化された表の整合性を検査しているという点です。

4.12 ありがちな疑問

Q1. なぜ列ではなく行を RS 符号化するのか

行列の持ち方自体は設計パラメータですが、Ligero では「行方向のランダム線形結合」と「列方向の局所開示」の役割分担が自然に働く配置を使います。

Q2. **すべての不正は本当に見つかるのか**

確率 1 ではありません。

ただし、不正な表がコードから遠いなら、ランダム線形結合とランダム列開示で、健全性誤差を十分小さくできます。

Q3. **では witness を少しずつ推測されないか**

零知識は、単なる spot-check のおかげではなく、**マスキングや MPC 的なプライバシーの設計**も含めて成り立ちます。

4.13 この章のまとめ

- Ligero は、回路満足性を符号化された行列の整合性検査へ落とす。
- 各行を RS 符号化した行列が **interleaved Reed–Solomon code** の語になる。
- ランダム線形結合により、多数の行や多数の制約を一つの検査に圧縮できる。
- 線形制約と乗算制約は、それぞれ対応する多項式検査として扱われる。
- 零知識は、開示量が少ないことだけでなく、マスキングや MPC 的プライバシー設計によって支えられる。
- 非対話化にはハッシュコミットメント、Merkle 木、Fiat–Shamir が重要である。

練習問題

1. interleaved RS 符号を「複数本の RS 符号語を並べた行列」と理解したとき、ランダム線形結合の意味を説明してください。
2. 線形制約 $Ax = b$ を、なぜランダム線形結合で一つにまとめられるのか説明してください。
3. 乗算制約が線形制約より難しいのはなぜか、次数の観点から述べてください。
4. Merkle 木が Ligero の非対話化に必要な理由を説明してください。
5. 「少数列しか開かない」だけでは零知識性が保証されない理由を考えてください。

第 7 章 Ligero を小さな回路で追う

この章では、Ligero を**小さな例**で追います。
本物のパラメータや soundness 解析はもっと複雑ですが、
「何がどこで検査されているのか」を掴むには、小例が最も有効です。

5.1 証明したい主張

次の公開主張を考えます。

> **公開値 $z = 8$ に対し、秘密値 a, b が存在して**
>
> $$a \cdot b + a = z$$
>
> **を満たす。**

たとえば witness として

$$a = 2, \quad b = 3$$

を持っていれば、

$$2 \cdot 3 + 2 = 8$$

なので主張は真です。

5.2 回路に直す

この関係を小さな算術回路にすると、内部配線値はこうなります。

1. 乗算ゲート

$$g_1 = a \cdot b$$

2. 加算ゲート

$$g_2 = g_1 + a$$

3. 出力制約

$$g_2 = z$$

witness を含めた配線値は

$$(a, b, g_1, g_2) = (2, 3, 6, 8)$$

です。

5.3 行列に並べる

簡単のため、これを 2×2 行列に並べます。

$$M = \begin{pmatrix} a & b \\ g_1 & g_2 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 6 & 8 \end{pmatrix}$$

本物の Ligero では、もっと大きな行列を使い、どの値をどこに置くかも細かく調整します。
ここでは「行列化して、行を符号化して、列を spot-check する」という構図だけを見ます。

5.4 各行を多項式として見る

有限体 $\mathbb{F}_{17}$ 上で考えましょう。
各行は 2 点のメッセージなので、一次多項式で補間できます。

第 1 行 $(2, 3)$
評価点を $\zeta_1 = 1, \zeta_2 = 2$ とすると、

$$p_1(1) = 2, \quad p_1(2) = 3$$

を満たす一次式は

$$p_1(X) = 1 + X$$

です。

第 2 行 $(6, 8)$
同様に、

$$p_2(1) = 6, \quad p_2(2) = 8$$

を満たす一次式は

$$p_2(X) = 4 + 2X$$

です。

5.5 RS 符号語にする

今度は評価点集合を $\eta = \{0, 1, 2, 3\}$ に広げて、各多項式を評価します。

第 1 行の符号語

$$p_1(0) = 1,\ p_1(1) = 2,\ p_1(2) = 3,\ p_1(3) = 4$$

なので

$$U_1 = (1, 2, 3, 4)$$

です。

第 2 行の符号語

$$p_2(0) = 4,\ p_2(1) = 6,\ p_2(2) = 8,\ p_2(3) = 10$$

なので

$$U_2 = (4, 6, 8, 10)$$

です。

したがって、符号化後の行列は

$$U = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 6 & 8 & 10 \end{pmatrix}$$

になります。

この時点で、元の 2 列の情報が 4 列に増え、冗長性が入っています。

5.6 行の membership test をやってみる

ここで検証者は、「各行が本当に RS 符号語なのか」を全部読む代わりに、
ランダム線形結合で検査します。

ランダム係数を選ぶ

たとえば検証者が

$$r = (5, 7)$$

を選んだとします。

線形結合を作る

すると

$$w = 5U_1 + 7U_2$$

です。成分ごとに計算すると、

$$\begin{aligned} w &= (5 \cdot 1 + 7 \cdot 4,\ 5 \cdot 2 + 7 \cdot 6,\ 5 \cdot 3 + 7 \cdot 8,\ 5 \cdot 4 + 7 \cdot 10) \\ &= (33, 52, 71, 90) \\ &\equiv (16, 1, 3, 5) \pmod{17} \end{aligned}$$

です。

これは本当に RS 符号語か

はい。

なぜなら

$$q(X) = 5p_1(X) + 7p_2(X)$$

とおけば、q も一次多項式であり、

$$\begin{aligned} q(X) &= 5(1+X) + 7(4+2X) \\ &= 33 + 19X \\ &\equiv 16 + 2X \pmod{17} \end{aligned}$$

となるからです。

実際、

- $q(0) = 16$
- $q(1) = 1$
- $q(2) = 3$
- $q(3) = 5$

で、確かに $w = (16, 1, 3, 5)$ になります。

ランダム列を開かせる

検証者は、たとえば評価点 $\eta = 1$（第 2 列）だけ開かせてもよいです。
そのとき元の値は

- 第 1 行：2
- 第 2 行：6

なので、

$$5 \cdot 2 + 7 \cdot 6 = 52 \equiv 1 \pmod{17}$$

であり、確かに w の第 2 成分と一致します。

同様に数列だけ spot-check して整合性を確認します。

5.7 もし不正な行が混じっていたら？

たとえば証明者が、2 行目を勝手に

$$(4, 6, 9, 10)$$

のように改ざんしたとします。
これは一次多項式の評価列ではありません。
なぜなら $(0, 1, 2, 3)$ 上で一次式なら、差分は一定になるはずなのに、

- $6 - 4 = 2$
- $9 - 6 = 3$
- $10 - 9 = 1$

でバラバラだからです。

このときランダム線形結合を取ると、高確率で w も低次数らしさを失います。
さらに、証明者が w だけ都合よく "低次数らしく" 偽装しても、ランダム列を開かせれば元の表と一致しなくなりやすい。

ここに Ligero の健全性の直観があります。

5.8 線形制約をこの例で見る

この回路の線形制約は、

1. 加算ゲート

$$g_2 - g_1 - a = 0$$

2. 出力制約

$$g_2 - z = 0$$

などです。

witness ベクトル

$$x = (a, b, g_1, g_2)$$

を使えば、これらは $Ax = b$ の形にまとめられます。

Ligero は何をするか

各線形制約を一つずつ確認する代わりに、検証者はランダム係数でまとめて

$$r^T(Ax - b) = 0$$

だけを見ます。
証明者は、その値を表す多項式を送り、
検証者は少数の点で元の符号化表と整合するか確認します。

この例での直観

たとえば加算制約だけを見ると、

$$g_2 - g_1 - a = 8 - 6 - 2 = 0$$

です。
もし証明者が g_2 を 9 に偽装していれば、

$$9 - 6 - 2 = 1 \neq 0$$

になります。
複数の線形制約をまとめてランダム重み付き和にしても、壊れた制約があれば高確率でゼロになりません。

5.9 乗算制約をこの例で見る

乗算制約は

$$g_1 = a \cdot b$$

です。
実際には

$$6 = 2 \cdot 3$$

なので正しい。

Ligero の考え方

a, b, g_1 に対応する符号化された値列があるとして、

$$a \circ b - g_1 = 0$$

を満たすことを検査したい。
ここで $\circ$ は成分ごとの積です。

しかし積が入るので、ただの線形関係ではありません。
そこで証明者は、「この積の関係が成り立つならこうなるはず」という**別の多項式**を送り、
検証者はその多項式の値と、ランダム列で開いた実値との整合を確認します。

この例での直観
もし証明者が $g_1 = 5$ と嘘をつけば、

$$a \cdot b - g_1 = 2 \cdot 3 - 5 = 1 \neq 0$$

です。
多数の乗算制約がある場合でも、それらをランダム係数でまとめた多項式検査に落とせます。

5.10 この小例で見えたもの

この例は小さすぎて、Ligero の本当のスケーリングは見えません。
しかし、次の重要な骨格は見えました。

1. “元の witness” ではなく “符号化後の表” を扱う
検査の対象は、もはや (a, b, g_1, g_2) そのものではありません。
それらを符号化した RS 評価表です。

2. ランダム線形結合が中心
行の membership test でも、線形制約でも、乗算制約でも、
「たくさんのものをランダムに混ぜて一つにする」という考え方が出てきます。

3. spot-check は列方向に行う
全部の列を開かず、ランダムな少数列だけを開く。
この一点が、通信量削減の本質です。

5.11 では平方根オーダはどこから来るか

この小例では 2 行 2 列なので実感しにくいですが、
一般に s 個のデータを $m \times \ell$ 行列にして、

- 行方向にまとめるコスト $\sim m$
- 列方向に開くコスト $\sim \ell$

なら、$m\ell \approx s$ のもとで $m + \ell$ を最小化することで

$$m \approx \ell \approx \sqrt{s}$$

が現れます。

この「**行方向圧縮 ＋ 列方向 spot-check**」の設計が、Ligero の平方根感を生みます。

5.12 ブール回路との関係

実際の応用では、SHA-256 のように**ブール回路**として自然な問題も多いです。
Ligero の原論文でも SHA-256 前像が代表例でした。

ブール回路を扱うには、

- ビット制約 $b(b-1) = 0$
- 論理ゲートを算術制約に変換

といったステップが必要です。

後続研究では、Boolean 向けの改良として **BooLigero** や **Ligero++** なども提案されています。
ただし、本書では骨格理解を優先し、算術回路の視点で進めます。

5.13 この章のまとめ

- 小さな回路でも、Ligero の流れは **回路 → 行列 → RS 符号化 →ランダム線形結合 → ランダム列開示** と整理できる。
- 行の membership test では、ランダム線形結合 $w = r^T U$ が中心的役割を果たす。
- 線形制約も乗算制約も、多数の条件をランダムに混ぜた多項式検査として扱われる。
- ここでの「行方向圧縮」と「列方向 spot-check」が、一般のサイズでは平方根オーダの通信へつながる。

練習問題

1. この章の例で、第 1 行 $(2, 3)$ に対応する一次多項式を自分で補間してください。
2. $r = (1, 4)$ を選んだ場合、$w = U_1 + 4U_2$ を計算してください。
3. 2 行目を $(4, 6, 9, 10)$ に改ざんすると一次式の評価列でなくなる理由を説明してください。

4. 線形制約と乗算制約で、証明者が送る多項式の役割の違いを説明してください。
5. 小例では見えにくい平方根オーダの直観を、一般の $m \times \ell$ 行列で説明してください。

第 8 章 Aurora の全体像

この章では、Aurora を「何を狙って作られた系なのか」「どの部品がどの役割を担うのか」という観点から俯瞰します。細部よりも、まず設計思想を掴む章です。

Aurora とは何か

Aurora は、Ben-Sasson, Chiesa, Riabzev, Spooner, Virza, Ward による透明な zk 証明系です。[Aurora19]
原論文の要約を一言で言えば、

R1CS に対する、transparent setup・軽量暗号・FRI ベースの succinct zk 証明

です。

原論文が強調する性質は次の通りです。

- 対象は **R1CS**
- **transparent setup**
- **plausibly post-quantum secure**
- **lightweight cryptography**
- 証明サイズは $O(\log^2 n)$（n は制約数）
- 証明生成は $O(n \log n)$ field operations
- 検証は $O(n)$

ここでまず気づくべきことは、「proof size は非常に短いが、verifier time は完全な定数・対数ではなく線形である」という点です。これは Aurora の性格を理解する上で重要です。

Aurora が解きたい問題

Aurora が念頭に置くのは、一般の計算を R1CS へ変換したあと、その satisfiable 性を **短い証明**で示したいという問題です。

より具体的には、次を同時に満たしたいわけです。

1. trusted setup が不要
2. pairing のような重い公開鍵暗号的構造に依存しない
3. 証明はできるだけ短い
4. witness は漏れない
5. できればポスト量子でも期待が持てる

この設計目標は、当時の zkSNARK 系列の中でかなり鮮明でした。Aurora は、STARK に近い透明性と、SNARK 的な短い証明を同時に狙う設計点にあります。

Aurora の基本アイデア

Aurora の大筋は次の 4 段階です。

1. **R1CS を、多項式やコード語の整合性チェックへ変換する**
2. **その整合性を、少数の algebraic checks に圧縮する**
3. **オラクルが本当に低次数なコード語に対応していることを FRI で確認する**
4. **オラクル全体は Merkle 木へコミットし、必要な点だけ開示する**

つまり Aurora は、

- 「制約の正しさ」と
- 「開示された表が本当に低次数多項式の評価表であること」

を同時に押さえる設計です。

なぜ R1CS から多項式へ行くのか

R1CS はもともと「各制約ごとの局所的な等式」です。
しかし、そのままだと制約数が多いと検査が重い。そこで Aurora は、制約列を低次数多項式の言葉へ埋め込みます。

直感的には、

- witness ベクトルや補助情報を大きな表として並べる
- その表を「低次数多項式の評価表」とみなす
- 制約違反があるなら、それはある多項式恒等式の破れとして現れる

という流れです。

ここで「低次数」であることが非常に大事です。低次数でなければ、証明者は都合のよい値だけを局所的に貼り合わせた “偽物の表” を作れてしまいます。そこで Aurora では、FRI を用いて **低次数性の保証** を本気で入れます。

Aurora を構成する主要部品

Aurora を、部品ごとに分けて見ましょう。

1. R1CS

入力となる計算はまず R1CS に落とされます。
Aurora はこの R1CS が satisfiable であること、すなわち witness が存在することを証明します。

2. Encoded IOP

Aurora は IOP 的な枠組みで動きます。[IOP16]
ただし単なる IOP ではなく、Reed–Solomon 的な符号化を前提とする「encoded」な見方が重要です。証明者の長いメッセージは、低次数多項式の評価表として扱われます。

3. Univariate sumcheck の新しい IOP

原論文の key ingredient として特に強調されるのが、**classical sumcheck の univariate analogue** です。[Aurora19]

通常の sumcheck は、多変数多項式を Boolean hypercube 上で足し上げる形を扱います。Aurora ではこれと似た精神を、Reed–Solomon 符号語の和に対して実現します。
この設計が、Aurora を「R1CS + RS codeword + FRI」の線へきれいに乗せています。

4. FRI

FRI は、開示されたオラクルが本当に低次数多項式の評価表に近いかを調べる低次数テストです。[FRI18]
Aurora では、この FRI が soundness のかなり深い部分を支えます。

ここでの直感はシンプルです。

- 低次数多項式の表は、ランダムな折りたたみや点検査に対して整合的に振る舞う。
- 偽物の表は、どこかでその整合性を失う。

5. Merkle コミットメントと Fiat–Shamir

IOP を非対話化するには、オラクル全体をハッシュで固定する必要があります。
Aurora は典型的に、オラクルを Merkle 木でコミットし、チャレンジを Fiat–Shamir で導きます。

これにより、証明は

- いくつかの根ハッシュ
- 必要位置の値
- 認証パス
- FRI 関連の開示情報

の集合として表現できます。

Aurora の複雑性をどう読むか

原論文の代表的な複雑性は次です。

- proof size: $O(\log^2 n)$
- prover: $O(n \log n)$
- verifier: $O(n)$

ここで n は制約数です。

proof size が短いとはどういうことか

制約数が数百万でも、証明サイズは polylog に抑えられます。
これは透明系としてはかなり魅力的です。

原論文では、128-bit security で「数百万制約でも 130kB 未満」という具体値が示されています。[Aurora19]
この数値は、Aurora が「透明なのに証明が短い」という印象を広く与えた理由の一つです。

verifier が $O(n)$ なのは弱点なのか

ここは初心者が誤解しやすい点です。

一見すると、検証が線形なら “succinct ではない” ように見えるかもしれません。しかし、少なくとも原論文の主張の文脈では、検証者は statement の記述自体を読む必要があります。R1CS が公開 statement の一部なら、それを読むコストはある程度避けられません。

ただし実務の感覚では、

- Groth16 のような非常に短い verifier
- 一部の preprocessing 系

と比べると、Aurora verifier は軽いとは言っても “極小” ではありません。
したがって Aurora は、**proof size の短さと transparent setup のバランスが強い**設計と見るのが自然です。

Aurora の何が新しかったのか

Aurora の歴史的な価値は、単に一つのプロトコルができたことだけではありません。少し抽象化すると、Aurora は次を示しました。

1. R1CS ベースでも、transparent な zk 証明をかなり短くできる
2. IOP と RS code と FRI の組合せは実用的設計線になりうる
3. hash ベースの部品だけでも、かなり強い性能を狙える

つまり Aurora は、「透明な zk 証明は proof size が大きい」という当時の固定観念をかなり押し広げた系でした。

Aurora の直感的な図式

Aurora の流れを、細部を捨てて模式化するとこうなります。

計算
↓
R1CS
↓
R1CS **の制約を多項式的主張へ変換**
↓
その主張を支えるオラクル群を構成
↓
オラクルが低次数コード語であることを FRI **で確認**
↓
Merkle + Fiat-Shamir **で非対話化**
↓
短い zk **証明**

重要なのは、Aurora が「R1CS を直接 bit-by-bit に見る」のではなく、**一度、符号化された多項式の世界へ持ち上げる**ことです。

Aurora が向く場面

本書の整理として、Aurora は次のような場面で発想上とても自然です。

- すでに R1CS へ落とすパイプラインがある
- transparent setup が重要
- pairing に依存したくない
- proof size をできるだけ抑えたい
- 低次数テストや FRI 系の思想を中心に据えたい

逆に、検証者ができるだけ軽くあってほしい場面や、回路が非常に規則的で GKR 的縮約が効く場面では、Virgo 的な発想がより魅力的になることがあります。

Aurora と STARK の関係をどう見るか

初心者は Aurora を「STARK の一種ですか」と聞きたくなるかもしれません。
厳密な分類論争に入る必要はありませんが、理解の助けとしては次のように考えるとよいです。

- Aurora は **FRI / IOP / transparent** という意味で STARK 的発想に近い
- ただし対象表現は R1CS であり、設計上の強調点も少し異なる
- 歴史的には、透明証明の設計空間を広げた重要な系として見るのがよい

本書では名称にこだわるよりも、**何をどの代数表現へ落としているか**に注目します。

Aurora を読むときの心の置き場

Aurora の原論文を読むと、最初は用語が多くて息苦しく感じます。
しかし読むべき核心は、実は次の 3 行です。

1. R1CS の satisfiability を、低次数オラクルの整合性へ翻訳する
2. その整合性を、univariate sumcheck 的な仕組みと FRI で検査する
3. オラクル全体は見せず、必要点だけ開示する

この骨格が見えれば、あとは個々の補題や多項式定義が「どの部品を支えるためのものか」として読みやすくなります。

この章のまとめ

- Aurora は、R1CS に対する transparent / hash-based / plausibly post-quantum な zk 証明系である。
- 証明サイズは $O(\log^2 n)$、証明生成は $O(n \log n)$、検証は $O(n)$。
- 核心は、R1CS を低次数多項式と Reed–Solomon 符号語の世界へ翻訳することにある。
- IOP 的に巨大なオラクルを扱い、FRI で低次数性を保証する。
- univariate sumcheck 的な考え方が Aurora の中核部品である。
- transparent で短い proof size を実現した点が、Aurora の大きな歴史的価値である。

次章では、Aurora を小さな例に沿って「何をどう証明しているのか」を手で追います。

第 9 章 Aurora を手で追う

この章では、Aurora の流れを小さな例で追います。
ただし最初に強調しておきますが、**ここで示すのは原論文の完全な数式展開ではなく、教育目的の模式図**です。Aurora の実際のプロトコルは、複数のオラクル、多項式恒等式、低次数検査を精密に組み合わせています。ここでは、その骨格を掴むことを目標にします。

例題

前章と同じく、次の式を証明対象にします。

$$z = xy + x + 2$$

公開 statement は $z = 20$。
秘密 witness は $x = 3, y = 5$ です。

中間値 $a = xy$ を導入すると、

$$a = xy, \qquad z = a + x + 2$$

です。

Step 1: R1CS に落とす

まず回路を R1CS にします。
変数ベクトルを

$$\mathbf{z} = (1, x, y, a, z)$$

と置くと、制約は

1. $x \cdot y = a$
2. $(a + x + 2) \cdot 1 = z$

です。

Aurora の出発点は、このような「線形形式の積 = 線形形式」の列です。

ここでの本質

Aurora は **各制約を 1 個ずつ検証したいわけではない**、という点が重要です。
制約数が数百万あれば、1 個ずつ見るのは無理です。
そこで Aurora は、これらの制約を一括して多項式的主張に包みます。

Step 2: 証明者が大きな表を作る

証明者は witness と補助データから、いくつかの大きな表を作ります。
直感的には、次のような情報が入っています。

- witness の符号化
- 中間値の符号化
- 制約ごとの左辺・右辺の値
- それらの整合性を束ねる補助オラクル

Aurora の重要な発想は、これらを単なる配列としてではなく、

> **低次数多項式をある領域で評価した表**

として扱うことです。

つまり、証明者は「私はこの大きな表を持っています」ではなく、「この表はある低次数多項式に由来しています」と主張します。

Step 3: 制約違反を多項式恒等式へまとめる

各制約が正しいとは、「制約残差」が 0 であることです。
教育用に簡単化して、各制約位置 α に対して

$$Q(\alpha) = 0$$

であってほしい残差関数 Q があると思ってください。

たとえば 2 制約しかないこの例では、

- 1 本目の制約で残差 0
- 2 本目の制約で残差 0

であってほしいわけです。

Aurora の本物のプロトコルはもっと洗練されていますが、直感としては

> 「多数の局所制約が全部正しい」
>
> $\Longrightarrow$
>
> 「ある多項式的な一括主張が成り立つ」

という翻訳が起きています。

どうして一括主張が嬉しいのか

一括主張になれば、後はそれを

- 和の主張
- 商多項式の主張
- vanishing polynomial で割り切れる主張
- 低次数性の主張

などへ変換して、ランダム検査で押さえられるからです。

Aurora の設計のうまさは、「局所制約列」から「少数の代数的主張」への翻訳にあります。

Step 4: univariate sumcheck 的な圧縮

Aurora の原論文が key ingredient として挙げるのが、**classical sumcheck の univariate analogue** です。

直感的には、検証者は

> 「この大きな領域全体で、残差がきちんと打ち消し合う / 0 になる」

という主張を、一点評価の主張へ圧縮したいわけです。

教育用の絵としては、

$$\sum_{\alpha \in H} Q(\alpha) = 0$$

のような形を想像するとよいです。
実際の Aurora では、Reed–Solomon 符号語上の和を扱う、より洗練された univariate チェックが使われます。

なぜ和を見るのか

和を取ると、多数の局所条件を 1 個の global condition にまとめられます。
そして sumcheck 的手法により、その global condition をランダム点での低次元チェックへ縮められます。

これは「大勢の生徒の答案を、1 枚ずつ読む代わりに、ランダムな集計式で一気に監査する」ようなものです。
集計式の設計が正しければ、1 人でも不正がいれば高確率で検出できます。

Step 5: でも "都合のよい表" を出されたら困る

ここで重大な問題があります。
証明者が「この表は低次数多項式の評価表です」と勝手に言っているだけなら、好きな値を貼った偽物かもしれません。

つまり Aurora で本当に検査したいのは 2 つです。

1. **代数的主張が正しいこと**
2. **その主張の入力となる表が、本当に低次数多項式から来ていること**

2 がないと、証明者は局所的にだけ辻褄の合う値を継ぎはぎして、global check をすり抜ける可能性があります。

Step 6: そこで FRI を使う

Aurora は FRI を使って、オラクルが本当に低次数であることを検査します。

FRI の直感はこうです。

1. 証明者は大きな評価表をコミットする。
2. ランダム係数を使って表を折りたたみ、より小さい表へ進む。
3. 折りたたみ前後の値が整合しているかを、少数の点で検査する。
4. 最終的に、非常に小さな表について直接確認できる段階まで落とす。

本当に低次数なら、この折りたたみ過程は整合的です。

偽物の表なら、どこかでほころびが出ます。

重要な直感

FRI は「点値をそのまま信用しない」ための装置です。
Aurora において FRI は、**証明者の長大なオラクル全体に対する誠実性検査**の役割を果たしています。

Step 7: オラクルは全部見せない。Merkle で固定する

Aurora は IOP 的発想の上に立っているので、証明者は巨大なオラクルを持っています。
しかし検証者は全部は見ません。

その代わり、証明者は各オラクルを

- 葉に評価値を置いた Merkle 木

でコミットし、根ハッシュだけを先に出します。

すると検証者は後から、

- 「この位置の値を見せてください」
- 「その値が本当に先ほどのコミットに属することを示してください」

と要求できます。

これにより、

- 全体は固定される
- 必要点だけ開けばよい
- 改ざんは Merkle 認証パスで防げる

という性質が得られます。

Step 8: Fiat–Shamir で非対話化する

本来 Aurora は対話的 IOP の流れを持っていますが、実用では 1 本の証明にしたい。
そこで、検証者が振るはずだったランダムコインをハッシュから導きます。

概念的には、

$$\rho_1 = H(\mathrm{root}_1, \mathrm{root}_2, \dots)$$

のようにしてチャレンジを生成し、そのチャレンジを使って次の開示位置や係数を決めます。
これを各ラウンドで繰り返すことで、対話なしで transcript 全体を固定できます。

Step 9: ゼロ知識性はどう確保するのか

ここまで聞くと、「いや、ランダム点の値をいろいろ開示したら witness が漏れないのか」と思うはずです。
その懸念は正しいです。

Aurora では、オラクルがそのまま witness の生データにならないように、**ランダムなマスキング**が施されます。
直感的には、

- 本質的な整合性は壊さない
- しかし開示される点値から witness の構造は逆算しにくい

ように、乱数で “ぼかし” を入れます。

これはゼロ知識プロトコルでよくあるパターンです。
本当に検査したい式だけは保存し、それ以外の自由度は乱数で埋めてしまうわけです。

1 本の Aurora 証明に何が入っているか

教育的に単純化すると、Aurora の非対話証明には次のようなものが入ります。

- **複数オラクルの** `Merkle root`
- **それに基づく** `Fiat-Shamir` **チャレンジから決まる開示値**
- **各開示値の** `Merkle` **認証パス**
- **低次数性を検査する** `FRI` **関連の開示情報**
- **最終チェックに必要な少数の** `field` **要素**

検証者はこれらを用いて、

1. 開示値が本当にコミット済みオラクルの一部か
2. 代数的整合性が成立するか
3. FRI の整合性が成立するか

を確認します。

例題に戻ると何が起きているのか

例題 $z = xy + x + 2$ に戻ると、本質は次です。

- 証明者は $x = 3, y = 5$ を知っている。
- その witness が R1CS を満たすことを、多項式化・符号化・低次数テストで包む。
- 検証者は x も y も知らない。
- しかし、オラクル群が「正しい witness から来た低次数構造」を持つことだけは確認できる。

したがって、検証者は

> 「$z = 20$ に対して、確かに何らかの witness がある」

と納得できます。

Aurora の理解でつまずきやすい点

1.「FRI だけで全部証明している」と思う

違います。
FRI は **低次数性の検査** です。
「制約が正しいか」という algebraic relation 自体は、別途きちんと作り込まれています。

2.「R1CS をそのままランダム点で見ている」と思う

違います。
R1CS は一度、低次数多項式とコード語の世界へ翻訳されます。
Aurora の巧みさはまさにこの翻訳にあります。

3.「proof size が短いなら verifier も極小だろう」と思う

そうとは限りません。
Aurora では proof size の短さと transparent setup が強い魅力ですが、verifier time は $O(n)$ です。この点は、他の設計と比較するときに重要です。

4.「ゼロ知識性はハッシュしているから自動で得られる」と思う

違います。

ハッシュや Merkle 木は主にコミットメントの役割です。

ゼロ知識性そのものは、マスキングされたオラクル設計によって得られます。

Aurora を一行で再定義する

ここまでを踏まえると、Aurora は次のように言い換えられます。

> **R1CS の satisfiability を、低次数多項式の整合性主張へ変換し、その主張を IOP + FRI + Merkle + Fiat–Shamir で短いゼロ知識証明にする系**

この文を自分の言葉で言い換えられるようになると、Aurora の理解はかなり進んでいます。

この章のまとめ

- Aurora の本当の狙いは、膨大な R1CS 制約を少数の代数検査へ圧縮することにある。
- そのために、witness や補助情報を低次数多項式の評価表として扱う。
- univariate sumcheck 的発想で、大域的主張をランダム点検査へ圧縮する。
- FRI は、その表が本当に低次数であることを保証する。
- Merkle 木と Fiat–Shamir により、巨大なオラクルを非対話的な短い証明へ変換する。
- ゼロ知識性は、オラクルへの適切なマスキングで確保する。

次章では、Virgo を同様に俯瞰し、Aurora と対照的な設計思想を見ていきます。

第 10 章 Virgo の全体像

この章では、Virgo の設計思想を俯瞰します。Aurora が **R1CS + RS 符号 + FRI** の世界なら、Virgo は **layered circuit + GKR + zkVPD** の世界です。両者とも transparent ですが、内部構造はかなり違います。

Virgo とは何か

Virgo は、Zhang, Xie, Zhang, Song による透明な零知識証明系です。[Virgo20]
原論文の中心主張は、**layered arithmetic circuit に対して、trusted setup なしで succinct な零知識証明を与える**ことです。

原論文が明示する代表的な複雑性は次の通りです。

- prover time: $O(C + n \log n)$
- proof size: $O(D \log C + \log^2 n)$
- verification time: $O(D \log C + \log^2 n)$（**structured circuit なら**）
- lightweight cryptography のみを使う
- plausibly post-quantum secure

ここで

- C: ゲート数
- D: depth
- n: 入力数

です。

Virgo が解きたい問題

Virgo の狙いを Aurora と対比すると分かりやすいです。

- Aurora:「R1CS を低次数コード語の world へ持ち込む」

- Virgo:「layered circuit を GKR で圧縮し、その評価主張を短く証明する」

つまり Virgo は、**layered arithmetic circuit を直接のネイティブ表現として大事にする**系です。とくに、層構造が規則的な計算に対して、GKR 由来の強みを使いたい、という意図があります。

Virgo の backbone は GKR

Virgo の最重要出発点は GKR です。[GKR08]

GKR の精神は、

> 「回路の全ゲートを 1 個ずつ検証する代わりに、各 layer の正しさを 1 つ下の layer の主張へ縮約していく」

というものです。

layer i の値表 V_i を multilinear extension $\tilde{V}_i$ として見ると、GKR は

- $\tilde{V}_i$ のランダム点での主張
- を
- $\tilde{V}_{i+1}$ のランダム点での主張

へ落としていきます。
この layer-by-layer の縮約が、Virgo の心臓部です。

でも GKR だけでは Virgo にならない

ここが大事です。
GKR は非常に美しいプロトコルですが、そのままでは「今日欲しい短い zk 証明系」にはなりません。

課題は少なくとも 3 つあります。

1. **対話的である**
2. **多項式評価の主張を効率よく扱いたい**
3. **ゼロ知識化したい**

Virgo の新しさは、GKR の縮約に対して **transparent な zero-knowledge verifiable polynomial delegation (zkVPD)** を組み合わせたことにあります。

zkVPD とは何か

VPD は、直感的には

> 「大きな多項式 f を自分で全部計算したくない verifier が、ある点 r での値 $f(r)$ を prover に "委任" し、その値が正しいと短く確かめる仕組み」

です。

zkVPD では、さらに

- その委任過程で witness に関する情報が漏れない
- 証明サイズと verification が対数的に抑えられる

ことを目指します。

Virgo の原論文が強調するのは、**Underlying Virgo is a new transparent zero knowledge verifiable polynomial delegation scheme** という点です。[Virgo20]

なぜ VPD が必要なのか

GKR の内部では、各 layer の値表を多項式として見たときのランダム点評価が何度も出てきます。もしそのたびに巨大な表を全部送っていたら、通信量も verification も重すぎます。

そこで Virgo は、

- 巨大な表を多項式としてコミットし
- 必要な点での値だけを
- 短い証明付きで開示する

仕組みとして zkVPD を使います。

この意味で、zkVPD は Virgo において「GKR の圧縮結果を外に見せる窓口」の役割を持っています。

Virgo の大まかな流れ

Virgo の模式図は次のように書けます。

```
layered arithmetic circuit
  ↓
各 layer の値表を multilinear extension として見る
  ↓
GKR により layer i の主張を layer i+1 の主張へ縮約
  ↓
途中で必要になる多項式評価の主張を zkVPD で短く証明
```

↓

`Merkle / ハッシュ中心の透明な非対話 proof にする`

Aurora と並べると、違いがはっきりします。

- Aurora は「オラクル全体の低次数性」が前景
- Virgo は「layer 間の縮約と評価主張」が前景

もちろん Virgo にも低次数的・多項式的な検査はありますが、読んだ印象の中心は GKR です。

structured circuit とは何か

Virgo を語るとき、避けて通れないのが **structured circuit** です。

原論文の verifier complexity

$$O(D \log C + \log^2 n)$$

は、**回路が structured なら**という条件付きです。

直感的意味

structured circuit とは、おおむね

> gate の接続規則(wiring predicate)を verifier が高速に計算できる回路

です。

GKR では、各 layer について

- どの gate が加算ゲートか
- どの gate が乗算ゲートか
- どの 2 つの子へ接続しているか

という wiring 情報を多項式的に扱います。
この wiring がランダム点で速く評価できると、verifier は回路全体を読み直さずに済みます。

どんな回路が structured か

典型的には次のようなものです。

- Merkle 木の各層

- 行列積や畳み込みのような規則的レイアウト
- FFT のような定型 wiring
- データ並列に層が反復する回路

逆に、接続が不規則で、個々の wire をほぼ全部読まないと wiring が分からない回路では、Virgo の verifier succinctness は活きにくくなります。

Virgo の複雑性をどう理解するか

原論文の prover time は

$$O(C + n \log n)$$

です。
直感的には、「回路サイズ C にほぼ線形で、さらに入力由来の符号化コストが $n \log n$ 乗る」と考えればよいです。

何が嬉しいのか

GKR 系の魅力は、**大きな計算を layer ごとに圧縮できるため、prover の重さが比較的素直に回路サイズへ比例する**ことです。
Virgo はそこへ zkVPD を合わせることで、proof size と verifier も対数的に抑えようとしています。

verifier の見方

structured circuit なら verifier は

- depth D
- gate 数 C
- 入力数 n

に対して、polylog 的なコストでかなり短く済みます。
これは Aurora の $O(n)$ verifier と好対照です。

ただし、その代わりに回路の layer 構造や wiring の規則性が重要になります。

Virgo の concrete result

原論文の実験では、Merkle tree with 256 leaves を計算する回路について、

- proof generation: 53 秒
- verification: 50 ms
- proof size: 253 KB

という具体値が報告されています。[Virgo20]

この数値だけで全体性能を判断してはいけませんが、Virgo が

- 透明で
- 証明が数百 KB 級で
- 検証がかなり速い

という設計点にあることはよく伝わります。

Virgo の新しさを一言でいうと

Virgo の価値は、

> **GKR の layer-by-layer 圧縮を、transparent かつ zero-knowledge な多項式委任で包み、短い proof system に仕立てたこと**

にあります。

これは Aurora の「R1CS を RS codeword に埋め込む」発想とはかなり違います。
Virgo はむしろ、「回路の階層構造そのものを活かし切る」設計です。

Virgo が向く場面

本書の整理として、Virgo が自然に見えるのは次のような状況です。

- 回路が layer 化しやすい
- wiring が規則的で structured
- verifier をできるだけ軽くしたい
- GKR 的な圧縮が効く
- ハッシュ中心・transparent な設計が欲しい

たとえば、

- Merkle 木
- 規則的な線形代数計算
- 同型な層が何度も繰り返される回路

などは、Virgo 的発想と相性がよいです。

Virgo のトレードオフ

Virgo にももちろん弱点や注意点があります。

1. layered / structured への依存

Virgo の強みは、回路が layer 化され、しかも wiring が規則的であるときに最大化します。
一般の “ぐちゃぐちゃな” 回路では、その良さが目減りします。

2. 実装と理論の距離

GKR、multilinear extension、zkVPD は、それぞれ単独でも十分難しい概念です。
したがって Virgo は、Aurora とは別の意味で「内部を理解するのに腰が要る」系です。

3. 後続研究による再検討

Virgo は後続研究で再検討され、zkVPD の設計・実装に関する脆弱性や制約が指摘されています。[VirgoRevisit25]
本書では第 9 章で改めて触れますが、歴史的価値とそのままの実装採用可能性は区別して読むべきです。

Aurora と見比べたときの第一印象

まだ細部に入る前ですが、ここで Aurora と Virgo の最初の対比をしておきます。

観点	Aurora	Virgo
入力表現	R1CS	layered arithmetic circuit
多項式の見方	RS codeword / univariate	multilinear extension / hypercube
中核技法	Encoded IOP + FRI	GKR + zkVPD
verifier の特徴	線形時間	structured なら polylog 的
得意な直感	低次数性の一括検査	layer-by-layer の縮約

この表だけでも、かなり性格が違うことが分かると思います。

Virgo を読むときの心の置き場

Virgo を初めて読む人は、しばしば次の 2 つを混同します。

1. GKR が何をしているのか
2. zkVPD が何をしているのか

この混同を避けるために、役割分担をはっきりさせましょう。

- **GKR**:「回路の正しさ」を層ごとに縮約する
- **zkVPD**:「途中で必要になる多項式評価の主張」を短く安全に証明する

この切り分けで読むと、Virgo の構造がかなり整理されます。

この章のまとめ

- Virgo は、layered arithmetic circuit に対する transparent な零知識証明系である。
- backbone は GKR であり、layer ごとの正しさを下の layer へ縮約する。
- その縮約の途中で必要になる多項式評価主張を、transparent な zkVPD で短く証明する。
- 複雑性は prover $O(C+n\log n)$、proof size と verifier は structured circuit なら $O(D\log C+\log^2 n)$。
- Aurora との最大の違いは、RS / FRI 型ではなく、multilinear / GKR 型であること。
- Virgo の強みは structured circuit と相性がよい点にある。

次章では、Virgo を小さな layered circuit で具体的に追い、GKR と zkVPD の役割分担を体感します。

第 11 章 Virgo を手で追う

この章では、Virgo の流れを小さな layered circuit で追います。
ここでも最初に断っておきますが、**以下は原論文の完全な transcript を再現するものではなく、GKR + zkVPD の役割分担を理解するための模式図**です。

例題となる layered circuit

次の 4 入力の回路を考えます。

$$z = (x_0 \cdot x_1) \cdot (x_2 + x_3)$$

具体的な値として

$$x_0 = 2, \quad x_1 = 3, \quad x_2 = 4, \quad x_3 = 1$$

を入れると、

- $a = x_0 x_1 = 6$
- $b = x_2 + x_3 = 5$
- $z = a \cdot b = 30$

です。

これを layer 化すると、

- layer 2: 入力 x_0, x_1, x_2, x_3
- layer 1: 中間ゲート a, b
- layer 0: 出力 z

という 3 層の回路になります。

Step 1: 各 layer の値を表にする

各 layer の gate 値を表にします。

入力 layer

$$V_2(00) = 2, \quad V_2(01) = 3, \quad V_2(10) = 4, \quad V_2(11) = 1$$

ここでは 2 ビット添字で 4 個の入力を表したと思えば十分です。

中間 layer

$$V_1(0) = 6, \qquad V_1(1) = 5$$

- gate 0 は乗算ゲートで x_0, x_1 を子に持つ
- gate 1 は加算ゲートで x_2, x_3 を子に持つ

出力 layer

$$V_0(0) = 30$$

このように、各 layer を「添字 $\mapsto$ 値」の表として持ちます。

Step 2: その表を multilinear extension に持ち上げる

GKR / Virgo は、これらの有限表をそのまま扱うのではなく、multilinear extension に持ち上げます。

- $\tilde{V}_2$: $\{0,1\}^2$ 上の値表を $\mathbb{F}^2$ 上へ拡張した多項式
- $\tilde{V}_1$: $\{0,1\}$ 上の値表を $\mathbb{F}$ 上へ拡張した多項式
- $\tilde{V}_0$: 出力の定数多項式に近いもの

直感的には、

> 「離散的な gate 値表を、ランダム点で評価できる低次数関数に変換する」

ということです。

これで verifier は、表全体を読む代わりに、ランダム点 r での $\tilde{V}_i(r)$ を主張として追えます。

Step 3: 検証したい主張を 1 個に圧縮する

本当に検証したいのは「出力が 30 で、しかも各ゲート計算が全部正しい」ことです。
しかし各ゲートを全部見るのは重い。

そこで GKR は、layer 全体の正しさを **1 個のランダム点評価の主張**に圧縮します。

この例だと出力 layer はほぼ 1 個の gate しかないので、

$$\tilde{V}_0(r_0) = 30$$

のような主張から始まると考えてよいです。実際には r_0 は trivial でも構いません。

重要なのはここから先です。
Virgo は「出力 gate が正しい」という主張を、「中間 layer の値表が正しい」という主張へ落としていきます。

Step 4: GKR が layer 0 の主張を layer 1 へ落とす

出力 gate は乗算ゲートで、

$$z = a \cdot b$$

でした。
したがって出力の正しさは、中間 layer の値 a, b の正しさに帰着できます。

しかし GKR は単に「a と b を送れ」とは言いません。
そうではなく、layer 1 の値表 V_1 全体に関する主張を、ある低次数多項式の和として組み立て、それを sumcheck で圧縮します。

何を sumcheck するのか

詳細な式を省いて直感だけ言うと、GKR は次のような情報を 1 本の多項式へまとめます。

- その gate が加算か乗算か
- どの子ゲートに接続しているか

- 子ゲートの値が何か
- それらから親ゲート値が正しく計算されるか

この「gate の配線と値の整合性」をまとめた多項式の総和を検査することで、layer 0 の主張を layer 1 のランダム点評価主張へ縮約します。

Step 5: sumcheck による圧縮のイメージ

layer 1 には 2 個の gate があります。
本当に正しさを確かめようとすると、

- gate 0 は乗算か
- gate 0 の子は正しいか
- gate 1 は加算か
- gate 1 の子は正しいか

などを全部見る必要があります。

GKR はこれを、多項式 g の和の検査

$$\sum_{b \in \{0,1\}^{\ell}} g(b)$$

へ落とします。
そして classical sumcheck により、その主張を最終的に**ランダム点での g の 1 回評価** へ圧縮します。

この圧縮の結果、検証者は「layer 1 全体の正しさ」を直接見るのではなく、

- $\tilde{V}_1(r_1)$
- ときには $\tilde{V}_1(r_1')$

のような少数の評価主張だけを追えばよくなります。

Step 6: 同じことを layer 1 から入力 layer へ繰り返す

中間 layer の 2 つの gate についても、正しさは入力 layer の値に依存します。

- $a = x_0 x_1$
- $b = x_2 + x_3$

ですから、layer 1 の主張も最終的には layer 2（入力 layer）の主張へ落ちます。

こうして GKR は、

出力 `layer` の主張
↓
中間 `layer` の主張
↓
入力 `layer` の主張

と縮約していきます。

これのどこがすごいのか

回路全体が大きくても、検証者は各 layer の gate を全列挙しません。
各段階で必要なのは、ランダム点での評価主張と、その整合性だけです。
これにより、巨大回路でも verifier の仕事がかなり小さくなります。

Step 7: では $\tilde{V}_i(r)$ は誰がどう計算するのか

ここで zkVPD が登場します。

GKR の縮約では「では $\tilde{V}_1(r_1)$ はいくつですか」「では $\tilde{V}_2(r_2)$ はいくつですか」という問いが何度も出ます。
しかし verifier は巨大な表を持ちたくありませんし、自分で多項式補間もしたくありません。

そこで prover が

$$y = \tilde{V}_i(r)$$

という値を提示し、その正しさを **短い証明**付きで示します。
これが VPD 的役割です。

Step 8: zkVPD の役割を直感で理解する

zkVPD は、Virgo の文脈では次のように理解するとよいです。

- prover は巨大な値表（またはそこから導かれる多項式）を内部に持っている
- verifier は、その多項式の特定点での値だけ知りたい
- しかも、その値が本当に内部表から導かれたものかを確かめたい

- さらに、その過程で表の中身を漏らしたくない

したがって zkVPD は、

> **巨大な表への完全アクセスを与えずに、必要な点評価だけを正しく・短く・秘匿的に委任する仕組み**

です。

GKR と zkVPD の役割分担

ここは重要なので、もう一度書きます。

- **GKR** は「正しさの論理」を作る
- **zkVPD** は「その論理で要求される点評価を短く安全に提供する」

Virgo を理解するとき、この分担を取り違えないことが最重要です。

Step 9: structured circuit が効く場所

Virgo の verifier が軽くなるためには、wiring predicate の評価が速くないといけません。
この例では配線が非常に単純なので、verifier は

- gate 0 は入力 00,01 を掛ける
- gate 1 は入力 10,11 を足す

と簡単に理解できます。

大規模回路でも、配線が規則的なら verifier は「ランダムな gate 添字を与えられたとき、その子添字を高速に計算する」ことができます。これが structured circuit の旨味です。

逆に、配線が無秩序だと verifier は wiring 自体を重く読まねばならず、Virgo の理論上の succinctness がそのまま出にくくなります。

Step 10: 非対話化とゼロ知識化

Virgo も最終的には非対話 proof system として使いたいので、ハッシュ・Merkle・Fiat–Shamir 的な部品を使います。

また、GKR が要求する評価主張や zkVPD のやり取りは、そのままだと witness 情報をにじませる可能性があります。したがって、Virgo でもランダムマスキングを含む zero-knowledge 化が必要です。

ここでの直感は Aurora と同じです。

- 検証に必要な整合性は残す
- それ以外の自由度は乱数で埋める

ただし、Aurora では「低次数オラクル全体の世界」が主役だったのに対し、Virgo では「multilinear evaluations の世界」が主役です。

この例で verifier が本当に納得するもの

例題では、最終的に verifier は次を信じます。

- 入力表 V_2 と整合する何らかの計算が存在し
- その計算の layer 1 の値表が正しく
- 出力 layer の値が 30 である

しかも、その過程で prover は x_0, x_1, x_2, x_3 の生値や全中間値表をそのまま提出しません。
検証者は「少数のランダム点での正しい評価主張」と「それを支える短い証明」しか見ません。

Virgo の理解でつまずきやすい点

1. GKR が "表の全部を見ている" と思う

違います。
GKR の本質は、表全体の正しさを sumcheck で圧縮することにあります。

2. zkVPD が GKR を置き換えていると思う

違います。
zkVPD は GKR の代替ではありません。GKR が作った評価主張を外から扱いやすくする部品です。

3. verifier が常に速いと思う

structured circuit という条件が重要です。
配線の規則性がないと verifier の旨味は薄れます。

4. Virgo は単に “FRI の代わりに別の PCS を使っただけ” と思う

そうではありません。
Virgo は回路の見方からして Aurora とかなり違います。Aurora の中心が R1CS と RS codeword なら、Virgo の中心は layered circuit と GKR です。

Virgo を一行で再定義する

ここまでを一文にすると、Virgo は次のように言えます。

> **layered arithmetic circuit の正しさを GKR で layer-by-layer に縮約し、その過程で現れる多項式評価主張を transparent な zkVPD で短く証明する系**

この一文が腑に落ちれば、Virgo の設計思想はかなり掴めています。

この章のまとめ

- Virgo は layer ごとの値表を multilinear extension として見る。
- GKR は、上の layer の正しさを下の layer の主張へ段階的に縮約する。
- 各段階で現れる「多項式のランダム点評価」の主張を、zkVPD が短く安全に支える。
- structured circuit なら verifier は wiring を高速評価でき、理論上の succinctness が活きる。
- Aurora が RS code / FRI を前景に出すのに対し、Virgo は GKR / multilinear evaluations を前景に出す。

次章では、もう一つの大きな流れである zk-STARK に進み、execution trace・AIR・FRI の世界を見ていきます。

第 12 章 zk-STARK の全体像

この章から、もう一つの主役である **zk-STARK** に入ります。
Ligero が「回路制約を符号化表の整合性に落とす」世界なら、
zk-STARK は「**長い計算全体を実行トレースとして捉え、それが局所遷移規則に従うことを示す**」世界です。

この違いを最初に強く意識しておくと、STARK の専門用語が整理しやすくなります。

6.1 STARK という名前

STARK は一般に

- **Scalable**
- **Transparent**
- **ARgument of Knowledge**

の頭文字として説明されます。

文脈によっては「zk-STARK」と言って、**zero-knowledge** 版を明示します。
本書では簡単のため、基本的に「STARK」と言ったら zero-knowledge 版を含む広い意味で使います。

各語の意味

- **Scalable**
  証明対象が大きくなっても、検証コストや証明サイズが比較的穏やかに増えること。
- **Transparent**
  秘密の trusted setup や trapdoor を要しないこと。
- **Argument of Knowledge**
  偽の主張で検証者を騙すには、実際には対応する witness を “持っているに等しい” ことを要求するタイプの argument であること。

6.2 STARK が狙う問題

STARK の思想が特に映えるのは、

> **長くて複雑な計算を、全部やり直さずに検証したい**

という状況です。

典型例

- ブロックチェーン上の大量取引の状態更新
- 大規模な逐次計算
- 巨大なデータベースに対する検索・検証
- 仮想マシン実行の正しさ
- あるアルゴリズムの実行過程全体の正しさ

Ligero も一般計算を扱えますが、STARK は特に
“実行の長さ” が大きい問題に向いた見方を持っています。

6.3 STARK の基本直観

STARK の核心を一文で言うと、

> **計算を「時刻ごとの状態の列」と見なし、その列が低次数多項式として一貫していることを示す**

です。

実行トレース
計算を 1 ステップずつ進めたときの内部状態を表にしたものです。
行が時刻、列がレジスタや変数だと思えばよいです。

AIR
各ステップで満たすべき局所遷移規則を、有限体上の多項式制約として書いたものです。
たとえば

- 次の時刻の第 1 レジスタは、現在の第 2 レジスタと等しい
- 次の時刻の第 2 レジスタは、現在の二つの和である

といった規則を、多項式が 0 になる条件として書きます。

FRI

巨大な評価表が「本当に低次数多項式から来ているか」を、
少数の問い合わせで検査する仕組みです。

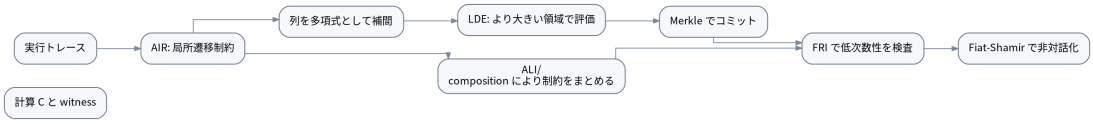
Merkle 木

巨大な評価表に対するコミットメントを与えます。

これらが合わさって STARK になります。

6.4 STARK の全体フロー

まずは細部を気にせず、一本の流れとして眺めましょう。



zk-STARK **の全体像**

Ligero と比べると、

- Ligero は「**行列の行**」が主役
- STARK は「**実行トレースの列**」が主役

という違いが見えます。

6.5 透明性とは何か

STARK の売り文句として最も有名なのが **transparent** です。
これは単に「公開されている」という意味ではなく、

> **秘密の setup、trapdoor、特権的な初期生成手順を前提にしない**

という技術的な意味を持ちます。

なぜ重要か

もしシステムが秘密の setup に依存し、その秘密が漏れると偽証が可能になるなら、
その setup 手順自体が社会的・制度的な弱点になります。

一方、transparent な系では、

- 公開乱数
- ハッシュ関数
- 符号理論

を中心に構成するため、その種の信頼集中を弱めやすい。

ブロックチェーンや公共的検証の文脈では、この点が特に強く評価されます。

6.6 “post-quantum secure” と言われる理由

STARK の原論文は、自らを **post-quantum secure** と位置づけています。
ただし、この表現は丁寧に読む必要があります。

どういう意味でそう言うのか

STARK の安全性の中核は、主に

- ハッシュ関数
- 公開乱数
- 符号理論
- ランダムオラクル的仮定

にあります。
少なくとも pairings や離散対数に強く依存する系に比べると、量子計算に対して “好ましい” 立場にある、という主張です。

何に注意すべきか

- 実際の安全性は使うハッシュ関数に依存する
- ランダムオラクルモデルの取り扱いには仮定がある
- 「絶対に量子安全」と軽々しく言うべきではない

したがって本書では、**“量子耐性志向”** あるいは **“量子計算に比較的親和的な仮定に立つ”** というニュアンスで理解することを勧めます。

6.7 STARK の代表的な長所

1. 大規模計算に強い

長い計算をトレースに変換し、局所遷移規則だけを見ればよいので、
逐次計算の証明と相性がよいです。

2. 透明性が高い

trusted setup を避けられることは、理論面でも実運用面でも大きな魅力です。

3. 検証が軽い

プロトコルの設計目標として、検証者は巨大な計算全体をやり直さずに済みます。
この性質が、L2 や公開検証に向きます。

4. 実運用の実績がある

現在のブロックチェーン領域では、STARK 系は単なる理論にとどまらず、
ロールアップやスケーリング基盤として実運用されています。

6.8 STARK の弱み・難しさ

1. 証明サイズが小さいとは限らない

多くの場合、pairing-based SNARK の証明より STARK 証明は大きいです。
透明性の代償の一つとみなせます。

2. 証明者はかなり重い

トレース生成、LDE、Merkle、FRI など、証明者が行う計算は多いです。

3. 概念の層が厚い

AIR、LDE、composition、FRI、Merkle、Fiat–Shamir など、多数の層が重なります。
初学者には Ligero よりも全体像が掴みにくいことがあります。

4. 実装は “理論どおり” ではないことも多い

現代の STARK 実装では、原論文の説明からさらに多くの最適化や派生手法が入っています。
本書ではまず、原論文系の骨格に集中します。

6.9 原論文の象徴的な例：DNA プロファイル照合

STARK の原論文で印象的なのは、
DNA プロファイルが巨大データベースに含まれないことを、データベース自体を公開せずに示す
という例です。

この例が示しているのは、STARK が単なる暗号マニア向けのおもちゃではなく、

- プライバシー
- 公共的検証
- 大規模データ処理

を同時に扱える、という方向性です。

本書では実装の詳細までは追いませんが、「STARK の社会的イメージ」を掴むにはよい題材です。

6.10 Ligero との第一印象の違い

この段階で、Ligero と STARK の違いを再度直観的に言うと、

- **Ligero**
  回路の満足性を、符号化された行列の spot-check に落とす
- **STARK**
  計算の実行過程を、低次数多項式の世界に持ち込み、トレース全体の一貫性を示す

です。

どちらもランダム化と符号理論を使いますが、
Ligero では「行列の表」を見ている感覚が強く、
STARK では「時間発展する計算」を見ている感覚が強い。

6.11 この章のまとめ

- STARK は、**大きな計算をトレースとして捉え、その正しさを低次数多項式の検査に落とす**証明系である。
- “Scalable” は、大きな計算でも検証が相対的に軽いことを目指す。
- “Transparent” は、trusted setup や trapdoor に依存しないことを意味する。
- STARK の骨格は、**実行トレース → AIR → LDE → Merkle → FRI** で理解するとよい。
- 次章では、STARK の最初の要となる **AIR**、すなわち「計算を状態遷移の多項式制約として書く」部分を詳しく見る。

練習問題

1. STARK の “transparent” が、単に「公開されている」という意味ではないことを説明してください。
2. Ligero と STARK の違いを、「何を圧縮しているか」という観点から述べてください。
3. 実行トレースとは何かを、アルゴリズムの実行履歴の比喩を使って説明してください。

4. STARK の長所と短所を、それぞれ二つずつ挙げてください。

第 13 章 AIR：計算を状態遷移として書く

STARK を理解する最大の山場の一つが **AIR** です。
AIR は **Algebraic Intermediate Representation** の略で、
計算を「**有限体上の状態遷移多項式系**」として書き直したものです。

言い換えると、

> **プログラムの実行が正しいとは、各ステップで局所的な遷移規則が満たされることだ**

という見方を、代数化したものです。

7.1 なぜ AIR が必要なのか

ゼロ知識証明でプログラム全体を扱うには、
最終的に「有限体上の多項式が 0 になるか」という形へ持ち込みたい。
しかし普通のプログラムは、そのままでは

- if 文
- ループ
- メモリアクセス
- レジスタ更新

などから成り、代数的ではありません。

そこでまずやることは、

1. 計算を時系列の**状態列**として表す
2. 1 ステップ先への遷移規則を書く
3. その規則を多項式制約にする

ことです。

これが AIR です。

7.2 実行トレースという見方

たとえばある計算が、各時刻で 2 つのレジスタを持つとします。
すると実行は次のような表で表せます。

時刻 t	レジスタ 1	レジスタ 2
0	a_0	b_0
1	a_1	b_1
2	a_2	b_2
$\vdots$	$\vdots$	$\vdots$

この表を **execution trace** と呼びます。

行と列の意味

- **行**：時間ステップ
- **列**：あるレジスタが時間とともに取る値

STARK は、このトレースを直接見るのではなく、
各列を多項式として表し、さらにその多項式が局所制約を満たすことを示します。

7.3 フィボナッチ列で AIR を作る

最も有名な教材例の一つがフィボナッチ列です。
状態を 2 レジスタ

$$(a_t, b_t)$$

とし、遷移を

$$(a_{t+1}, b_{t+1}) = (b_t,\ a_t + b_t)$$

で定めます。

初期値を

$$(a_0, b_0) = (1, 1)$$

とすると、トレースはこうなります。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	8

何を証明したいか

たとえば「4 ステップ後に $b_4 = 8$ になる」ということを、
途中の全計算を全部見せずに示したい、と考えます。

7.4 遷移規則を多項式で書く

現在状態を (X_1, X_2)、次状態を (Y_1, Y_2) と書きましょう。
フィボナッチ遷移なら、

$$Y_1 = X_2, \qquad Y_2 = X_1 + X_2$$

です。

これを「0 になる多項式」として書き換えると、

$$P_1(X, Y) = Y_1 - X_2$$
$$P_2(X, Y) = Y_2 - X_1 - X_2$$

です。
各ステップで

$$P_1 = 0, \qquad P_2 = 0$$

が成り立てば、そのステップの遷移は正しい。

これが AIR の核心です。

AIR の見方

AIR は「正しい計算とはどんな局所法則を満たすか」を書く仕様書です。
言語仕様や回路仕様に相当するものを、多項式制約に翻訳したものだと思うとよいです。

7.5 境界条件（boundary constraints）

遷移規則だけでは、どこから始まった計算か分かりません。
そのため STARK では、**境界条件**も一緒に扱います。

フィボナッチ例なら、

$$a_0 = 1, \qquad b_0 = 1$$

が初期条件です。
さらに公開したい出力条件として、

$$b_4 = 8$$

を入れてもよい。

なぜ必要か

遷移規則だけなら、「どこか別の初期値から始まる別の列」でも満たしてしまうからです。
証明したいのは、**その初期値・その出力**を持つ計算の正しさです。

7.6 AIR を “局所制約” と見る

STARK の非常に大事な感覚はこれです。

> **巨大な計算でも、1 行先との関係は局所的である。**

フィボナッチなら、各時刻の更新はたった二つの式で書けます。
もし 100 万ステップあっても、規則そのものは変わりません。

この「局所規則の繰り返し」が、STARK のスケーラビリティの源です。

CPU 実行の比喩

CPU が長いプログラムを実行するときも、本質的には「1 クロックごとの状態遷移」の繰り返しです。

STARK は、この思想を代数的に押し進めたものと見られます。

7.7 列を多項式として補間する

実行トレースの各列を、そのまま表として持つのではなく、多項式にします。

たとえば時刻集合 $H = \{0, 1, 2, 3, 4\}$ のような領域の上で、
列 $(a_0, a_1, a_2, a_3, a_4)$ を補間して多項式 $A(X)$ にし、
同様に $B(X)$ を作ります。

すると

$$A(t) = a_t, \qquad B(t) = b_t \quad (t \in H)$$

です。

何が嬉しいか

多項式にすることで、

- 低次数性が語れる
- より大きい領域に拡張できる
- 制約を多項式恒等式として書ける

ようになります。

ここで STARK は、「表そのもの」ではなく「表を生成する低次数多項式」に注目し始めます。

7.8 AIR 制約をトレース多項式に持ち込む

フィボナッチ例の AIR 制約

$$Y_1 - X_2 = 0, \qquad Y_2 - X_1 - X_2 = 0$$

を、トレース多項式で表すと、ざっくり

$$A(\mathrm{next}(x)) - B(x) = 0$$

$$B(\mathrm{next}(x)) - A(x) - B(x) = 0$$

のような関係になります。

ここで $\mathrm{next}(x)$ は、「時刻を 1 つ進めた点」に対応します。
実装や文献では、この “1 ステップ先” を有限体上で自然に表すよう、
評価領域を工夫して選びます。

重要な直観

行ごとに「次行との差」をチェックする代わりに、
列多項式の間に成り立つ代数的関係として見るわけです。

7.9 低次数が重要な理由を AIR の文脈で見る

ここで初学者が疑問に思うのは、

> 「遷移規則が正しいだけなら、なぜ低次数まで必要なのか」

という点です。

答えは、**低次数性がないと、証明者が各点で都合よく値を合わせた “偽の表” を作れてしまう**からです。

低次数が与える一貫性

- ある列は短いトレースから来た一つの多項式の評価である
- したがって、点ごとの値はバラバラには選べない
- 少数のランダム検査でも、多項式全体の一貫性が使える

つまり AIR は「何を満たせば正しい計算か」を定め、
低次数性は「その計算表がでたらめな寄せ集めでない」ことを保証します。

7.10 AIR の一般形

一般の AIR は、次のような要素から成ります。

状態幅（width）

1 行あたりに何個の状態変数を持つか。
レジスタ数だと思えばよいです。

遷移制約の数

1 ステップの遷移が、何本の多項式方程式で記述されるか。

遷移制約の次数

各制約多項式の次数。

高すぎると後の証明コストが増えます。

サイクル数（cycle count）

実行ステップ数。

トレースの長さに相当します。

境界条件

初期状態や最終出力など、特定位置で満たすべき条件。

STARK の設計では、これらのパラメータを小さくすることが重要です。

7.11 AIR は回路と何が違うのか

Ligero は回路的な見方が自然でした。

では AIR は回路とどう違うのでしょうか。

回路の見方

- 各ゲートの接続を明示する
- 依存関係は DAG 的
- 一回限りの大きな制約系として見る

AIR の見方

- 状態が時間とともに更新される
- 同じ局所規則が繰り返される
- 長い逐次計算に自然

したがって、**計算を「時間発展するプロセス」と見たいか、「一つの回路網」と見たいか**で、自然な表現が変わります。

7.12 フィボナッチ例を AIR としてまとめる

ここで例をきれいにまとめます。

状態

$$(X_1, X_2) = (a_t, b_t)$$

遷移制約

$$P_1(X, Y) = Y_1 - X_2 = 0$$
$$P_2(X, Y) = Y_2 - X_1 - X_2 = 0$$

初期条件

$$a_0 = 1, \qquad b_0 = 1$$

出力条件

$$b_4 = 8$$

これだけで、「フィボナッチを 4 ステップ回した計算」というオブジェクトが
代数的仕様として書けました。
次章では、この仕様をどうやって**低次数検査の問題**に変えるかを見ます。

7.13 この章のまとめ

- AIR は、計算を**有限体上の状態遷移多項式系**として表したものである。
- 実行トレースは「行が時刻、列がレジスタ」の表である。
- 遷移規則は、現在状態 X と次状態 Y の間の多項式制約 $P_i(X, Y) = 0$ として書ける。
- 初期値や出力値は境界条件として表す。
- STARK では、トレース列を多項式として補間し、その低次数性と制約成立を証明する。
- AIR は、STARK の “何が正しい計算か” を定義する仕様書だと理解するとよい。

練習問題

1. フィボナッチ例で、状態を (a_t, b_t) としたときの遷移制約を自分で書いてください。

2. 境界条件が必要な理由を説明してください。
3. 「遷移規則が成り立つ」ことと「低次数性がある」ことは、どのように役割が違うか説明してください。
4. 自分で簡単な状態遷移（たとえばカウンタ）を一つ考え、AIR 風に書いてみてください。

第 14 章 FRI・ALI・Composition による制約圧縮

前章で、STARK がまず AIR によって「何が正しい計算か」を多項式制約として書くことを見ました。
しかしそれだけでは足りません。

証明者はまだ、勝手な表を持ち出して

- たまたま見られた場所だけ整合している
- それ以外はでたらめ

ということができてしまいます。

そこで必要なのが、この章の主役である

- **LDE（低次数拡張）**
- **Merkle コミットメント**
- **ALI（または composition による制約の集約）**
- **FRI（低次数近接性検査）**

です。

この章では、STARK の各部品が何を担当しているのかを、役割ごとに分けて理解します。

8.1 まず役割分担を先に言う

STARK の中核部品は、よく混同されます。
最初に役割を一枚の表にしておきます。

部品	役割
AIR	何が正しい計算かを定義する
トレース多項式	実行トレースの各列を表す
LDE	トレース多項式を大きい領域に冗長化する

部品	役割
Merkle 木	大きな評価表にコミットする
ALI / composition	多数の制約を少数の多項式へ集約する
FRI	その多項式が本当に低次数に近いことを検査する
Fiat–Shamir	対話を非対話証明へ変換する

この表を頭に置いて読めば、用語の迷子になりにくいです。

8.2 LDE：なぜ大きい領域で評価し直すのか

トレース多項式 $A(X), B(X), \ldots$ は、もともとは小さな領域 H 上の値を補間して得られます。
しかし STARK では、そこからさらに大きな領域 L で評価し直します。
これが **Low-Degree Extension（LDE）** です。

直観

短い表だけを持っていると、ランダム検査の余地が少ない。
そこで評価点を増やして冗長化し、
「この列は低次数多項式から来ている」という構造を見やすくします。

何が嬉しいか

- 冗長性が増える
- ランダム照会の自由度が増える
- 制約評価や FRI が回しやすくなる
- Merkle 木で大きなオラクルとして扱える

Ligero との似ている点

Ligero でも、短い行を多数点評価して RS 符号語にしました。
STARK の LDE も発想としては近いですが、対象が「行」ではなく「トレース列」や「制約多項式」である点が異なります。

8.3 Merkle 木：大きなオラクルを固定する

LDE すると、証明者が持つ評価表は大きくなります。
検証者がそれを全部受け取るのは現実的ではありません。
そこで証明者は、その評価表を **Merkle 木** でコミットします。

流れ

1. LDE で得た長い評価列を葉に並べる
2. ハッシュで Merkle root を作る
3. root だけを先に公開する
4. 後で検証者が指定した位置だけ、値と認証パスを開く

何を防ぐのか

検証者のランダム質問を見てから、証明者が都合よく表を作り直すことを防ぎます。

つまり、**“最初に表を固定させる”** のが Merkle の役割です。

大事な点

Merkle 木は低次数性を証明しません。

それは FRI の仕事です。

Merkle 木はあくまで**コミットメント**です。

8.4 ALI：多数の制約をどうやってまとめるか

AIR には、多数の遷移制約や境界条件があります。

これを一本ずつ別々に検査すると重すぎます。

そこで STARK では、それらをランダム係数でまとめます。

これが **ALI（Algebraic Linking IOP）** の直観です。

何をしているのか

- 遷移制約 $P_1, P_2, \dots, P_s$ がある
- 各制約が各時刻で 0 になるべき
- それらをランダム係数 $\alpha_1, \dots, \alpha_s$ で混ぜて

$$\alpha_1 P_1 + \cdots + \alpha_s P_s$$

  のような一つの合成制約にする

なぜ効くのか

どれか一つでも制約が壊れていれば、ランダムに混ぜた結果も高確率で壊れます。

これは Ligero のランダム線形結合と極めて近い思想です。

“linking” の意味

各列多項式や境界条件が、正しい関係に “つながっている” ことを、一つの代数的対象に束ねる。
この意味で ALI は「制約の糊付け」の役割を持ちます。

8.5 composition polynomial という見方

現代の STARK 解説では、ALI という語よりも
composition polynomial という言い方が前面に出ることがあります。

直観

たくさんの制約違反を、一つの多項式 $C(X)$ の非零性に押し込める、という見方です。
正しい計算なら $C(X)$ は低次数で、しかも特定領域で適切な振る舞いをします。
不正があれば、その性質が壊れます。

初学者にとっての理解

ALI と composition polynomial は、詳細には文献流儀の違いもありますが、
本書ではまず

> **“たくさんの制約をランダムに混ぜて、少数の多項式検査へ圧縮する部分”**

として捉えれば十分です。

8.6 低次数性を本当にどう調べるか：FRI

ここでようやく **FRI** が登場します。
FRI は **Fast Reed–Solomon Interactive Oracle Proof of Proximity** の略で、

> **この長い評価表が、本当に低次数多項式の評価列に近いか**

を検査するプロトコルです。

“近い” とは

評価表が、ある RS 符号語——つまりある低次数多項式の評価列——と
ハミング距離の意味で十分近いかどうかを見ます。

なぜ “proximity” なのか

完全に一致しているかを全部読むのは不可能です。

そこで、「低次数コードから十分遠いなら高確率で拒否する」という近接性検査を行います。

8.7 FRI の直観：大きな問題を半分にたたむ

FRI の最も大事な感覚は、

> **低次数らしさの検査を、再帰的に “半分サイズの問題” へたたむ**

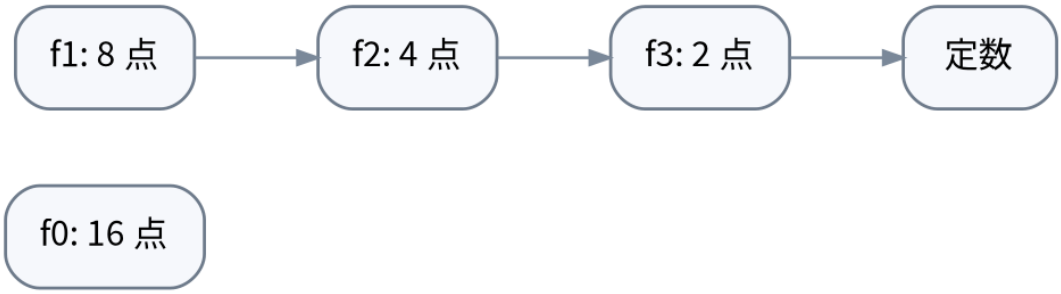
ことです。

ごく粗い絵

長さ 16 の評価列があるとします。

FRI はこれを、一回のラウンドで長さ 8 の新しい列へと “折りたたみ” ます。

さらにそれを長さ 4、長さ 2、最後は定数へとたたんでいきます。



FRI **の折り畳みイメージ**

各段階で、前の列と次の列が整合しているかを少数クエリで検査します。

なぜこれが低次数と関係するのか

もし元の列が本当に低次数多項式の評価列なら、

適切に折りたたんだ次の列もまた低次数多項式の評価列であるはずです。

逆に、元の列が低次数から遠ければ、この整合性がどこかで壊れます。

8.8 FRI をもう少しだけ具体的に

FRI の詳細はかなり技術的ですが、直観だけを追うとこうです。

1. 証明者は、元の評価表 $f^{(0)}$ に対して、半分サイズの新しい表 $f^{(1)}$ を用意する

2. 検証者はランダム点を選び、
   - $f^{(0)}$ の少数点
   - $f^{(1)}$ の対応点 を問い合わせる
3. それらが「折りたたみ規則」に従うかを確かめる
4. 同じことを再帰的に繰り返す
5. 最後に、定数や非常に低次数のものまで落ちたら受理する

どこが "Fast" なのか

FRI が重要なのは、原論文系の文脈で

- 証明者計算が線形時間
- 検証者計算が対数的

という非常に良いスケーリングを与えたからです。

これが STARK 全体の実用性に大きく貢献しました。

8.9 FRI は何を証明し、何を証明しないか

ここを混同しないことが大切です。

FRI が証明するもの

- ある長い評価表が、低次数多項式の評価列に近いこと

FRI が直接は証明しないもの

- その多項式が正しい計算から来たこと
- 境界条件を満たすこと
- AIR 制約を満たすこと

それらは、ALI / composition の段階で「低次数であるべき対象」に埋め込まれています。
つまり

- **AIR / ALI が "何を証明すべきか" を作り**
- **FRI が "それが本当に低次数か" を検査する**

という役割分担です。

8.10 零知識は STARK でどう入るか

ここまでだと、トレース多項式の値をそのまま見せることになり、秘密が漏れかねません。
そのため STARK では、公開するオラクルに対して**ブラインディング**や**マスキング**を入れます。

直観

- トレース多項式そのものを裸で出さない
- ランダムな低次数成分を足す
- ただし、制約検査が壊れないように慎重に設計する

注意

零知識化の具体手法は、STARK の世代や実装によって細部がかなり異なります。
本書では、まず

> **“トレースをそのまま見せないために、制約を保ったままランダム化する必要がある”**

という理解で十分です。

8.11 Fiat–Shamir と非対話化

対話型の FRI/ALI は、そのままでは検証者からの乱数送信が必要です。
非対話型 STARK にするには、Ligero と同様、

- 各 Merkle root
- それまでの transcript
- 必要なら公開入力

をハッシュして、次のチャレンジを導きます。

こうして、

1. 証明者が root を出す
2. その root からハッシュで乱数を導く
3. 次のオラクルを出す
4. またハッシュで乱数を導く

という流れで、検証者が実際にはいなくても transcript を構成できます。

8.12 STARK の各部品を Ligero と見比べる

ここまで来ると、Ligero との共通性も見えてきます。

観点	Ligero	STARK
ランダム線形結合	行や制約をまとめる	AIR 制約を composition にまとめる
符号化	各行を RS 符号化	トレース列や composition を低次数コードとして扱う
コミットメント	Merkle で表にコミット	Merkle でオラクルにコミット
spot-check	ランダム列を開く	ランダム位置を開く
低次数検査	各種整合性検査の中に現れる	FRI が中核として担う

つまり両者は、違う世界観を持ちながらも、

- ハッシュ
- 符号
- ランダム化
- 局所照会

を共有しているのです。

8.13 この章のまとめ

- LDE は、短いトレースを大きい領域へ拡張して冗長化する。
- Merkle 木は、大きな評価表を先に固定させるためのコミットメントである。
- ALI / composition は、多数の AIR 制約をランダムにまとめて少数の多項式検査へ圧縮する。
- FRI は、その多項式が本当に低次数コードに近いかを再帰的に検査する。
- STARK の零知識化には、オラクルのブラインディングが必要である。
- 役割分担を一言で言えば、**AIR が仕様、ALI が集約、FRI が低次数検査、Merkle が固定**である。

練習問題

1. LDE が必要な理由を、「評価点を増やす意味」という観点から説明してください。
2. Merkle 木と FRI の役割の違いを整理してください。
3. ALI / composition が、Ligero のランダム線形結合とどのように似ているか説明してくださ

い。

4. FRI の「半分にたたむ」という直観を、自分の言葉で言い換えてください。
5. FRI が “低次数性” だけを扱い、“計算の正しさそのもの” は別の部品が担う、という役割分担を説明してください。

第 15 章 zk-STARK をフィボナッチで追う

この章では、zk-STARK を**フィボナッチ列**の例で最後まで追います。
もちろん本物の STARK 実装はもっと大きく、もっと複雑です。
しかし「何にコミットし、何を低次数として示し、何をランダムに開くのか」は、小例でも十分に見えます。

9.1 証明したい主張

公開主張を次のようにしましょう。

> **初期値 $(1,1)$ から始めてフィボナッチ遷移を 4 回行うと、最終的に $b_4 = 8$ になる。**

この主張自体は手で確かめられます。
しかし STARK の目的は、この小さな例を通じて**大きな計算でも同じ骨格が働く**ことを見ることです。

9.2 実行トレースを作る

状態は 2 レジスタ

$$(a_t, b_t)$$

とし、遷移規則は

$$(a_{t+1}, b_{t+1}) = (b_t, a_t + b_t)$$

です。
するとトレースはこうなります。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	8

この表が、STARK における witness の中心です。
ブロックチェーンなら、これが巨大な VM 実行トレースになります。

9.3 AIR を書く

前章の内容をそのまま使うと、AIR は次の通りです。

遷移制約

$$P_1(X, Y) = Y_1 - X_2 = 0$$

$$P_2(X, Y) = Y_2 - X_1 - X_2 = 0$$

境界条件

$$a_0 = 1, \qquad b_0 = 1, \qquad b_4 = 8$$

これで、「正しいフィボナッチ計算」とは何かが決まりました。

9.4 列を多項式にする

トレースの 2 本の列

$$(1, 1, 2, 3, 5), \qquad (1, 2, 3, 5, 8)$$

を、それぞれ多項式 $A(X), B(X)$ に補間します。

初学者向けの理解

ここで本当に必要なのは、「列がたくさんの数の表ではなく、低次数多項式として扱われる」という視点です。

多項式そのものを明示的に書かなくても、
「A と B は時刻点でこの値を取る低次数多項式」と理解すれば十分です。

9.5 小領域から大領域へ：LDE

補間しただけでは、まだトレースは短いです。
そこで A と B を、より大きな領域 L 上で評価します。

たとえば概念的には

- 元の時刻点集合 H は 5 点
- 拡張領域 L はもっと多い点を持つ

とします。

すると、

- A の LDE
- B の LDE

という二つの長い評価表が得られます。

何が起きたか

元の短いトレース列が、長いオラクルへ変わりました。
この長いオラクルが Merkle 木でコミットされ、ランダム照会の対象になります。

9.6 Merkle でコミットする

証明者は LDE された評価表全体を Merkle 木に入れ、root を公開します。
これで、

- トレース列はもう後から変更できない
- しかし検証者は全部を読む必要がない

という状態になります。

この時点で検証者が知っていること

- まだ中身は知らない
- ただし「ある大きな表」が root によって固定されたことは知っている

STARK の証明は、この “固定されたが見えていない表” に対して進みます。

9.7 制約を一つにまとめる

AIR 制約は本来たくさんの点でチェックされるべきです。
しかしそれを全部やるのは重い。
そこでランダム係数を使って、制約を一つの composition polynomial にまとめます。

フィボナッチ例の直観

制約は主に三種類あります。

1. 遷移制約 $Y_1 - X_2 = 0$
2. 遷移制約 $Y_2 - X_1 - X_2 = 0$
3. 境界条件 $a_0 = 1, b_0 = 1, b_4 = 8$

これらにランダム係数 $\alpha, \beta, \gamma, \ldots$ を掛けて足し合わせ、
「正しいトレースなら、この合成多項式は低次数になるはずだ」という形へ変換します。

重要な点

検証者は、個々の制約がどこで壊れているかを逐一見ません。
ランダムに混ぜた結果だけを見ます。
この思想は Ligero と非常によく似ています。

9.8 もし最終値を偽装したらどうなるか

仮に証明者が、最後の値を 8 ではなく 9 にした偽トレースを持ち出したとします。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	9

このとき、少なくとも

- 遷移制約が壊れる
  （$5 + 3 = 8$ なのに 9 になっている）
- あるいは境界条件 $b_4 = 8$ が壊れる

のどちらかです。

その結果、composition polynomial の “低次数であるはず” という性質が崩れます。
FRI はそこを突きます。

9.9 FRI をこの例で見る

FRI は、「その合成オラクルが低次数に近いか」を検査します。

ごく粗い流れ

1. 証明者は、composition polynomial の評価表を用意する
2. それも Merkle でコミットする
3. 検証者はランダムチャレンジに応じて、折りたたみ系列

$$f^{(0)}, f^{(1)}, f^{(2)}, \ldots$$

   を検査する
4. 各段階で少数点だけ開く
5. 最後に、定数近くまでたためるなら受理する

初学者向けの見方

FRI は「**この大きな表は、本当に一本の低次数多項式から来たのか？**」を、
全部読まずに確認するための再帰的検査です。

9.10 検証者は具体的に何を見るのか

STARK の検証者は、直観的には次のものを見ます。

1. いくつかの Merkle root
   これで大きな表が固定される
2. ハッシュから導いたランダムチャレンジ
   どの制約をどう混ぜるか、どこを開かせるかを決める
3. ランダム位置の値と Merkle path
   本当に root に整合する値かどうかを見る
4. FRI の折りたたみ整合性
   低次数らしさが段階ごとに保たれているかを見る

ここでのポイントは、**計算全体の全表は見ていない**ことです。

見ているのは、ごく少数の値と、それらの間の代数的関係だけです。

9.11 なぜ少数点で十分なのか

この例を通じて分かるのは、STARK の検査力が三層構造になっていることです。

第 1 層：コミットメント
証明者は表を後出し変更できない。

第 2 層：ランダム集約
多数の制約を、ランダムに混ぜた少数の制約へ圧縮する。

第 3 層：低次数検査
圧縮後の対象が本当に低次数多項式由来かを FRI で見る。

この三層がそろうことで、「少数点だけ見れば十分」が成立します。

9.12 零知識はこの例でどう効くか

フィボナッチ例はそもそも秘密が少ないので、零知識のありがたみが見えにくいです。
しかし実際の応用では、トレースは秘密情報を大量に含みます。

たとえば

- 秘密鍵計算
- 取引の中間状態
- 非公開データベースへの照会
- 機密な業務計算

などです。

そのため STARK では、トレース多項式や制約オラクルにブラインディングを入れ、
開かれる少数点からでも内部状態が再構成されにくいようにします。

9.13 フィボナッチから実運用への橋渡し

「フィボナッチでは簡単すぎる」と感じるかもしれません。
その通りです。
しかし実運用では、同じ構図がそのまま大きくなります。

フィボナッチ例

- レジスタ数が少ない
- 遷移規則が単純
- ステップ数が少ない

ロールアップや VM 実行

- レジスタ・メモリ・命令ポインタなど多数の状態
- 命令ごとに複雑な AIR
- ステップ数が膨大
- 入出力やメモリ整合性も境界条件に入る

違うのは規模であって、骨格は同じです。

9.14 この章のまとめ

- zk-STARK の小例では、**実行トレース → AIR → 列の補間 → LDE → Merkle → composition → FRI** という流れが確認できる。
- 検証者はトレース全体を見ず、ランダム位置の少数開示だけを見る。
- 制約違反があると composition polynomial の低次数性が壊れ、FRI に捕まりやすくなる。
- フィボナッチ例は小さいが、巨大な VM 実行やロールアップも同じ骨格で理解できる。

練習問題

1. フィボナッチ例の実行トレースを、自分で 5 ステップ分書いてみてください。
2. 最終値を 9 に偽装したとき、どの制約が壊れるか説明してください。
3. Merkle root、composition polynomial、FRI がそれぞれ何を担当しているか整理してください。
4. フィボナッチではなく、簡単なカウンタ更新規則に対して STARK の流れを口頭で説明してみてください。

第 16 章性能比較：理論量・具体感・トレードオフ

この章では、**Ligero / Aurora / Virgo / zk-STARK** を性能面で比較します。
ただし最初に強く注意しておきます。

> **この 4 方式の性能比較は、完全な apples-to-apples 比較ではありません。**

理由は 3 つあります。

1. **入力表現が違う**
   Ligero は回路、Aurora は R1CS、Virgo は layered circuit、zk-STARK は execution trace / AIR を自然入力とします。
2. **論文が報告する量が違う**
   Ligero は「通信量」を前面に出し、Aurora / Virgo は「proof size・prover・verifier complexity」を前面に出し、zk-STARK はファミリとして発展しているため end-to-end の定数が変種ごとにかなり違います。
3. **実装成熟度が違う**
   Ligero / Aurora / Virgo は研究・教育上とても重要ですが、現在の大規模実運用は主に STARK 系が牽引しています。そのため「理論量」と「実務感覚」を分けないと比較を誤ります。[StarknetDocs] [StarkExDocs]

そこで本章では、比較を次の 3 層に分けます。

- **原論文が主張する理論量**
- **原論文が示した代表的な具体値**
- **実装・ユースケースを踏まえた本書の解釈**

まず理論量の表を置く

方式	原論文ベースの主な複雑性	読み方
Ligero	通信量 $\widetilde{O}(\sqrt{\lvert C\rvert})$ の public-coin ZK argument。[Ligero17]	回路サイズに対して平方根型の通信を狙うのが中核。
Aurora	proof size $O(\log^2 n)$、prover $O(n \log n)$、verifier $O(n)$。[Aurora19]	transparent なのにかなり短い proof size を実現するが、verifier は線形。
Virgo	prover $O(C + n \log n)$、proof size $O(D \log C + \log^2 n)$、verifier $O(D \log C + \log^2 n)$（structured circuit なら）。[Virgo20]	深さ D と構造性が効く。特に verifier の軽さが魅力。
zk-STARK	原論文群・公式説明では、proof size と verification resources は polylog 的、prover は大規模計算に対してスケーラブル。FRI 自体は線形 prover / 対数 verifier の arithmetic complexity を持つ。[STARK18] [FRI18]	ただし「STARK」はファミリ名であり、具体定数は AIR、幅、blowup、ハッシュ、再帰の有無でかなり変わる。

ここでの注意は、**Ligero の $\widetilde{O}(\sqrt{\lvert C\rvert})$ は通信量の話であり、Aurora / Virgo の proof size と完全に同一尺度ではない**ことです。

また、zk-STARK については「単一の固定プロトコル」ではなく、原論文・DEEP-FRI / DEEP-ALI・実装スタック・再帰合成を含む広いファミリとして読む必要があります。[STARK18]

原論文の具体値で見る

理論量だけでは温度感が見えません。

そこで、原論文が強調した具体値を並べます。

方式	原論文・公式資料で強調された代表例	何が読み取れるか
Ligero	SHA-256 preimage benchmark で、soundness 2^{-40} に対し通信約 44KB、prover 約 140ms、verifier 約 62ms。さらに 4096 インスタンスの償却では 1 インスタンスあたり約 2KB 通信、約 151ms prover、約 500 μ s verification を報告。[Ligero17]	「中規模回路でも意味のある concrete performance」と「多数インスタンスの償却」を強く打ち出している。

方式	原論文・公式資料で強調された代表例	何が読み取れるか
Aurora	128-bit security で、数百万 constraints 規模でも proof は 130KB 未満と報告。[Aurora19]	transparent でありながら proof size をかなり短く抑えたい、という設計意図が見える。
Virgo	256 leaves の Merkle tree で prover 約 53 秒、verification 約 50ms、proof size 約 253KB を報告。[Virgo20]	verifier の軽さは魅力だが、prover はかなり重い。構造化された回路で威力を発揮する設計である。
zk-STARK	単一の canonical benchmark を 1 行で代表させにくい。原論文・公式資料は「polylog の proof / verification resources」「大規模計算へのスケーラビリティ」を強調する。さらに実務では再帰 STARK や STARK+SNARK 圧縮が使われることも多く、単純比較が難しい。[STARK18] [Risc0Docs]	STARK は “1 本の数値” よりも、“長い trace を扱える表現力” と “実運用エコシステム” で見るべき。

特に重要なのは、**Virgo の verifier の軽さ**と、**Aurora の短い proof size** は魅力的だが、どちらも prover の重さや表現前提まで含めて読む必要があることです。
Ligero は concrete な通信の軽さが直感しやすく、zk-STARK は trace 型計算への自然さとエコシステムで優位が出やすい、という見方ができます。

Proof size をどう読むか

多くの読者はまず proof size を気にします。
しかし proof size だけで選ぶと危険です。

Ligero

Ligero の魅力は、単に「proof が小さい」よりも、**通信が回路サイズよりかなり小さくできる**ことにあります。
とくに平方根型の通信と multi-instance amortization が主役です。[Ligero17]

Aurora

Aurora は、この 4 方式の中でも「transparent なのに proof size がかなり短い」という印象を持ちやすい方式です。

ただし verifier が $O(n)$ なので、「proof size が短い = verification も極小」とは限りません。[Aurora19]

Virgo

Virgo の proof size は、depth D と structured 性が効くかどうかで読み味が変わります。
proof size だけなら Aurora が魅力的に見える場面でも、verifier complexity を含めると Virgo が有利に見えることがあります。[Virgo20]

zk-STARK

zk-STARK は、単体では proof が pairing-based SNARK ほど小さくないことが多い一方、transparent で大規模 trace に自然に乗ることが大きな価値です。
実用では recursive proof や外側の圧縮 proof を使う場合もあり、**“STARK 単体の proof size” と “製品として配布される最終 proof size” は一致しない**ことがあります。[Risc0Docs]

Verifier をどう読むか

Verifier の軽さは、公開検証・オンチェーン検証・多数クライアント配布で非常に重要です。

Ligero

Ligero の verifier は十分軽い concrete 値を持ち得ますが、設計の中心は「平方根型通信」と「多数インスタンスの償却」です。[Ligero17]

Aurora

Aurora は proof size が非常に短い反面、verifier が $O(n)$ であることを忘れてはいけません。
したがって「proof を小さく配りたい」目的には魅力がありますが、「verification を極限まで小さくしたい」なら別の選択肢も見えてきます。[Aurora19]

Virgo

Virgo の最大の売りは、structured circuit に対する verifier の軽さです。
もし対象回路が本当に規則的で、wiring predicate を効率よく評価できるなら、Virgo は verifier 観点でかなり魅力的です。[Virgo20]

zk-STARK

zk-STARK は、公式説明でも verification resources が polylog 的であることが強調されます。[STARK18]
ただし end-to-end のオンチェーン検証では、

- フィールド演算のコスト
- ハッシュ選択
- 再帰の有無
- 最終的にどの proof をチェーンに載せるか

が大きく効くため、理論式だけでなく実装　を見る必要があります。

Prover をどう読むか

現代の zk 証明では、ほとんどの場合 **prover が一番重い** です。
4 方式も例外ではありません。

Ligero

Ligero の prover は比較的素直な部品で構成でき、原論文の concrete benchmark も親しみやすいです。
ただし回路サイズが大きくなると、行列配置・符号化・コミットメントのコストは当然効いてきます。

Aurora

Aurora の prover は $O(n \log n)$ ですが、実装の実感としては

- 多数のオラクル
- FRI
- Merkle 木
- zero-knowledge masking

の管理が重くなります。
理論式以上に、メモリと transcript 管理が効きやすい方式です。

Virgo

Virgo の prover はしばしば重いと感じられます。
その代わり、回路構造が本当にきれいなら verifier で取り返す設計です。
「prover を重くして verifier を軽くしたいか」が大きな分岐になります。

zk-STARK

zk-STARK の prover は一般にかなり重いですが、長い逐次計算を trace として自然に処理できることが巨大な利点です。
大規模 trace を相手にするとき、回路フラット化より trace 表現のほうが有利に働くことが多く、この表現の自然さが prover の重さを補う場合があります。

メモリと定数因子は複雑性式より重要なことがある

理論量だけ読むと見落としがちですが、実装では次がとても重要です。

- オラクル表の個数
- Merkle 木の構築量
- FRI の中間表
- layer ごとの値表
- multilinear evaluation 用データ構造
- range check / bit decomposition のコスト
- non-native arithmetic の有無

とくに Aurora / zk-STARK は「低次数オラクルをたくさん持つ」感覚が強く、Virgo は「layer ごとの値表と wiring 計算」が強い。
Ligero は構造が比較的見通しよい一方で、大規模化するとやはりメモリ局所性とコミットメントが支配的になります。

方式別に一言で言うと

ここまでを一文に圧縮すると、次のようになります。

- **Ligero**: 通信量と concrete simplicity のバランスが魅力。多数インスタンスでも面白い。
- **Aurora**: transparent で短い proof size を強く狙うが、verifier は線形。
- **Virgo**: structured circuit に刺さる verifier 軽量性が魅力だが、prover は重い。

- **zk-STARK**: 単一数値で語るより、trace 型計算への自然さと大規模実運用で評価すべき。

比較を誤らないためのチェックリスト

新しい論文や実装を読むときは、必ず次を確認してください。

1. **何を入力表現にしているか**
   回路か、R1CS か、layered circuit か、execution trace か。
2. **何を proof size と呼んでいるか**
   対話通信量なのか、Fiat-Shamir 後の最終証明列なのか、再帰圧縮後のサイズなのか。
3. **verifier complexity はどの前提か**
   structured circuit なのか、公開入力読み込み込みか、前処理ありか。
4. **セキュリティ水準は同じか**
   80-bit, 98-bit, 128-bit では数字が大きく変わります。
5. **field / hash / recursion の条件は何か**
   end-to-end の数字はここでかなり変わります。

この章のまとめ

- 4 方式の性能比較は、入力表現が違うため単純比較できない。
- Ligero は平方根型通信と concrete benchmark、Aurora は短い proof size、Virgo は structured verifier、zk-STARK は trace 型大規模計算への自然さが目立つ。
- proof size, prover, verifier のどれを重視するかで評価は変わる。
- 特に zk-STARK はファミリとして発展しているため、「1 本の数値」より「どの種類の計算を自然に書けるか」で見るべきである。

次章では、これらの違いをユースケースに引きつけて、**どの問題にどの方式が自然か**を整理します。

第 17 章ユースケース比較：どの問題にどの方式が自然か

この章では、性能比較を「どんな問題に向くか」という形に翻訳します。
最初に最重要の原則を述べます。

> **4 方式は、理論的にはどれも広いクラスの計算を表現できる。違いは表現力そのものより、“どの arithmetization が自然で、どの定数因子が小さくなりやすいか” にある。**

したがって「この方式はこの問題にしか使えない」と考えるのは誤りです。
正しくは、「この問題をこの方式で書くと、表現が自然で、回路 / trace / layer / R1CS がきれいに現れ、性能上も得をしやすい」と考えるべきです。

まず結論の表

ユースケース	第一候補として自然な方式	理由
固定サイズの回路 relation（ハッシュ前像、単純認証、制約充足）	Ligero / Aurora	回路 or R1CS として素直に書ける。Ligero は通信の軽さ、Aurora は短い proof size が魅力。
既に R1CS パイプラインがある一般 relation	Aurora	R1CS がネイティブ入力。低次数オラクルへそのまま乗せやすい。
規則的な layered circuit（Merkle tree、算術木、規則 wiring）	Virgo	structured circuit と GKR の相性がよい。verifier をかなり軽くできる。
長い逐次計算、VM 実行、状態遷移系	zk-STARK	execution trace / AIR が自然。ロールアップや zkVM と噛み合う。
同一回路の多数インスタンス	Ligero	multi-instance amortization が魅力。
transparent setup を保ちつつ短い proof を重視	Aurora	proof size $O(\log^2 n)$ を前面に出す設計。
verifier の軽さを最優先し、回路が構造的	Virgo	structured 前提で verifier が魅力的。

ユースケース	第一候補として自然な方式	理由
実運用のエコシステムと長期拡張性を重視	zk-STARK	現行の大規模運用・zkVM・ロールアップ基盤が厚い。

ユースケース 1: 秘密を隠したまま「条件成立」だけ示したい

最も教科書的なのはこの類型です。

- ハッシュ前像を知っている
- パスワード条件を満たす
- 資格条件を満たす
- 非公開データがある閾値条件を満たす

Ligero が自然な場面

relation が固定サイズの回路として素直なら、Ligero はとても自然です。
原論文でも SHA-256 preimage を代表例に置いており、回路を行列化して符号化し、少数の列開示で検査する構図が理解しやすいです。[Ligero17]

特に向くのは、

- 同型の relation を多数回証明したい
- 回路が極端に深い trace 系ではない
- 通信量を回路サイズよりかなり小さくしたい

という場面です。

Aurora が自然な場面

同じ類型でも、すでに relation が R1CS に落ちているなら Aurora が自然です。
R1CS の satisfiability をそのまま低次数オラクル整合性へ翻訳できるため、一般 relation の「透明な succinct argument」として整理しやすいです。[Aurora19]

ユースケース 2: 一般計算を R1CS-first で扱いたい

これは研究や実装でかなり多いです。

- コンパイラや DSL が R1CS を吐く

- constraint system の最適化資産がある
- 既存の relation が R1CS 単位で整理されている

この状況では Aurora の見方が自然です。
R1CS に落としたあとで、

- 低次数オラクルへどう埋め込むか
- FRI で何を保証するか
- zero-knowledge masking をどう入れるか

という流れが一本の設計線として見えます。

どういう計算が例になるか

- 一般の NP relation
- 算術回路に落ちる認証・署名知識・整合性主張
- ある程度不規則な回路
- trace として見るより「制約の束」として見たほうが自然な問題

ユースケース 3: Merkle tree や規則的回路を軽い verifier で見たい

この領域では Virgo の魅力が出やすいです。

Virgo は、単に「透明」だから有用なのではなく、**layered circuit を GKR で縮約する**ため、対象回路が本当に規則的だと verifier 側の利得が大きくなります。[Virgo20]

Virgo が向きやすい例

- Merkle tree のような木構造計算
- 規則的な算術木
- wiring predicate を速く評価できる回路
- layer ごとの fan-in / fan-out が素直な回路

ただし注意

Virgo の「structured circuit 向き」は、曖昧語のまま使うと危険です。
本当に大事なのは、

- gate 添字から接続先をどう求めるか

- wiring predicate をランダム点でどう計算するか
- layer ごとの大きさがどう揃うか

です。
これが曖昧なら、理論上の verifier complexity の魅力を現実に引き出せないことがあります。

ユースケース 4: 長い逐次計算、VM、ロールアップ

ここは zk-STARK の本丸です。

なぜ STARK と相性がよいのか

状態遷移系の計算は、本質的に

- ある時刻の状態
- 次時刻への遷移
- 境界条件
- 長い履歴

から成ります。
これはそのまま execution trace と AIR の言葉で書けます。[STARK18]

つまり zk-STARK は、こうした計算に対して

> **計算を無理やり大きな平坦回路へ押し込む前に、まず trace として自然に表現できる**

という大きな利点を持ちます。

典型例

- zkVM の命令実行
- validity rollup の状態更新
- CPU / VM 実行の整合性証明
- 長いログ処理
- 反復アルゴリズムや逐次遷移

公式ドキュメントでも、StarkEx と Starknet は STARK 技術で state transition の正しさを証明する方向として説明されています。[StarkExDocs] [StarknetDocs]

ユースケース 5: 多数インスタンスの一括証明

これは Ligero が特に面白い場面です。
原論文は multi-instance amortization を強く打ち出しており、同一回路の多数証明をまとめたときの通信・検証償却が魅力です。[Ligero17]

どういう場面か

- 同じ回路に対して多数ユーザの witness を証明したい
- 認証・照合・定型クレームが大量にある
- 1 件ずつ proof を配るより、まとめて処理したい

もちろん他方式でも batching は可能ですが、Ligero はこの方向を理解しやすいかたちで持っています。

ユースケース 6: transparent で短い proof が欲しい

この問いには Aurora がよく候補に上がります。
Aurora の設計は、「trusted setup を避けながら、proof size をかなり短くしたい」という方向をかなり明確に持っています。[Aurora19]

ただし、ここでも忘れてはいけないのは

- proof size が短い
- verifier time が最小

は同じではない、ということです。

proof を配布・保存・伝送したい文脈では Aurora の短さは魅力ですが、verification の線形性まで含めて全体設計を考える必要があります。

ユースケース 7: verifier をとにかく軽くしたい

回路が構造的なら Virgo がかなり魅力的です。
特に、

- 公開クライアントが多数いる
- 軽い verification を大量に回したい

- 証明者を重くしてでも verifier を軽くしたい

という状況では、Virgo 的発想は強いです。[Virgo20]

ただし production 選定では注意

Virgo には 2025 年の再検討論文があり、zkVPD の脆弱性や実装上の制約が指摘されています。[VirgoRevisit25]
したがって「理論的に verifier が軽い」ことと「今そのまま安全に採用できる」ことは別です。

ユースケース 8: 研究・教育で設計思想を学びたい

この観点では、4 方式すべてに価値があります。

Ligero から学べること

- 行列化の意味
- spot-check の感覚
- 平方根型通信がどこから出るか
- RS 符号の初学者向け直感

Aurora から学べること

- R1CS から低次数オラクルへ行く設計
- transparent succinct argument の考え方
- FRI を「低次数性保証の中核」として使う見方

Virgo から学べること

- layered circuit と GKR
- multilinear extension と hypercube の幾何学
- evaluation delegation が証明系にどう組み込まれるか

zk-STARK から学べること

- trace / AIR / FRI の骨格
- 状態遷移系の証明

- 大規模計算と公開検証の接続

研究の入り口としては、4 方式を同時に学ぶ価値は非常に高いです。

方式選択の実務的な順序

実際に問題に当たったら、次の順で考えると整理しやすいです。

Step 1: まず「自然な表現」を決める

- 回路か
- R1CS か
- layered circuit か
- execution trace か

Step 2: 次に何を最優先するか決める

- proof size
- verifier
- prover
- 実装容易性
- エコシステム
- batching

Step 3: それから方式を選ぶ

この順序を逆にして「先に方式名を決める」と、無理な arithmetization で苦しみやすくなります。

4 方式を使い分ける一言まとめ

- **Ligero** を考えたくなるのは、回路 relation が自然で、通信を軽くしたく、同型インスタンスの償却にも関心があるとき。
- **Aurora** を考えたくなるのは、R1CS-first の一般 relation を transparent に、しかも短い proof で扱いたいとき。
- **Virgo** を考えたくなるのは、layered で structured な回路に対し verifier を強く軽量化したいとき。
- **zk-STARK** を考えたくなるのは、長い逐次計算・VM・状態遷移を自然に表したいとき。

この章のまとめ

- 4 方式はすべて広いクラスの計算を表せるが、得意な arithmetization が違う。
- Ligero / Aurora は固定的な relation に、Virgo は structured layered circuit に、zk-STARK は長い trace に自然に対応しやすい。
- production の方式選択では、性能表より先に「自然な表現」を見るべきである。
- 特に zk-STARK は rollup / zkVM 文脈、Virgo は structured verifier 文脈、Aurora は R1CS-first 文脈、Ligero は concrete simplicity / amortization 文脈で読むと整理しやすい。

次章では、研究実装・公式エコシステム・セキュリティ上の注意点をまとめます。

第 18 章実装・エコシステム・研究上の注意点

この章では、**理論としての 4 方式**から一歩進んで、

- 研究実装するとき何に気をつけるか
- 現在どの方式にどの程度のエコシステムがあるか
- 後続研究で何が再検討されているか

を整理します。

最初に最重要メッセージを述べます。

> **歴史的に重要で、理論的に美しいことと、そのまま本番採用してよいことは別です。**

本章では、2026-04-05 時点で確認できる原論文・後続研究・公式ドキュメントに基づく事実と、本書としての設計上の助言を分けて書きます。

まず 4 方式の現在位置を一表にする

方式	学術的重要性	実装・教育上の価値	現在の実運用エコシステム	本章での注意点
Ligero	非常に高い	高い。sublinear communication の入門として優秀	限定的。主に研究・教育・比較対象	理論の美しさは高いが、現在の主流 production stack ではない。
Aurora	非常に高い	高い。R1CS + FRI の祖型として重要	限定的。主に研究・設計理解	proof size は魅力だが verifier 線形性とオラクル管理を忘れない。
Virgo	非常に高い	高い。GKR + zkVPD の設計線を学ぶうえで重要	研究実装はあるが本番採用は慎重に	2025 年の再検討論文で脆弱性・制約が指摘された。

方式	学術的重要性	実装・教育上の価値	現在の実運用エコシステム	本章での注意点
zk-STARK	極めて高い	高い。trace / AIR / FRI の中核系譜	強い。StarkEx, Starknet, RISC Zero, Miden など	「STARK はファミリ」であり、実務では recursion / compression まで含めて見る必要がある。

Ligero: どこまでを期待すべきか

Ligero は、2017 年以降の transparent / setup-free / sublinear communication 系の重要な出発点の一つです。[Ligero17]

良い点

- 設計が比較的見通しよい
- 行列化、RS 符号、spot-check という教材性が高い
- 同種多数インスタンスの償却を考える入口として優れている

実装上の位置づけ

現時点では、Ligero は**理論比較・教育・研究試作**で読む価値が特に高い方式です。
公式・公開コードとしては libiop が Ligero を含む transparent post-quantum zkSNARK constructions を提供しています。[libiop]

実務上の注意

Ligero を production の第一候補として採るというより、

- transparent proof の祖型を学ぶ
- sublinear communication の感触を掴む
- 後続改良(Ligero++, BooLigero など)へ進む

ための足場として見るのが自然です。

Aurora: 何が今でも重要か

Aurora は、「R1CS に対する transparent succinct argument」をはっきり打ち出した重要論文です。[Aurora19]

今でも重要な理由

- R1CS-first の設計感覚が学べる
- Encoded IOP + FRI の接続が見える
- transparent で proof size を強く意識した設計点を理解できる

実装上の位置づけ

Aurora も libiop の主要構成要素の一つとして実装されています。[libiop]
その意味で、Aurora は「生きている production stack の中心」より、**FRI 系 transparent proof を学ぶための基準点**として価値が高いです。

実装で効く注意点

Aurora を実装する際に本当に重いのは、理論式の $O(n \log n)$ だけではありません。

- 低次数オラクルの管理
- FRI パラメータ
- Merkle 木
- zero-knowledge masking
- transcript の順序と domain separation

が大きな実装負荷になります。
したがって「pairing がないから実装が簡単」と考えるのは危険です。

Virgo: 2025 年以降は必ず後続研究まで読む

Virgo は、GKR と transparent zkVPD を組み合わせた非常に重要な設計です。[Virgo20]
しかし、本章で最も強く言いたい注意点も Virgo にあります。

文献上の事実

2025 年の後続研究 **Revisiting Virgo: A Study of Vulnerabilities, Limitations, and Optimizations** は、Virgo の zkVPD の設計と実装に関して、

- exploitable な security issue
- 一部マスキング手続きの欠落
- フィールド選択上の制約
- range check の負担の再評価
- 改良版 VPD / zkVPD の提案

を報告しています。[VirgoRevisit25]

これをどう読むべきか

したがって、Virgo は

- **設計思想として極めて重要**
- **しかし原論文をそのまま素朴に実装して安全とは限らない**

と理解すべきです。

この区別は研究実装で特に重要です。
「論文どおりに書いたので安全なはず」という考えは避けてください。

公開実装

Virgo の参照実装リポジトリは公開されており、論文のテストを再現しやすい土台があります。[VirgoImpl]
ただし、後続研究の指摘を踏まえずに参照実装をそのまま本番品質と見なすべきではありません。

zk-STARK: 現在の実務で最も厚いエコシステム

4 方式の中で、現在もっとも厚い production ecosystem を持つのは zk-STARK 系です。

StarkEx / Starknet

公式ドキュメントでは、StarkEx は mainnet use case を支え、Starknet は STARK 技術を基盤とする validity rollup / execution environment として説明されています。[StarkExDocs]

[StarknetDocs]
ここで重要なのは個別の業績数字よりも、**STARK が state transition の正しさを大規模に運用する技術として定着している**ことです。

Winterfell

Winterfell は open-source の STARK prover / verifier ライブラリで、公式には「research project」であり production-ready ではないと説明されています。Winterfell
この位置づけはとても健全で、研究・教育・試作の土台として使うのに向いています。

RISC Zero

RISC Zero の公式ドキュメントは、コア proof system が DEEP-ALI と FRI を実装し、その上に recursive proof system と Groth16 compression を組み合わせていることを説明しています。[Risc0Docs]
この事実から分かるのは、**現代の実務 STARK は "STARK 単体" ではなく、再帰・圧縮・VM 設計を含むスタックとして理解すべき**だということです。

Polygon Miden

Polygon Miden の公式資料は、Miden VM 上で STARK-based proof を生成し、再帰 proof を経て L1 へ提出する流れを説明しています。[MidenDocs]
これもまた、「trace / AIR / recursion / settlement」が一つの系として設計されている例です。

FRI 系の Fiat-Shamir 安全性は自明ではない

Aurora や zk-STARK で FRI を使うとき、しばしば

- transparent だから安全そう
- hash-based だから量子にも強そう

と雑に理解されがちです。
しかし後続研究は、**Fiat-Shamir 後の FRI 系 SNARK の安全性をきちんと解析する必要がある**ことを示しました。[FSFRI23]

実装での教訓

- transcript の順序を曖昧にしない

- domain separation を徹底する
- どの root / challenge / query position をどの順にハッシュするか固定する
- round ごとの識別子を省かない

「全部連結してハッシュしておけばよい」は危険です。

transparent / lightweight は「楽に実装できる」を意味しない

Ligero, Aurora, Virgo, zk-STARK はいずれも pairing を主役にしないため、lightweight cryptography と説明されることがあります。
しかしこれは、

- 数学的仮定が比較的軽い
- trusted setup を要しない
- ハッシュと有限体演算が中心

という意味であって、**エンジニアリングが簡単**という意味ではありません。

実際の実装では、

- 巨大配列のメモリ管理
- Merkle 木の構築
- field arithmetic の最適化
- blinding の入れ忘れ防止
- transcript の固定
- range check / bit decomposition

がすぐに支配的になります。

field choice は最後の微調整ではない

これは 4 方式すべてに共通する重要点です。

なぜ重要か

field choice は、

- FFT / evaluation domain の取りやすさ
- non-native arithmetic の有無
- hash の算術化コスト

- range check の重さ
- 具体セキュリティ
- recursion の組みやすさ

を全部左右します。

特に Virgo の後続研究は、「都合のよい field を前提にしていたつもりが、別の演算を入れた瞬間にコストが爆発する」問題を改めて意識させてくれます。[VirgoRevisit25]

range check と bit decomposition を甘く見ない

多くの zk 実装で、本体計算より重いのは次です。

- 比較
- 範囲制約
- ブール制約
- ハッシュ入出力の bit 化
- 小整数と大きな有限体の橋渡し

これは Ligero / Aurora / Virgo / zk-STARK のどれでも起こります。
特に「命題そのもの」は簡単でも、その前後の range check が膨大というケースは珍しくありません。

メモリは計算時間と同じくらい重要

論文では prover time に目が行きがちですが、実装では peak memory が先に詰まることがよくあります。

Ligero / Aurora / zk-STARK 側で起こりやすいこと

- 大きなオラクル表を複数持つ
- FRI の各段階で中間表を持つ
- Merkle 木の葉・内部ノードを抱える

Virgo 側で起こりやすいこと

- layer ごとの値表
- multilinear evaluation 用の中間配列
- zkVPD のコミットメント対象

そのため、

- streaming 可能か
- GPU / SIMD に乗せやすいか
- cache-friendly か
- distributed prover にしやすいか

まで含めて設計すべきです。

本書としての実務的助言

最後に、本書としての助言を短くまとめます。

Ligero

理論比較・教育・多インスタンス償却の感覚を学ぶために非常に有益。
production の第一候補というより、transparent sublinear argument の祖型として学ぶ価値が高い。

Aurora

R1CS-first の transparent proof 設計を学ぶうえで極めて重要。
proof size の魅力を強く持つが、verifier 線形性と実装のオラクル管理を忘れない。

Virgo

structured circuit で verifier を軽くしたい設計として非常に面白い。
ただし 2025 年以降は、後続研究を読まずに「そのまま実装」は避けるべき。

zk-STARK

現代の production ecosystem では最も厚い。
ただし STARK 単体ではなく、trace / AIR / FRI / recursion / compression を含むスタックとして理解するのが実態に近い。

この章のまとめ

- 理論的に重要であることと、本番採用してよいことは別である。

- Ligero / Aurora は研究・教育・比較軸として今も重要である。
- Virgo は極めて重要だが、2025 年の再検討論文を踏まえた扱いが必須である。
- zk-STARK は現在もっとも厚い実運用エコシステムを持つが、単体プロトコルではなくスタックとして理解すべきである。
- FRI 系では Fiat-Shamir 後の transcript 設計が重要であり、transparent だから自動的に安心とは言えない。

次章では、ここまでの理解を定着させるための演習問題を並べます。

第 19 章 演習問題

この章では、本書全体の理解を定着させるための演習問題を並べます。
解答を本文にすべて書くと本が重くなるので、ここでは **狙い** と**短いヒント** を付けます。

A. ゼロ知識証明の基本

問題 1

完全性・健全性・零知識性を、それぞれ 2〜4 行で説明してください。
さらに、どれか 1 つでも欠けると実務で何が困るかを述べてください。

ヒント

- 完全性: 正しい人が通る
- 健全性: 嘘つきが通らない
- 零知識性: witness を漏らさない

問題 2

「計算の正しさの証明」を relation $R(x, w) = 1$ の形で表現してください。
x に何を入れ、w に何を入れるか、自分の言葉で明示してください。

ヒント

- x: 公開入力と公開出力
- w: 秘密入力と中間値
- R: 回路や遷移規則が正しいこと

問題 3

proof と argument の違いを説明し、Ligero / Aurora / Virgo / zk-STARK を日常語で「証明」と呼ぶときに何を省略しているかを述べてください。

ヒント

無条件健全性か、計算量仮定に依るか、という違いを意識してください。

B. 有限体・多項式・符号

問題 4

$\mathbb{F}_{17}$ 上で、一次多項式 $f(X) = 3X + 2$ を評価点 $1, 2, 3, 4$ で評価しなさい。
さらに、その評価表を 1 点だけ改ざんした列が「同じ 1 次多項式の評価列ではない」ことを説明してください。

ヒント

1 次多項式は 2 点で決まります。

問題 5

「低次数多項式は剛性を持つ」とはどういう意味かを、
(a) 補間
(b) 異なる多項式の一致点数
の 2 つの観点から説明してください。

問題 6

Merkle 木がなぜ IOP / STARK / Aurora / Ligero で重要なのかを、
「大量データを先に固定する」という言葉を使って説明してください。

C. R1CS・layered circuit・GKR

問題 7

次の式を算術回路に分解してください。

$$t = (a + b)(c + 5)$$

中間変数を導入し、加算ゲートと乗算ゲートを明示してください。

問題 8

問題 7 の回路を R1CS に直してください。
変数ベクトルを自分で定め、各制約を

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$$

の形にしてください。

問題 9

layered arithmetic circuit とは何かを説明し、なぜ Virgo がその表現を重視するのかを述べてください。

ヒント

GKR は layer i の主張を layer $i + 1$ の主張へ落としていきます。

問題 10

multilinear extension を一言で説明し、Virgo においてそれが何を可能にしているかを書いてください。

D. Ligero

問題 11

Ligero で、長い witness ベクトルをわざわざ行列に並べるのはなぜでしょうか。
「行方向」と「列方向」という語を使って説明してください。

問題 12

総データ量が s で、行数 m、列数 ℓ の行列を使い、コストが大ざっぱに $m + \ell$ とみなせるとします。
$m\ell \approx s$ のもとで $m + \ell$ を最小にする直感を説明してください。

ヒント

m と ℓ を釣り合わせるとよいです。

問題 13

Ligero の random linear combination と spot-check は、それぞれ何をしているのでしょうか。
「大量の制約の圧縮」と「局所的不正の発見」という語を使って説明してください。

問題 14

Ligero の multi-instance amortization が魅力的なのはどんな場面か、具体例を 2 つ挙げてください。

E. Aurora

問題 15

Aurora が R1CS をネイティブ表現にする利点を、150〜300 字で説明してください。

問題 16

Aurora において FRI は何を担当し、何を担当しないでしょうか。
「低次数性」「制約そのもの」「オラクル整合性」という語を使って説明してください。

問題 17

Aurora の proof size が短いことと、verifier が極小であることは同じではありません。
なぜそう言えるのかを複雑性の観点から説明してください。

F. Virgo

問題 18

Virgo において GKR と zkVPD の役割を比較してください。
「layer 縮約」と「評価主張の委任」という語を使うと整理しやすいです。

問題 19

Virgo で “structured circuit” とは何かを、wiring predicate の観点から説明してください。

問題 20

Virgo の verifier complexity が魅力的に見えても、そのまま production 採用を決めてはいけない理由を説明してください。

ヒント

2025 年の後続研究で何が再検討されたかを思い出してください。

G. zk-STARK

問題 21

execution trace と AIR の違いを説明してください。
それぞれが「データ」と「仕様」のどちらに近いかも述べてください。

問題 22

フィボナッチ遷移

$$(a_{t+1}, b_{t+1}) = (b_t, a_t + b_t)$$

に対して、トレース表と境界条件を自分で書いてください。

問題 23

zk-STARK において LDE、composition polynomial、FRI、Merkle 木はそれぞれ何を担当するか、表にして整理してください。

問題 24

なぜ zk-STARK は VM 実行やロールアップと相性がよいと言えるのでしょうか。
「長い状態遷移」「trace」「AIR」という語を使って説明してください。

H. 比較問題

問題 25

Ligero / Aurora / Virgo / zk-STARK を、
「何を自然な入力表現とみなすか」
という軸で 1 行ずつ説明してください。

問題 26

4 方式を、
(a) univariate 的な見方
(b) multilinear / hypercube 的な見方
の観点から分類してください。

問題 27

4 方式すべてに共通する「ランダム線形結合」的な思想を、自分の言葉で説明してください。

問題 28

「proof size が短い」ことと「verifier が軽い」ことが独立である例として、Aurora と Virgo を比較してください。

問題 29

「STARK は一つの固定プロトコルではなくファミリとして読むべきだ」とはどういう意味か、実装スタックの観点から説明してください。

I. 設計・研究の視点

問題 30

field choice が最後の微調整ではない理由を、少なくとも次の 2 つに触れて説明してください。

- FFT / evaluation domain
- non-native arithmetic

- range check
- hash arithmeticization

問題 31

なぜ peak memory を prover time と同じくらい重視すべきなのでしょうか。
Aurora / Virgo / zk-STARK から少なくとも 2 方式を選び、どこでメモリが膨らみやすいか述べてください。

問題 32

FRI 系で transcript の domain separation が重要な理由を説明してください。

問題 33

Virgo の 2025 年後続研究から、研究実装者が得るべき教訓を 3 点書いてください。

J. 発展問題

問題 34

あなたが「transparent で、一般 relation も逐次計算も扱いたい」新しい証明系を設計するとします。
Ligero / Aurora / Virgo / zk-STARK から、それぞれ何を借りるかを 400〜800 字で提案してください。

問題 35

次の 4 つのアプリケーションについて、第一候補として自然な方式を選び、その理由を 3〜5 行で述べてください。

1. SHA-256 前像の知識証明を大量にまとめて処理したい
2. 既に R1CS コンパイラが整っている一般 relation を transparent に証明したい
3. Merkle tree を多用する structured circuit の verifier を軽くしたい
4. zkVM で長い実行 trace を証明したい

学習の目安

次に即答できるようになれば、本書の目標はかなり達成できています。

- なぜ低次数性が重要なのか。
- Ligero の平方根型通信の直感は何か。
- Aurora で FRI は何を保証するのか。
- Virgo で GKR と zkVPD はどう役割分担するのか。
- zk-STARK で execution trace と AIR はどう違うのか。
- 4 方式を「自然な入力表現」でどう使い分けるか。

紙とペンで実際に書いてみると理解が一段深まります。

特に、R1CS、small AIR、Ligero の小例、GKR の layer 縮約は、手で追ってこそ骨格が見えます。

第 20 章参考文献と学習ロードマップ

この章では、本書で中心的に参照した文献をまとめます。
本書は教科書として書いているため、本文では直観的説明を優先した箇所もあります。複雑性・定義・安全性主張の原型に立ち返るときは、必ずここに挙げる原著論文や公式資料を参照してください。

最重要文献

[Ligero17]

Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam,
Ligero: Lightweight Sublinear Arguments Without a Trusted Setup,
ACM CCS 2017, pp. 2087–2104.
DOI: `10.1145/3133956.3134104`
- PDF: https://acmccs.github.io/papers/p2087-amesA.pdf

本書での役割

Ligero の原点。平方根型通信、interleaved RS 符号、random linear combination、multi-instance amortization、SHA-256 concrete benchmark の根拠。

[Aurora19]

Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, Nicholas P. Ward,
Aurora: Transparent Succinct Arguments for R1CS,
EUROCRYPT 2019, pp. 103–128.
DOI: `10.1007/978-3-030-17653-2_4`
- DOI: https://doi.org/10.1007/978-3-030-17653-2_4 - ePrint: https://eprint.iacr.org/2018/828

本書での役割

Aurora の複雑性、R1CS 向け transparent succinct argument という位置づけ、proof size $O(\log^2 n)$ の根拠。

[Virgo20]

Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, Dawn Song,
Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof,
IEEE Symposium on Security and Privacy 2020, pp. 859–876.
DOI: `10.1109/SP40000.2020.00052`
- DOI: https://doi.org/10.1109/SP40000.2020.00052 - ePrint: https://eprint.iacr.org/2019/1482

本書での役割

Virgo の複雑性、layered circuit + GKR + zkVPD の骨格、structured circuit 前提、代表 benchmark の根拠。

[STARK18]

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev,
Scalable, Transparent, and Post-Quantum Secure Computational Integrity,
IACR ePrint 2018/046.
- ePrint: https://eprint.iacr.org/2018/046 - StarkWare mirror: https://starkware.co/wp-content/uploads/2022/05/STARK-paper.pdf

本書での役割

STARK 系の原点。execution trace, AIR, ALI, FRI, transparency, scalability の骨格。

背景文献

[IOP16]

Eli Ben-Sasson, Alessandro Chiesa, Nicholas Spooner,
Interactive Oracle Proofs,
TCC 2016-B, pp. 31–60.
DOI: `10.1007/978-3-662-53644-5_2`

- DOI: https://doi.org/10.1007/978-3-662-53644-5_2 - ePrint: https://eprint.iacr.org/2016/116

本書での役割

Aurora・STARK 系で前提となる IOP モデルの基礎。

[FRI18]

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev,
Fast Reed-Solomon Interactive Oracle Proofs of Proximity,
ICALP 2018, LIPIcs 107, 14:1–14:17.
DOI: `10.4230/LIPIcs.ICALP.2018.14`
- DOI: https://doi.org/10.4230/LIPIcs.ICALP.2018.14 - Dagstuhl: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

本書での役割

FRI の原典。Aurora・zk-STARK の低次数近接性検査を理解するための最重要文献。

[GKR08]

Shafi Goldwasser, Yael Tauman Kalai, Guy N. Rothblum,
Delegating Computation: Interactive Proofs for Muggles,
STOC 2008, pp. 113–122.
DOI: `10.1145/1374376.1374396`
- DOI: https://doi.org/10.1145/1374376.1374396 - PDF copy: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/2008-DelegatingComputation.pdf

本書での役割

Virgo が依拠する GKR の原点。layer-by-layer 縮約の考え方を与える。

後続研究・注意喚起

[LigeroPP20]

Ariel Gabizon, Zachary J. Williamson, Oana Ciobotaru,
Ligero++: Efficient Zero-Knowledge Arguments with Light Verification,
IACR ePrint 2020/142.

- ePrint: https://eprint.iacr.org/2020/142

本書での役割

Ligero 系の後続改良を見る入口。Ligero を学んだあとの自然な次の一歩。

[VirgoRevisit25]

Changchang Ding, Yan Huang,
Revisiting Virgo: A Study of Vulnerabilities, Limitations, and Optimizations,
Cybersecurity 8, Article 118, 2025.
DOI: `10.1186/s42400-025-00450-w`
- DOI: https://doi.org/10.1186/s42400-025-00450-w

本書での役割

Virgo の zkVPD 設計・実装に関する脆弱性、制約、改良点を理解するための必読文献。

[FSFRI23]

Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, Michał Zając,
Fiat-Shamir Security of FRI and Related SNARKs,
ASIACRYPT 2023, pp. 3–40.
DOI: `10.1007/978-981-99-8724-5_1`
- DOI: https://doi.org/10.1007/978-981-99-8724-5_1 - ePrint: https://eprint.iacr.org/2023/1071

本書での役割

FRI 系プロトコルを Fiat-Shamir 後にどう読むか、という安全性理解を更新するための文献。

[Fractal20]

Alessandro Chiesa, Dev Ojha, Nicholas Spooner,
Fractal: Post-Quantum and Transparent Recursive Proofs from Holography,
EUROCRYPT 2020, pp. 769–793.
DOI: `10.1007/978-3-030-45721-1_27`
- DOI: https://doi.org/10.1007/978-3-030-45721-1_27 - ePrint: https://eprint.iacr.org/2019/1076

本書での役割

Aurora / STARK 周辺の transparent recursive proof の流れを見るための補助文献。

[Spartan20]

Srinath Setty,
Spartan: Efficient and General-purpose zkSNARKs without Trusted Setup,
CRYPTO 2020, pp. 704–737.
DOI: `10.1007/978-3-030-56877-1_25`
- DOI: https://doi.org/10.1007/978-3-030-56877-1_25 - ePrint: https://eprint.iacr.org/2019/550

本書での役割

transparent / no trusted setup / general-purpose という比較軸を広げるための文献。

実装・公式資料(2026-04-05 閲覧)

[libiop]

scipr-lab,
libiop (GitHub repository).
- https://github.com/scipr-lab/libiop

本書での役割

Ligero, Aurora, Fractal などを含む transparent post-quantum zkSNARK constructions の参照実装群。

[VirgoImpl]

sunblaze-ucb,
Virgo (GitHub repository).
- https://github.com/sunblaze-ucb/Virgo

本書での役割

Virgo の参照実装。論文のテストを追うための入口。

Winterfell

facebook/winterfell,

Winterfell (GitHub repository).

- https://github.com/facebook/winterfell

本書での役割

任意計算向け STARK prover / verifier の研究ライブラリ。production-ready ではないことが明記されている点も重要。

[StarkDocs]

StarkWare,

What is STARK?

- https://starkware.co/stark/

本書での役割

STARK の high-level な性質(polylog verification resources, transparency, post-quantum の位置づけ)を簡潔に確認する入口。

[StarkExDocs]

StarkWare,

StarkEx Documentation.

- https://docs.starkware.co/starkex/index.html

本書での役割

STARK 系技術の実運用を理解するための公式資料。

[StarknetDocs]

Starknet Foundation / Starknet documentation,

Introduction to Starknet's protocol.

- https://docs.starknet.io/learn/protocol/intro

本書での役割

STARK を用いた validity rollup / execution environment の公式説明。

[Risc0Docs]

RISC Zero,
How does RISC Zero work? / Understanding RISC Zero.
- https://dev.risczero.com/api/2.1/zkvm/understanding-risc0 - https://dev.risczero.com/api/2.2/zkvm/architecture/proof-system

本書での役割

現代の STARK 系 zkVM が、DEEP-ALI・FRI・recursive proof・外側の圧縮 proof をどう組み合わせるかを知るための公式資料。

[MidenDocs]

Polygon Miden,
Miden / FAQ / protocol overview.
- https://0xmiden.github.io/miden-docs/ - https://docs.polygon.technology/miden/faq/ - https://github.com/0xPolygonMiden/miden-base

本書での役割

STARK-based VM / rollup としての Miden の設計感を追うための資料。

どの順に読むとよいか

ルート A: まず全体像

1. 本書の第 4 章・第 16 章・第 17 章を読む

2. [Ligero17], [Aurora19], [Virgo20], [STARK18] の abstract と introduction を読む

3. 気になった方式の詳細章に戻る

ルート B: FRI 系を深く知る

1. [Aurora19]
2. [FRI18]
3. [STARK18]
4. [FSFRI23]
5. 必要なら [Fractal20]

ルート C: GKR / Virgo 系を深く知る

1. [GKR08]
2. [Virgo20]
3. [VirgoRevisit25]
4. 必要なら [Spartan20]

ルート D: 実装・エコシステムを見る

1. [libiop], Winterfell, [VirgoImpl]
2. [StarkExDocs], [StarknetDocs]
3. [Risc0Docs], [MidenDocs]

原論文を読むときのコツ

Ligero

最初に theorem を完全に追うより、

- なぜ行列化するのか
- なぜ平方根型通信が出るのか
- random linear combination と spot-check の役割は何か

を追うと読みやすいです。

Aurora

R1CS から低次数オラクルへ行く設計線を見失わないことが重要です。
FRI は「最後の添え物」ではなく、オラクルが本当に低次数に由来することを支える核心です。

Virgo

GKR と zkVPD を混同しないこと。
GKR が証明の論理、zkVPD が評価主張の外部化です。
2025 年以降は [VirgoRevisit25] を併読してください。

zk-STARK

trace / AIR / composition / FRI / Merkle の役割分担を崩さないこと。
「STARK はすごい」という抽象印象だけで読むと、各部品の必要性が分からなくなります。

この章のまとめ

- 4 方式の核となる一次文献は [Ligero17], [Aurora19], [Virgo20], [STARK18] である。
- FRI, IOP, GKR はそれぞれ [FRI18], [IOP16], [GKR08] が基礎になる。
- Virgo と FRI については、後続研究 [VirgoRevisit25], [FSFRI23] を必ず読むべきである。
- 実務理解には論文だけでなく、公式実装・公式ドキュメントも重要である。

最後に、用語が混ざったら付録の用語集へ戻ってください。

付録 用語集

本書で使った重要語を、**短く引ける辞書**としてまとめます。
厳密な定義そのものではなく、学習の足場としての説明を優先します。

AIR (Algebraic Intermediate Representation)

計算を有限体上の**状態遷移制約**として表したもの。
zk-STARK で「何が正しい計算か」を定義する仕様書に相当する。

ALI (Algebraic Linking IOP)

多数の AIR 制約や整合性条件を、ランダム係数で少数の代数的主張へまとめる考え方。
現代の解説では composition polynomial という言葉に吸収されることも多い。

argument

健全性が計算量仮定に依存する証明系。
実用的な zk 証明の多くは strict な意味では proof ではなく argument である。

batching / amortization

多数の同型インスタンスをまとめて証明し、1 件あたりの通信や検証コストを下げること。
Ligero では特に重要な観点。

boundary constraint

execution trace の最初や最後など、特定位置の値を固定する制約。
AIR の一部として現れる。

codeword (符号語)

ある誤り訂正符号に属する長い列。
Reed-Solomon 符号では、低次数多項式の評価列が符号語になる。

commitment

値を先に固定しつつ、中身は後で必要箇所だけ開示できる仕組み。
Merkle 木は大量データへのコミットメントとして使われる。

completeness (完全性)

statement が真で、正直な prover が正しい witness を持っているなら verifier が受理する性質。

composition polynomial

多数の制約をランダム線形結合で 1 本または少数の多項式へ圧縮したもの。
正しい計算なら低次数になるよう設計されることが多い。

constraint system

計算の正しさを、大量の局所等式や不等式として表したもの。
R1CS は代表的な constraint system。

DEEP-FRI / DEEP-ALI

STARK 系で soundness を改善するために導入された後続技法。
FRI / ALI の思想を保ちながら concrete security を改善する方向として理解するとよい。

domain separation

ハッシュ transcript で、ラウンド番号・メッセージ種別・用途を明確に分けること。
Fiat-Shamir 実装では非常に重要。

execution trace

計算を時刻順の表として並べたもの。
行が時間、列がレジスタや状態変数。zk-STARK で主役になる。

evaluation domain

多項式を評価する点集合。
FRI や Reed-Solomon 符号では、どの領域で評価するかが重要。

Fiat-Shamir 変換

公開コイン対話型プロトコルを、ハッシュ関数で challenge を生成して非対話化する手法。

FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity)

長い評価表が低次数多項式の評価列に近いかを検査する IOPP。
Aurora と zk-STARK の中核部品。

GKR

Goldwasser-Kalai-Rothblum プロトコル。
layered circuit の正しさを、上層から下層へ主張を縮約しながら検査する。

hash-based / lightweight cryptography

pairing のような重い公開鍵部品ではなく、主にハッシュ関数と有限体演算を中心に設計する方向。
「実装が簡単」という意味ではない。

hypercube

$\{0,1\}^\ell$ からなる離散点集合。
Virgo では layer の値表をこの上の関数として見る。

interleaved Reed-Solomon code

複数本の Reed-Solomon 符号語を縦に積んだもの。
Ligero の主要代数構造。

interactive proof / interactive argument

prover と verifier が複数回メッセージをやり取りする証明系。
多くの方式はまず対話型として理解すると構造が追いやすい。

IOP (Interactive Oracle Proof)

大きなオラクルへのランダムアクセスと対話を組み合わせた証明モデル。
Aurora や STARK を理解する基盤。

IOPP (Interactive Oracle Proof of Proximity)

長い列が、ある符号や集合に近いことを検査する IOP。
FRI は Reed-Solomon 符号に対する代表的 IOPP。

layered arithmetic circuit

回路を層ごとに並べ、各層が次の層からのみ入力を受けるよう整えた回路。
Virgo / GKR で重要。

LDE (Low-Degree Extension)

小さい領域上の列を多項式として補間し、より大きな領域で再評価したもの。
zk-STARK や Aurora で冗長性を増やすために使う。

low-degree test

ある関数表が低次数多項式に由来するかを検査する仕組み。
FRI はその代表例。

multilinear extension

Boolean hypercube 上の関数を、全空間へ拡張した多変数多項式。
Virgo / GKR で中心的。

Merkle tree

長い配列に対する root hash を与える木型コミットメント。
任意位置の値を短い認証パスで開示できる。

multi-instance amortization

同一回路に対する多数の証明要求をまとめ、1 件あたりのコストを下げること。
Ligero の代表的な強みの一つ。

non-native arithmetic

ある証明系の基礎有限体とは別の体や整数型の演算を埋め込んで扱うこと。
しばしば非常にコストが高い。

oracle

巨大な表や配列にランダムアクセスできるブラックボックス。
実装では Merkle 木にコミットされた配列として実現されることが多い。

PCP

証明全体を読まずに、少数箇所の検査だけで高確率に正誤判定できるという考え方。
現代の IOP / STARK / Ligero の背景理論。

plausibly post-quantum secure

既知の主要仮定が量子計算に対して比較的壊れにくいと期待される、という意味。
厳密な保証を主張しているとは限らない。

proof

健全性が計算量仮定に依らない、より強い証明概念。
実用系では strict な意味で proof でない場合が多い。

prover

witness を持ち、statement が真であることを示そうとする側。

public-coin

verifier のメッセージが基本的にランダム文字列だけであるタイプの対話型証明。
Fiat-Shamir で非対話化しやすい。

range check

値がある範囲に入っていることを保証する制約。
理論上の主計算より重くなりやすい。

Reed-Solomon code

低次数多項式を多数点で評価して得られる符号。
Ligero, Aurora, zk-STARK に何らかの形で現れる。

R1CS (Rank-1 Constraint Satisfaction)

各制約を

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$$

の形にそろえた制約系。Aurora の自然言語。

recursion

ある proof system の証明を、さらに別の proof で包むこと。
現代の STARK 系実装では非常に重要。

soundness (健全性)

偽の statement を不正 prover が高確率で受理させられない性質。

spot-check

巨大な表全体を読まずに、少数位置だけを開いて整合性を検査すること。
Ligero の感覚を理解する鍵。

statement

公開される主張。
たとえば「このハッシュの前像を知っている」「この状態更新は正しい」など。

structured circuit

規則的な wiring を持ち、random point での wiring predicate 評価が効率よくできる回路。
Virgo の verifier complexity を引き出す前提として重要。

sumcheck

多変数多項式の hypercube 上の総和主張を、ラウンドごとに 1 変数多項式へ分解し、最後にランダム点評価へ落とす対話型プロトコル。
GKR と深く結びつく。

trace representation

計算を回路ではなく時系列の状態列として表す見方。
zk-STARK が特に得意とする表現。

transcript

対話全体のメッセージ列。
Fiat-Shamir ではこの transcript をハッシュして challenge を導出する。

transparent

trusted setup や秘密 trapdoor を必要としない性質。
Ligero, Aurora, Virgo, zk-STARK はいずれもこの方向に属する。

univariate viewpoint

巨大な表を「1 変数低次数多項式の評価表」として見る見方。
Ligero, Aurora, zk-STARK で強い。

verifier

proof を検査し、statement を受理するか判断する側。

witness

statement を成り立たせる秘密情報。
ゼロ知識証明では、これを公開せずに条件成立だけを示す。

wiring predicate

あるゲートがどの子ゲートと接続しているかを表す述語。
Virgo / GKR で structured circuit を支える重要部品。

zero-knowledge (零知識)

verifier が、statement が真であること以外の本質的情報を witness について得ない性質。
厳密には simulator の存在で定式化される。

zkVPD (zero-knowledge verifiable polynomial delegation)

多項式の点評価主張を、零知識を保ちながら verifier が短く確かめられるようにする仕組み。
Virgo の中核部品。